

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

NGUYỄN THỊ VÂN

NGHIÊN CỨU VÀ XÂY DỰNG BỘ TEST CASE CÙNG GIẢI PHÁP KIỂM
THỬ TỰ ĐỘNG CHO CHATBOT NHÀ XE VỮ HÁN

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC
NGÀNH KỸ THUẬT PHẦN MỀM

THÁI NGUYÊN, NĂM 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN

TỐT NGHIỆP ĐẠI HỌC

NGÀNH KỸ THUẬT PHẦN MỀM

Đề tài:

**NGHIÊN CỨU VÀ XÂY DỰNG BỘ TEST CASE CÙNG GIẢI PHÁP KIỂM
THỬ TỰ ĐỘNG CHO CHATBOT NHÀ XE VŨ HÁN**

Sinh viên thực hiện: NGUYỄN THỊ VÂN

Lớp: KTPM K21A

Giảng viên hướng dẫn: THS. NGUYỄN THU PHƯƠNG

THÁI NGUYÊN, NĂM 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành và sự tri ân sâu sắc đối với các thầy cô của trường Đại học Công nghệ thông tin và truyền thông Thái Nguyên, đặc biệt là các thầy cô giáo trong khoa Công nghệ thông tin, những người đã tận tình truyền đạt kiến thức và kinh nghiệm quý báu cho em trong suốt những năm học qua. Chính nền tảng tri thức đó đã giúp em có đủ kỹ năng để thực hiện đề án này. Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc tới cô THS. Nguyễn Thu Phương đã tận tâm và nhiệt tình hướng dẫn, giúp đỡ em chỉnh sửa những thiếu sót trong quá trình làm đề án.

Em cũng xin chân thành cảm ơn gia đình, bạn bè đã luôn ở bên cạnh ủng hộ, khích lệ tinh thần em để em có thể hoàn thành tốt nhất nhiệm vụ nghiên cứu này.

Dù đã cố gắng nỗ lực hết mình, nhưng do kiến thức và kinh nghiệm còn hạn chế, đề án chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp, chỉ bảo từ các thầy, cô để kiến thức của em ngày càng hoàn thiện hơn.

LỜI CAM ĐOAN

Em xin cam đoan đồ án tốt nghiệp với đề tài “*Nghiên cứu và xây dựng bộ test case cùng giải pháp kiểm thử tự động cho chatbot nhà xe Vũ Hán*” là công trình nghiên cứu độc lập của cá nhân em, không sao chép từ đồ án liên quan nào. Các số liệu, kết quả thực nghiệm được trình bày trong đồ án là hoàn toàn trung thực, khách quan và do em trực tiếp thực hiện. Mọi sự tham khảo sử dụng trong đồ án đều được trích dẫn các nguồn tài liệu trong báo cáo và danh mục tài liệu tham khảo. Nếu có bất kỳ sự gian lận nào, em xin chịu hoàn toàn trách nhiệm trước nhà trường và pháp luật.

Thái Nguyên, ngày tháng năm 2026

Sinh viên

Nguyễn Thị Vân

MỤC LỤC

LỜI CẢM ƠN.....	2
LỜI CAM ĐOAN	3
MỤC LỤC	4
DANH MỤC BẢNG	7
DANH MỤC HÌNH ẢNH.....	8
LỜI NÓI ĐẦU.....	9
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	11
1.1 Khái niệm về kiểm thử phần mềm	11
1.2 Phân loại và các kỹ thuật kiểm thử	11
1.2.1 Các chiến lược kiểm thử	11
1.2.2 Các phương pháp kiểm thử	12
1.2.3 Các kỹ thuật kiểm thử	13
1.2.4 Các cấp độ kiểm thử phần mềm	15
1.2.5 Các nguyên tắc trong kiểm thử phần mềm.....	17
1.2.6 Quy trình kiểm thử phần mềm	19
1.3 Tìm hiểu về kiểm thử tự động.....	19
1.3.1 Khái niệm về kiểm thử tự động.....	19
1.3.2 Mục đích của kiểm thử tự động	20
1.3.3 So sánh kiểm thử tự động và kiểm thử thủ công.....	20
1.4 Tìm hiểu về Apache JMeter	21
1.4.1 Tổng quan về công cụ	21
1.4.2 Ưu nhược điểm.....	21
1.4.3 Đặc trưng và cách thức hoạt động của Jmeter	22
1.5 Tìm hiểu về Playwright.....	23
1.5.1 Tổng quan về công cụ	23
1.5.2 Ưu nhược điểm của Playwright.....	23
1.5.3 Tính năng nổi bật của Playwright	24
1.6 Tìm hiểu về OWASP ZAP.....	26
1.6.1 Tổng quan về OWASP ZAP	26
1.6.2 Tính năng nổi bật của OWASP ZAP	26
1.7 Tìm hiểu công cụ SonarQube	27

1.7.1	Tổng quan về SonarQube	27
1.7.2	Tính năng nổi bật.....	27
1.7.3	Ưu điểm.....	28
1.8	Đặc thù kiểm thử chatbot và hệ thống hội thoại	28
1.8.1	Kiểm thử nhận diện ý định (Intent Recognition Testing)	29
1.8.2	Kiểm thử trích xuất thực thể (Entity Extraction Testing)	29
1.8.3	Kiểm thử luồng hội thoại nhiều bước (Multi-turn Dialog Management Testing).....	29
1.8.4	Kiểm thử tính bền bỉ với ngôn ngữ tự nhiên (Robustness Testing)	30
1.8.5	Kiểm thử phản hồi sai lệch tri thức (Hallucination Testing)	30
1.8.6	Kiểm thử tính nhất quán của phản hồi (Response Consistency Testing).31	
1.8.7	Kiểm thử Fallback và Chuyển giao nhân viên hỗ trợ (Fallback & Human Escalation)	31
CHƯƠNG 2: LẬP KẾ HOẠCH VÀ THIẾT KẾ TÀI LIỆU KIỂM THỬ		32
2.1	Phân tích yêu cầu và đối tượng kiểm thử.....	32
2.1.1	Tổng quan hệ thống chatbot nhà xe Vũ Hán.....	32
2.1.2	Đối tượng kiểm thử	32
2.1.3	Phân tích yêu cầu chức năng.....	33
2.1.4	Phân tích yêu cầu phi chức năng.....	34
2.1.5	Kiến trúc tổng thể hệ thống chatbot	35
2.1.6	Phân tích API phục vụ kiểm thử	35
2.2	Lập kế hoạch kiểm thử.....	36
2.2.1	Mục tiêu kiểm thử	36
2.2.2	Phạm vi kiểm thử	37
2.2.3	Nguồn lực và môi trường kiểm thử.....	38
2.3	Chiến lược kiểm thử.....	39
2.3.1	Các giai đoạn thực hiện kiểm thử.....	39
2.3.2	Chiến lược kiểm thử dựa trên rủi ro (Risk-Based Testing).....	40
2.3.3	Tiêu chí và điều kiện kiểm thử.....	41
2.3.4	Chiến lược quản lý lỗi và dữ liệu kiểm thử.....	42
2.4	Thiết kế kịch bản và bộ Test Case chi tiết	43
2.4.1	Nhóm chức năng nhận diện Intent	43
2.4.2	Nhóm chức năng tra cứu giá vé	47

2.4.3	Nhóm chức năng tra cứu lịch chạy.....	49
2.4.4	Nhóm chức năng đặt vé.....	52
2.4.5	Kiểm tra alias địa điểm	56
2.4.6	Câu hỏi thường gặp	57
2.4.7	Nhóm chức năng gửi hàng	60
2.4.8	Nhóm chức năng chuyên chăm sóc khách hàng	62
2.5	Phương pháp đo lường độ bao phủ kiểm thử (Test Coverage).....	64
2.5.1	Bảng đối soát độ bao phủ theo chức năng.....	64
2.5.2	Chỉ số đo lường định lượng.....	65
CHƯƠNG 3: THỰC HIỆN VÀ BÁO CÁO KẾT QUẢ KIỂM THỬ		67
3.1	Lý do lựa chọn giải pháp và công cụ kiểm thử.....	67
3.2	Kiểm thử chức năng tự động với Playwright.....	67
3.2.1	Cấu hình Framework kiểm thử tự động Playwright.....	67
3.2.2	Kiến trúc thư mục theo mô hình POM.....	68
3.2.3	Lớp đối tượng giao diện.....	69
3.2.4	Kịch bản thực thi kiểm thử tự động dữ liệu tập trung.....	71
3.2.5	Báo cáo kết quả thực thi.....	73
3.3	Kiểm thử hiệu năng với Apache Jmeter.....	75
3.3.1	Cấu hình kịch bản kiểm thử tải (Load Testing Setup)	75
3.3.2	Kết quả kiểm thử hiệu năng	77
3.4	Kiểm thử bảo mật với OWASP ZAP.....	78
3.4.1	Quy trình quét tự động (Automated Scanning).....	78
3.4.2	Kiểm thử thủ công (Manual Explore)	80
3.5	Phân tích mã nguồn với SonarQube	81
3.5.1	Phân loại chi tiết các chỉ số chất lượng (Metrics Classification).....	81
3.5.2	Báo cáo Độ phức tạp mã nguồn (Cyclomatic Complexity)	82
3.6	Tự động hóa quy trình Kiểm thử (CI/CD Pipeline).....	84
3.7	Quản lý lỗi bằng Jira.....	87
3.8	Báo cáo tổng hợp và đánh giá chất lượng phần mềm	89
KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN.....		91
TÀI LIỆU THAM KHẢO		92
PHỤ LỤC		94

DANH MỤC BẢNG

Bảng 1.1 So sánh kiểm thử thủ công và kiểm thử tự động	20
Bảng 2.1 Yêu cầu chức năng hệ thống	33
Bảng 2.2 Yêu cầu phi chức năng	34
Bảng 2.3 Ma trận phân tích rủi ro	40
Bảng 2.4 Test Case nhận diện Intent	43
Bảng 2.5 Test Case tra cứu giá vé	47
Bảng 2.6 Test Case tra cứu lịch chạy	50
Bảng 2.7 Test Case đặt vé	52
Bảng 2.8 Test Case Alias địa điểm	57
Bảng 2.9 Test Case FAQ	58
Bảng 2.10 Test Case gửi hàng hóa	61
Bảng 2.11 Test Case chuyển nhân viên hỗ trợ	63
Bảng 2.12 Bảng đối soát độ bao phủ theo chức năng	64
Bảng 3.1 Thống kê trạng thái lỗi trên Jira	88
Bảng 3.2 Tổng hợp kết quả đánh giá chất lượng phần mềm	89

DANH MỤC HÌNH ẢNH

Hình 1.1: Các cấp độ kiểm thử phần mềm	15
Hình 1.2: Quy trình kiểm thử phần mềm	19
Hình 3.1: Thiết lập tham số kiểm thử trong Playwright.....	68
Hình 3.2: Cấu trúc POM của thư mục dự án	69
Hình 3.3: Định nghĩa thành phần giao diện.....	70
Hình 3.4: Đóng gói hành vi người dùng.....	70
Hình 3.5: Đọc dữ liệu và khởi tạo vòng lặp	71
Hình 3.6: Khung định nghĩa của ca kiểm thử.....	72
Hình 3.7: Giao diện báo cáo tổng quan của Playwright	74
Hình 3.8: Giao diện tab Test Steps	74
Hình 3.9: Giao diện tab Error	75
Hình 3.10: Giao diện cấu hình Thread Group	75
Hình 3.11: Cấu hình HTTP Request Defaults	76
Hình 3.12: Cấu hình CSV Data Set Config.....	76
Hình 3.13: Giao diện HTTP Request	77
Hình 3.14: Báo cáo APDEX.....	77
Hình 3.15: Thống kê kết quả kiểm thử hiệu năng	78
Hình 3.16: Báo cáo Alerts của OWASP ZAP	80
Hình 3.17: Kiểm thử thủ công với Fuzz	81
Hình 3.18: Kết quả phân tích mã nguồn với SonarQube	82
Hình 3.19: Báo cáo Cyclomatic Complexity.....	83
Hình 3.20: Biểu đồ xu hướng chất lượng	83
Hình 3.21 Mô hình thư mục .github/workflows/	84
Hình 3.22: Ví dụ về file yml của công cụ Playwright	85
Hình 3.23: Giao diện GitHub Actions	86
Hình 3.24 Artifacts của Playwright	86
Hình 3.25 Artifacts của JMeter	87
Hình 3.26 Artifacts của ZAP	87
Hình 3.27 Bảng quản lý lỗi của dự án	88

LỜI NÓI ĐẦU

1. Lý do chọn đề tài

Trong kỷ nguyên công nghệ số hóa mạnh mẽ hiện nay, chatbot đang trở thành công cụ quan trọng và phổ biến trong việc hỗ trợ khách hàng của các doanh nghiệp nhằm nâng cao chất lượng dịch vụ khách hàng và tối ưu chi phí vận hành. Đặc biệt trong ngành vận tải hành khách, nơi nhu cầu tư vấn lịch trình, giá vé, đặt chỗ và hỗ trợ gửi hàng diễn ra liên tục 24/7, chatbot đóng vai trò quan trọng trong việc giảm tải cho tổng đài và tăng tốc độ phản hồi cho khách hàng.

Nhà xe Vũ Hán là doanh nghiệp phát triển mạnh tại khu vực miền Bắc với hệ thống tuyến đường rộng lớn. Việc triển khai chatbot hỗ trợ khách hàng là bước đi cần thiết để nâng cao trải nghiệm người dùng. Tuy nhiên, chatbot là hệ thống tương tác phức tạp, dễ phát sinh lỗi ở nhiều kịch bản thực tế (tin nhắn không rõ ràng, yêu cầu đổi/ hủy vé, khiếu nại ...).

Kiểm thử thủ công truyền thống cho thấy nhiều hạn chế: tốn thời gian, chi phí cao, khó đảm bảo tính lặp lại và độ bao phủ toàn diện. Do đó, việc nghiên cứu và xây dựng bộ test case kết hợp giải pháp kiểm thử tự động cho chatbot Nhà xe Vũ Hán là hết sức cần thiết, góp phần đảm bảo chất lượng hệ thống trước khi đưa vào vận hành chính thức và trong giai đoạn bảo trì sau này.

2. Mục đích của đề tài

Đề tài được thực hiện nhằm mục đích tìm hiểu về kiểm thử phần mềm, kiểm thử tự động phần mềm, đặc biệt là kiểm thử tự động cho chatbot. Xây dựng bộ test case toàn diện, có hệ thống cho chatbot Nhà xe Vũ Hán. Đề xuất và triển khai giải pháp kiểm thử tự động sử dụng công cụ kiểm thử tự động. Cung cấp tài liệu tham khảo và quy trình kiểm thử có thể tái sử dụng cho các chatbot tương tự.

3. Phạm vi và cấu trúc của đề tài của đề tài

Đề tài có phạm vi như sau:

- Đối tượng nghiên cứu: Hệ thống chatbot hỗ trợ khách hàng của nhà xe Vũ Hán trên nền tảng web.
- Kịch bản kiểm thử: Tập trung vào các nghiệp vụ cốt lõi bao gồm: Tra cứu lịch các tuyến, báo giá vé, quy định gửi hàng và quy trình đặt vé.
- Về giới hạn: Đề tài thực hiện kiểm thử trên môi trường giả lập người dùng cuối (End-to-End) thông qua trình duyệt web, tập trung vào tính chính xác của phản hồi tri thức.

Đồ án được chia thành 3 chương chính:

- Chương 1: Cơ sở lý thuyết
- Chương 2: Lập kế hoạch và thiết kế tài liệu kiểm thử
- Chương 3: Thực hiện và báo cáo kết quả kiểm thử

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1 Khái niệm về kiểm thử phần mềm

Khái niệm kiểm thử phần mềm được định nghĩa dưới nhiều góc độ khác nhau. Theo Bảng chú giải thuật ngữ chuẩn IEEE, kiểm thử phần mềm là quá trình khảo sát hệ thống hoặc các thành phần của hệ thống trong những điều kiện xác định, thông qua việc quan sát, ghi lại kết quả và đánh giá các khía cạnh liên quan. Ở một khía cạnh khác, Myers (1979) trong tác phẩm *The Art of Software Testing* lại nhấn mạnh kiểm thử là quá trình thực thi chương trình với mục đích phát hiện lỗi. Bên cạnh đó, theo định nghĩa từ Wikipedia, kiểm thử phần mềm được hiểu là hoạt động khảo sát sản phẩm hoặc dịch vụ trong môi trường triển khai thực tế nhằm cung cấp thông tin về chất lượng cho các bên liên quan, từ đó tối ưu hóa hiệu quả vận hành của phần mềm trong đa dạng các ngành nghề.

Bản chất của kiểm thử phần mềm là chuỗi các hoạt động được chuẩn hóa nhằm xác thực mã nguồn hoạt động chính xác theo đúng mục tiêu thiết kế, đồng thời ngăn chặn các hành vi ngoài ý muốn. Giai đoạn này đóng vai trò quan trọng trong chu kỳ phát triển hệ thống, cung cấp cơ sở thực tế giúp cả đội ngũ phát triển lẫn khách hàng đánh giá chính xác mức độ hoàn thiện của sản phẩm so với yêu cầu ban đầu.

1.2 Phân loại và các kỹ thuật kiểm thử

Kiểm thử phần mềm có thể được phân loại dựa trên nhiều yếu tố khác nhau như chiến lược kiểm thử, phương pháp kiểm thử và kỹ thuật kiểm thử.

- Dựa vào Chiến lược kiểm thử ta có thể phân chia kiểm thử thành: Kiểm thử thủ công và Kiểm thử tự động.
- Dựa vào Phương pháp kiểm thử ta có thể chia thành: Kiểm thử tĩnh và Kiểm thử động.
- Dựa vào Kỹ thuật kiểm thử ta có thể chia thành: Kiểm thử hộp đen, Kiểm thử hộp trắng và Kiểm thử hộp xám.

Ngoài ra, kiểm thử phần mềm còn được phân thành nhiều loại khác như kiểm thử chức năng, kiểm thử phi chức năng, kiểm thử cấu trúc phần mềm, kiểm thử xác nhận và kiểm thử hồi quy nhằm đảm bảo hệ thống hoạt động ổn định và đúng yêu cầu.

1.2.1 Các chiến lược kiểm thử

- Kiểm thử thủ công là phương pháp mà tester thực hiện toàn bộ quá trình kiểm thử bằng tay, từ việc xây dựng test case đến thực hiện các bước kiểm tra trên hệ

thống. Tester sẽ nhập dữ liệu đầu vào, thực hiện các thao tác như nhấn nút, chuyển trang hoặc nhập thông tin, sau đó quan sát kết quả thực tế và so sánh với kết quả mong đợi trong test case. Cuối cùng, tester ghi nhận kết quả kiểm thử và các lỗi phát hiện được. Hiện nay, kiểm thử thủ công vẫn được sử dụng phổ biến trong nhiều công ty và dự án phần mềm nhờ tính linh hoạt và dễ triển khai.

- Kiểm thử tự động là phương pháp sử dụng công cụ hoặc chương trình để tự động thực hiện các bước kiểm thử với ít sự can thiệp từ con người. Phương pháp này giúp giảm các thao tác lặp lại, tiết kiệm thời gian và nâng cao hiệu quả kiểm thử. Các công cụ kiểm thử tự động có thể đọc dữ liệu từ các tệp bên ngoài như Excel hoặc CSV, tự động nhập dữ liệu vào hệ thống, so sánh kết quả thực tế với kết quả mong đợi và xuất báo cáo sau khi kiểm thử hoàn tất. Kiểm thử tự động đặc biệt phù hợp với các kịch bản cần thực hiện nhiều lần hoặc kiểm thử trên quy mô lớn.

1.2.2 Các phương pháp kiểm thử

Có 2 phương thức kiểm thử chính là: Kiểm thử tĩnh và Kiểm thử động.

1.2.2.1 Kiểm thử tĩnh – Static testing

Theo tiêu chuẩn quốc tế về kiểm thử phần mềm, kiểm thử tĩnh là phương pháp đánh giá chất lượng sản phẩm thông qua việc rà soát, phân tích tài liệu đặc tả yêu cầu, tài liệu thiết kế hệ thống và cấu trúc mã nguồn mà không cần thực thi chương trình. Ở phương thức thủ công, các kỹ sư phần mềm sẽ tiến hành kiểm tra logic một cách chi tiết trên các bản phác thảo thiết kế nhằm phát hiện và loại bỏ các sai sót hệ thống ngay từ giai đoạn sơ khởi của chu trình phát triển.

Mặt khác, quy trình kiểm thử tĩnh hiện đại cũng được tối ưu hóa thông qua các công cụ tự động. Trình biên dịch (Compiler) hoặc trình thông dịch (Interpreter) sẽ đóng vai trò phân tích mã nguồn, tự động xác thực tính hợp lệ của cú pháp và kiểm tra các ràng buộc logic kỹ thuật trước khi mã nguồn được chuyển sang giai đoạn kiểm thử động.

1.2.2.2 Kiểm thử động – Dynamic testing

Trái ngược với kiểm thử tĩnh, kiểm thử động là phương pháp đánh giá chất lượng phần mềm thông qua việc thực thi trực tiếp mã nguồn trên máy tính để phân tích hành vi và trạng thái vận hành của hệ thống. Tiến trình này được triển khai dựa trên các kịch bản kiểm thử (test cases) cụ thể nhằm theo dõi phản ứng và sự biến đổi dữ liệu của hệ thống dưới tác động của các tham số đầu vào thay đổi theo thời gian. Đặc trưng chính của kiểm thử động là phần mềm bắt buộc phải được biên dịch thành công và đưa vào

trạng thái chạy thực tế. Quá trình kiểm tra được thực hiện bằng cách truyền tải các tập dữ liệu đầu vào, sau đó tiến hành đối chiếu kết quả đầu ra thực tế với các giá trị kỳ vọng ban đầu.

Dựa trên phân cấp quy trình phát triển, các phương pháp kiểm thử động được cấu trúc thành bốn cấp độ chính bao gồm: Kiểm thử đơn vị (Unit Testing), Kiểm thử tích hợp (Integration Testing), Kiểm thử hệ thống (System Testing) và Kiểm thử chấp nhận sản phẩm (Acceptance Testing).

1.2.3 Các kỹ thuật kiểm thử

Ba chiến lược kiểm thử phổ biến hiện nay gồm kiểm thử hộp đen, kiểm thử hộp trắng và kiểm thử hộp xám. Mỗi chiến lược sẽ có cách tiếp cận khác nhau nhằm kiểm tra chất lượng và độ chính xác của phần mềm.

1.2.3.1 Kiểm thử hộp đen (Black box testing)

Kiểm thử hộp đen (còn gọi là kiểm thử hướng vào/ra) là chiến lược đánh giá phần mềm mà không cần quan tâm đến cấu trúc mã nguồn hay cách thức vận hành nội bộ của hệ thống. Kỹ sư kiểm thử tiếp cận chương trình như một "hộp kín", hoàn toàn tập trung vào việc truyền dữ liệu đầu vào và kiểm tra kết quả đầu ra. Mục tiêu chính của phương pháp này là phát hiện các trường hợp hệ thống hoạt động sai lệch so với tài liệu đặc tả yêu cầu ban đầu.

Các phương pháp kiểm thử hộp đen:

- **Phân lớp tương đương (Equivalence Partitioning):** Chia dữ liệu đầu vào thành các nhóm tương đương để giảm số lượng test case.
- **Phân tích giá trị biên (Boundary Value Analysis):** Kiểm tra các giá trị ở giới hạn trên và dưới của dữ liệu đầu vào.
- **Kiểm thử mọi cặp (All-pairs Testing):** Kiểm tra các cặp dữ liệu kết hợp với nhau nhằm giảm số lượng trường hợp kiểm thử.
- **Kiểm thử Fuzz (Fuzz Testing):** Nhập dữ liệu bất thường hoặc ngẫu nhiên để kiểm tra khả năng xử lý lỗi của hệ thống.
- **Kiểm thử dựa trên mô hình (Model-Based Testing):** Thiết kế test case dựa trên mô hình hoạt động của hệ thống..
- **Kiểm thử khám phá (Exploratory Testing):** Tester vừa kiểm thử vừa tìm hiểu hệ thống mà không cần kịch bản cố định trước.

Kiểm thử dựa trên đặc tả hướng tới việc xác thực các chức năng của phần mềm nhằm đảm bảo tính đồng nhất với tài liệu thiết kế hệ thống. Quy trình được vận hành

theo cơ chế hướng dữ liệu, trong đó kỹ sư QA chỉ tương tác qua giao diện để truyền tham số đầu vào và ghi nhận kết quả đầu ra. Việc triển khai đòi hỏi các kịch bản kiểm thử (test cases) phải được thiết kế chi tiết để đối chiếu chính xác giữa hành vi thực tế và giá trị kỳ vọng. Dù là giai đoạn bắt buộc trong quy trình đảm bảo chất lượng, phương pháp này vẫn tồn tại những hạn chế nhất định và không thể loại bỏ hoàn toàn mọi rủi ro tiềm ẩn của sản phẩm.

1.2.3.2 Kiểm thử hộp trắng (White box testing)

Trái ngược với phương pháp hộp đen, kiểm thử hộp trắng (hoặc kiểm thử hướng logic) là chiến lược tập trung vào việc phân tích và đánh giá cấu trúc nội bộ của chương trình. Quy trình này đòi hỏi kỹ sư QA phải truy cập trực tiếp vào mã nguồn, cấu trúc dữ liệu và các thuật toán xử lý bên trong hệ thống. Dựa trên cơ sở đó, các kịch bản kiểm thử sẽ được thiết lập nhằm mục đích xác thực tính chính xác trong logic thực thi và các nhánh rẽ của chương trình.

Các phương pháp kiểm thử hộp trắng:

- **Kiểm thử API (API Testing):** Kiểm tra hoạt động của các API và dữ liệu trao đổi giữa các hệ thống.
- **Kiểm thử bao phủ mã nguồn (Code Coverage):** Đánh giá mức độ các đoạn mã đã được thực thi trong quá trình kiểm thử.
- **Gán lỗi (Fault Injection):** Chủ động tạo lỗi để kiểm tra khả năng xử lý của hệ thống.
- **Kiểm thử đột biến (Mutation Testing):** Thay đổi nhỏ trong mã nguồn để đánh giá chất lượng test case.
- **Kiểm thử tĩnh (Static Testing):** Kiểm tra mã nguồn hoặc tài liệu mà không cần chạy chương trình.

Phương pháp kiểm thử hộp trắng còn được áp dụng để đánh giá mức độ bao phủ và tính hoàn thiện của các bộ kịch bản được thiết kế theo chiến lược hộp đen. Kỹ thuật này cho phép đội ngũ phát triển rà soát sâu vào các phân hệ khuất hoặc ít khi được thực thi, từ đó đảm bảo toàn bộ các tính năng chính của hệ thống đều được kiểm tra, không bỏ sót lỗi hỏng logic.

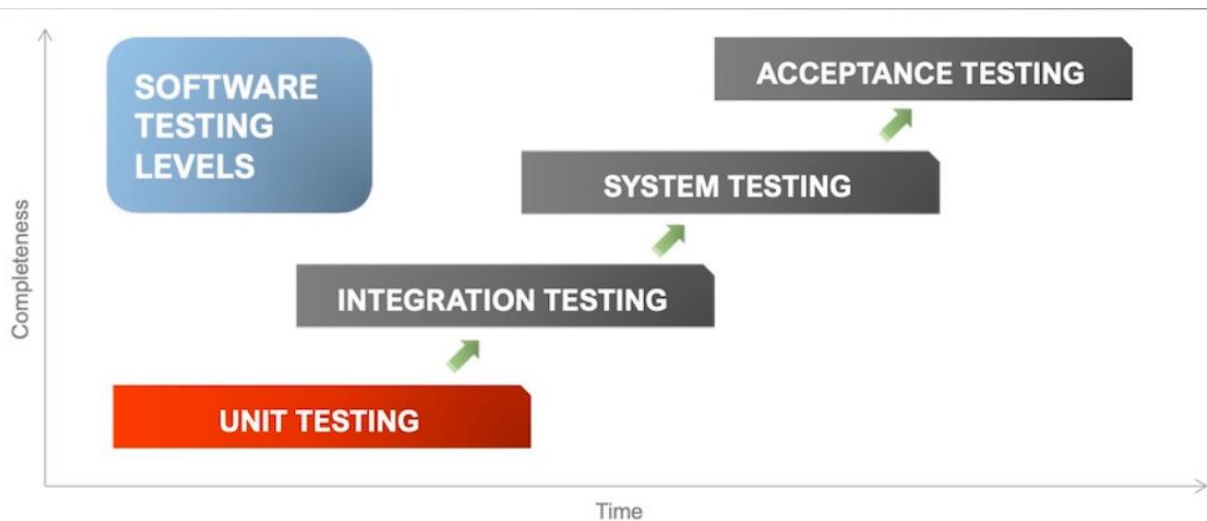
1.2.3.3 Kiểm thử hộp xám – Gray box testing

Kiểm thử hộp xám là sự kết hợp giữa hai chiến lược hộp đen và hộp trắng. Phương pháp này cho phép kỹ sư QA tiếp cận một phần cấu trúc dữ liệu và giải thuật nội bộ để thiết kế các kịch bản kiểm thử (test cases), nhưng quá trình thực thi lại được

tiến hành trên giao diện người dùng. Đặc trưng này thể hiện rõ nhất trong giai đoạn kiểm thử tích hợp (Integration Testing) giữa các mô-đun do các lập trình viên khác nhau phát triển, nơi dữ liệu chỉ được tương tác qua lại giữa các giao diện kết nối. Ngoài ra, kiểm thử hộp xám còn bao gồm việc phân tích cấu trúc hệ thống để thiết lập các kịch bản đối chiếu chuyên sâu như xác định giá trị biên hoặc kiểm tra cơ chế phản hồi thông báo lỗi.

1.2.4 Các cấp độ kiểm thử phần mềm

Trong kiểm thử phần mềm có bốn cấp độ chính gồm: kiểm thử đơn vị (Unit Testing), kiểm thử tích hợp (Integration Testing), kiểm thử hệ thống (System Testing) và kiểm thử chấp nhận (Acceptance Testing).



Hình 1.1: Các cấp độ kiểm thử phần mềm

1.2.4.1 Kiểm thử đơn vị (Unit Test)

❖ Định nghĩa

Kiểm thử đơn vị là quá trình kiểm tra các thành phần phần mềm có cấu trúc nhỏ nhất trong mã nguồn như hàm (Function), thủ tục (Procedure), lớp (Class) hoặc phương thức (Method). Hoạt động này chủ yếu do lập trình viên trực tiếp thực thi ngay trong giai đoạn viết mã và duy trì liên tục suốt chu trình phát triển sản phẩm. Do đó, quy trình này đòi hỏi người thực hiện phải có sự am hiểu sâu sắc về kiến trúc thiết kế và mã lệnh của chương trình.

❖ Mục đích

- **Xác thực dữ liệu:** Đảm bảo tính chính xác của luồng thông tin xử lý và kết quả đầu ra dựa trên các tham số đầu vào của từng đơn vị cấu trúc.

- **Tối ưu hóa độ bao phủ logic:** Rà soát và kiểm tra toàn bộ các nhánh rẽ bên trong mã lệnh nhằm phát hiện và định vị các câu lệnh phát sinh lỗi.
- **Quản trị rủi ro sớm:** Chủ động nhận diện các lỗ hổng kỹ thuật và rủi ro tiềm ẩn của hệ thống ngay từ cấp độ cơ bản nhất.

*Yêu cầu

- **Thiết lập kịch bản:** Đòi hỏi phải xây dựng sẵn các kịch bản kiểm thử (Test Cases/Test Scripts) chi tiết, định nghĩa rõ ràng các tham số đầu vào, quy trình thực thi và giá trị kỳ vọng ở đầu ra.
- **Tính tái sử dụng:** Các bộ dữ liệu và kịch bản kiểm thử này cần được lưu trữ hệ thống để phục vụ cho công tác tái kiểm thử (Regression Testing) sau này.

1.2.4.2 Kiểm thử tích hợp (Integration Test)

Kiểm thử tích hợp là quá trình kết hợp nhiều thành phần hoặc module của hệ thống lại với nhau để kiểm tra khả năng hoạt động và trao đổi dữ liệu giữa chúng. Sau khi các module đã được kiểm thử riêng lẻ bằng Unit Test, Integration Test sẽ tiếp tục kiểm tra xem các module có phối hợp đúng với nhau hay không. Mục đích của kiểm thử tích hợp là phát hiện các lỗi liên quan đến giao tiếp dữ liệu, kết nối API hoặc luồng xử lý giữa các thành phần trong hệ thống.

Trong kiểm thử tích hợp thường có các loại kiểm thử sau:

1. **Kiểm thử cấu trúc (Structure Test):** Kiểm thử cấu trúc tập trung vào việc kiểm tra luồng xử lý bên trong chương trình như câu lệnh, điều kiện hoặc nhánh xử lý. Loại kiểm thử này giúp đảm bảo các thành phần trong hệ thống hoạt động đúng theo logic đã thiết kế.
2. **Kiểm thử chức năng (Functional Test):** Kiểm thử chức năng dùng để kiểm tra xem hệ thống có hoạt động đúng theo yêu cầu nghiệp vụ hay không. Phương pháp này chủ yếu quan tâm đến dữ liệu đầu vào và kết quả đầu ra mà không đi sâu vào cấu trúc bên trong chương trình.
3. **Kiểm thử hiệu năng (Performance Test):** Kiểm thử hiệu năng được thực hiện nhằm đánh giá tốc độ xử lý, thời gian phản hồi và mức độ ổn định của hệ thống trong quá trình hoạt động.
4. **Kiểm thử khả năng chịu tải (Stress Test):** Kiểm thử khả năng chịu tải giúp xác định giới hạn hoạt động của hệ thống khi có số lượng lớn người dùng truy cập hoặc xử lý dữ liệu trong cùng một thời điểm. Qua đó có thể đánh giá độ ổn định và khả năng đáp ứng của hệ thống khi chịu áp lực cao.

1.2.4.3 Kiểm thử hệ thống (System Test)

Kiểm thử hệ thống là quá trình kiểm tra toàn bộ hệ thống sau khi các thành phần đã được tích hợp hoàn chỉnh nhằm đảm bảo hệ thống hoạt động đúng theo yêu cầu đặt ra. Giai đoạn này thường được thực hiện độc lập với nhóm phát triển để đảm bảo tính khách quan. Một số loại kiểm thử phổ biến trong System Test gồm:

- **Kiểm thử chức năng (Functional Test):** Kiểm tra các chức năng của hệ thống có hoạt động đúng yêu cầu hay không.
- **Kiểm thử hiệu năng (Performance Test):** Đánh giá tốc độ xử lý và thời gian phản hồi của hệ thống.
- **Kiểm thử chịu tải (Load/Stress Test):** Kiểm tra khả năng hoạt động của hệ thống khi có nhiều người dùng truy cập cùng lúc.
- **Kiểm thử bảo mật (Security Test):** Đảm bảo dữ liệu và hệ thống được an toàn.
- **Kiểm thử phục hồi (Recovery Test):** Kiểm tra khả năng khôi phục của hệ thống khi xảy ra lỗi hoặc mất dữ liệu.

Kết luận: Tùy theo yêu cầu và thời gian của dự án mà nhóm kiểm thử sẽ lựa chọn các loại kiểm thử phù hợp.

1.2.4.4 Kiểm thử chấp nhận – Acceptance Test

Sau khi hoàn thành kiểm thử hệ thống, phần mềm sẽ được thực hiện kiểm thử chấp nhận nhằm xác nhận hệ thống đã đáp ứng đúng yêu cầu của khách hàng. Giai đoạn này thường do khách hàng hoặc bên thứ ba thực hiện trước khi sản phẩm được đưa vào sử dụng chính thức. Acceptance Test gồm hai loại phổ biến:

- **Alpha Test:** Người dùng kiểm thử phần mềm tại môi trường phát triển. Các lỗi hoặc góp ý sẽ được ghi nhận để lập trình viên chỉnh sửa và hoàn thiện hệ thống.
- **Beta Test:** Phần mềm được đưa cho người dùng trải nghiệm trong môi trường thực tế. Những lỗi phát sinh hoặc phản hồi từ người dùng sẽ được gửi lại cho nhóm phát triển để tiếp tục cải thiện sản phẩm.

1.2.5 Các nguyên tắc trong kiểm thử phần mềm

Kiểm thử phần mềm là quá trình kiểm tra hệ thống nhằm phát hiện lỗi và đảm bảo phần mềm hoạt động đúng yêu cầu. Một chiến dịch kiểm thử hiệu quả không chỉ tối ưu hóa độ ổn định của mã nguồn mà còn đảm bảo sản phẩm vận hành đúng theo đặc tả yêu cầu. Tuy nhiên, để dung hòa giữa áp lực thời gian và giới hạn về mặt nguồn lực,

quy trình đảm bảo chất lượng phần mềm cần được định hướng dựa trên 7 nguyên lý chính sau đây:

*** Nguyên lý 1: Kiểm thử chỉ ra sự hiện diện của lỗi**

Hoạt động kiểm thử được thực thi nhằm chứng minh hệ thống tồn tại khiếm khuyết chứ không thể khẳng định sản phẩm hoàn toàn "sạch bug". Việc không tìm thấy lỗi trong môi trường giả lập không đồng nghĩa với việc mã nguồn đạt trạng thái hoàn hảo tuyệt đối khi triển khai trên môi trường thực tế.

***Nguyên lý 2. Kiểm thử toàn bộ là bất khả thi**

Do sự bùng nổ về mặt dữ liệu, tính phức tạp của các cấu trúc rẽ nhánh và sự đa dạng của các kịch bản đầu vào/đầu ra, việc kiểm thử mọi trường hợp là điều không thể thực thi. Thay vào đó, kỹ sư QA/Tester cần áp dụng các kỹ thuật phân tích rủi ro để thiết lập mức độ ưu tiên cho các phân hệ cốt lõi.

***Nguyên lý 3. Chiến lược kiểm thử sớm**

Tiến trình kiểm thử cần được tích hợp ngay từ những giai đoạn đầu tiên của vòng đời phát triển phần mềm (SDLC), bao gồm khâu phân tích đặc tả yêu cầu và thiết kế hệ thống. Việc phát hiện sai sót từ giai đoạn sơ khởi giúp giảm thiểu tối đa chi phí khắc phục sai lệch cấu trúc.

***Nguyên lý 4. Sự tập trung của lỗi**

Theo nguyên lý Pareto (quy luật 80/20), phần lớn các lỗi nghiêm trọng thường có xu hướng tích tụ tại một số ít các module trọng yếu hoặc các phân hệ xử lý logic phức tạp. Việc nhận diện chính xác khu vực dễ lỗi này cho phép tối ưu hóa hiệu suất tìm kiếm khiếm khuyết của đội ngũ kiểm thử.

***Nguyên lý 5. Nghịch lý thuốc trừ sâu**

Nếu một tập hợp kịch bản kiểm thử (test cases) được lặp đi lặp lại liên tục, xác suất phát hiện các lỗi mới sẽ giảm dần theo thời gian do hệ thống đã được tối ưu hóa để thích ứng với các kịch bản cũ. Để duy trì tính hiệu quả, các bộ kiểm thử cần được rà soát, cập nhật định kỳ và kết hợp linh hoạt với quy trình kiểm thử hồi quy (regression testing).

***Nguyên lý 6. Kiểm thử phụ thuộc vào ngữ cảnh**

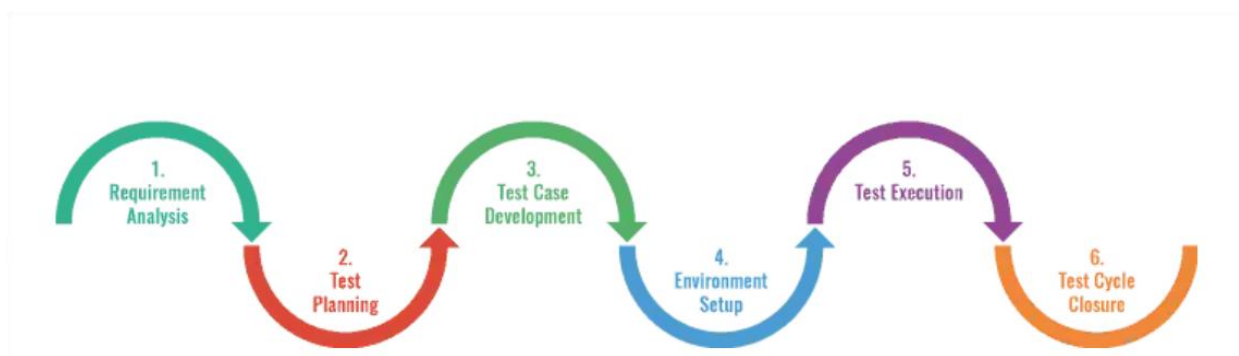
Phương thức, kỹ thuật và chiến lược kiểm thử phải được tùy biến động dựa trên đặc thù của từng loại sản phẩm. Một hệ thống quản trị giao dịch tài chính hoặc thương

máy điện tử sẽ đòi hỏi các tiêu chí nghiêm ngặt về bảo mật và tải hệ thống, khác biệt hoàn toàn với quy trình kiểm thử một ứng dụng hiển thị thông tin tĩnh.

***Nguyên lý 7. Quan niệm sai lầm về việc “hết lỗi”**

Một sản phẩm phần mềm dù được tối ưu hóa để loại bỏ 99% lỗi kỹ thuật vẫn có thể bị coi là một dự án thất bại nếu nó được xây dựng dựa trên một tài liệu đặc tả sai lệch, dẫn đến việc không đáp ứng đúng kỳ vọng và nhu cầu thực tế từ phía người dùng cuối.

1.2.6 Quy trình kiểm thử phần mềm



Hình 1.2: Quy trình kiểm thử phần mềm

1. Phân tích yêu cầu (Requirement Analysis)
2. Lập kế hoạch kiểm thử (Test Planning)
3. Thiết kế test case (Test Case Development)
4. Thiết lập môi trường kiểm thử (Test Environment Setup)
5. Thực hiện kiểm thử (Test Execution)
6. Kết thúc và tổng kết kiểm thử (Test Cycle Closure)

Mỗi giai đoạn kiểm thử sẽ có mục tiêu, dữ liệu đầu vào và kết quả đầu ra riêng. Tuy nhiên, tất cả đều hướng đến mục tiêu chung là đảm bảo phần mềm hoạt động ổn định, đúng yêu cầu và đáp ứng tốt nhu cầu của người dùng.

1.3 Tìm hiểu về kiểm thử tự động

1.3.1 Khái niệm về kiểm thử tự động

Kiểm thử tự động là phương pháp sử dụng các công cụ hoặc phần mềm để tự động thực hiện các test case thay cho thao tác thủ công. Các công cụ này có thể tự nhập dữ liệu kiểm thử, so sánh kết quả thực tế với kết quả mong đợi và tạo báo cáo sau khi kiểm thử hoàn tất. Nhờ đó, quá trình kiểm thử trở nên nhanh hơn, giảm sai sót và tiết kiệm thời gian cho tester.

1.3.2 Mục đích của kiểm thử tự động

- **Tăng tốc độ thực thi:** Chạy các kịch bản kiểm thử (test cases) nhanh hơn nhiều so với thao tác thủ công, đặc biệt với các bộ kiểm thử lớn.
- **Tăng độ tin cậy và chính xác:** Loại bỏ lỗi do con người gây ra khi thực hiện các tác vụ lặp đi lặp lại hoặc đơn điệu.
- **Hỗ trợ kiểm thử lặp lại (Regression Testing):** Tự động hóa các bài kiểm tra hồi quy để đảm bảo tính năng mới không làm hỏng tính năng cũ.
- **Phát hiện lỗi sớm:** Giúp phát hiện bug ngay trong giai đoạn phát triển, từ đó giảm chi phí và thời gian sửa lỗi.
- **Tối ưu hóa nguồn lực:** Giải phóng tester khỏi các công việc lặp lại, cho phép họ tập trung vào các kịch bản phức tạp hơn.
- **Đảm bảo tính nhất quán:** Thực hiện chính xác các bước kiểm thử theo quy trình đã thiết lập.

1.3.3 So sánh kiểm thử tự động và kiểm thử thủ công

Bảng 1.1 So sánh kiểm thử thủ công và kiểm thử tự động

Tiêu chí	Kiểm thử thủ công	Kiểm thử tự động
Cách thực hiện	Người kiểm thử trực tiếp thao tác trên hệ thống, chạy test case bằng tay và quan sát kết quả.	Sử dụng công cụ và script để tự động chạy kiểm thử mà không cần sự can thiệp liên tục của con người.
Tốc độ và quy mô	Chậm, phụ thuộc vào số lượng người kiểm thử, rất khó mở rộng khi kiểm thử nhiều kịch bản.	Nhanh, có thể chạy nhiều kịch bản cùng lúc, dễ mở rộng.
Độ chính xác	Dễ bị ảnh hưởng bởi yếu tố con người nên có thể sai sót hoặc không nhất quán.	Kết quả ổn định, lặp lại chính xác do chạy theo script cố định.
Chi phí	Chi phí ban đầu thấp nhưng tốn nhân lực lâu dài.	Chi phí ban đầu cao nhưng tiết kiệm khi kiểm thử lặp nhiều lần.

Tính linh hoạt	Rất linh hoạt, có thể thay đổi cách kiểm thử ngay lập tức, nên phù hợp với kiểm thử khám phá, UX.	Kém linh hoạt hơn vì phải sửa script khi hệ thống thay đổi.
----------------	---	---

1.4 Tìm hiểu về Apache JMeter

1.4.1 Tổng quan về công cụ

Apache JMeter là công cụ mã nguồn mở được phát triển bởi Apache Software Foundation, dùng để kiểm thử hiệu năng và kiểm thử tải cho phần mềm. Công cụ này được viết bằng ngôn ngữ Java và có thể hoạt động trên nhiều nền tảng khác nhau.

JMeter thường được sử dụng để đánh giá khả năng xử lý của hệ thống khi có nhiều người dùng truy cập cùng lúc. Công cụ có thể mô phỏng số lượng lớn người dùng ảo để kiểm tra tốc độ phản hồi và độ ổn định của ứng dụng. Ngoài ứng dụng web, JMeter còn hỗ trợ kiểm thử nhiều giao thức như HTTP, HTTPS, JDBC, FTP và SOAP.

1.4.2 Ưu nhược điểm

1.4.2.1 Ưu điểm

- Là công cụ mã nguồn mở và hoàn toàn miễn phí.
- Giao diện dễ sử dụng, phù hợp cho người mới bắt đầu.
- Có thể chạy trên nhiều hệ điều hành khác nhau.
- Hỗ trợ mô phỏng nhiều người dùng cùng lúc.
- Kết quả kiểm thử được hiển thị dưới dạng biểu đồ, bảng hoặc báo cáo trực quan.
- Hỗ trợ nhiều loại kiểm thử như Load Testing và Performance Testing.
- Có thể mở rộng chức năng bằng các plugin hỗ trợ.
- Hỗ trợ nhiều giao thức như HTTP, JDBC, FTP, SOAP, ...
- Có khả năng ghi lại thao tác người dùng trên trình duyệt để phục vụ kiểm thử.
- Có thể tích hợp với Selenium và BeanShell để hỗ trợ kiểm thử tự động.

1.4.2.2 Nhược điểm

- Khó sử dụng với các bài kiểm thử phức tạp.
- Báo cáo kết quả có thể khó hiểu với người mới.
- Không hỗ trợ tốt các ứng dụng sử dụng nhiều JavaScript hoặc AJAX.

- Tiêu tốn khá nhiều bộ nhớ khi chạy tải lớn.
- Chủ yếu hỗ trợ kiểm thử ứng dụng web, chưa phù hợp với ứng dụng desktop.

1.4.3 Đặc trưng và cách thức hoạt động của Jmeter

1.4.3.1 Đặc điểm của JMeter

Apache JMeter là giải pháp mã nguồn mở tối ưu phục vụ công tác kiểm thử hiệu năng, đo lường tải lượng và đánh giá sức chịu tải của hệ thống. Công cụ này sở hữu các đặc tính kỹ thuật chính sau

- **Đa dạng hóa giao thức kết nối:** Tích hợp khả năng tương thích với nhiều chuẩn giao thức mạng và cơ sở dữ liệu phổ biến bao gồm HTTP, HTTPS, FTP, JDBC, SOAP, cũng như các dịch vụ Web RESTful.
- **Giả lập hành vi người dùng:** Cho phép cấu hình các luồng thực thi (Threads) nhằm mô phỏng đồng thời hành vi tương tác từ số lượng lớn người dùng ảo, từ đó đánh giá độ ổn định của ứng dụng dưới các kịch bản thực tế khác nhau.
- **Xây dựng kịch bản tải phức tạp:** Hỗ trợ thiết lập các kịch bản kiểm thử hiệu năng đa tầng, giúp đo lường giới hạn chịu tải (Stress Testing) và duy trì độ bền (Endurance Testing) của hệ thống.
- **Đo lường thời gian phản hồi (Response Time):** Tự động ghi nhận, phân tích thời gian xử lý của các truy vấn (Requests) và kết xuất các chỉ số chi tiết về độ trễ mạng (Latency).
- **Phân tích và kết xuất báo cáo (Dashboard Report):** Cung cấp hệ thống báo cáo trực quan dưới dạng biểu đồ, thống kê chi tiết về tỷ lệ lỗi (Error Rate), throughput (băng thông) và mức độ tiêu hao tài nguyên phần cứng.
- **Kiến trúc mở rộng linh hoạt:** Hỗ trợ nhúng các đoạn mã tối ưu hóa bằng ngôn ngữ Java, Groovy hoặc JavaScript, đồng thời cho phép tích hợp hệ sinh thái Plugins phong phú từ bên thứ ba để tùy biến tính năng.

1.4.3.2 Cách thức hoạt động

Về nguyên lý hoạt động, Apache JMeter tự động khởi tạo các truy vấn giả lập để tạo áp lực tải lên hệ thống đích, từ đó đo lường và ghi nhận các chỉ số về thời gian phản hồi. Quy trình thực thi kiểm thử hiệu năng thông qua công cụ này được chuẩn hóa theo 4 bước chính sau:

- **Bước 1: Thiết lập kịch bản thử nghiệm (Test Plan):** Khởi tạo cấu trúc kịch bản bằng cách định nghĩa các yêu cầu dịch vụ (HTTP Requests). Kỹ sư kiểm thử tiến hành cấu hình chi tiết các tham số kỹ thuật bao gồm địa chỉ URL, phương

thức định tuyến (GET/POST), các tham số truyền tải (Parameters) và tiêu đề (Headers).

- **Bước 2: Cấu hình tham số tải (Thread Group):** Xác lập các thông số đo lường hiệu năng của hệ thống. Các chỉ số cần cấu hình bao gồm số lượng người dùng ảo đồng thời (Number of Threads), thời gian tăng trưởng tải (Ramp-up Period), chu kỳ lặp (Loop Count) và khoảng thời gian trễ giữa các truy vấn (Timers).
- **Bước 3: Thực thi kiểm thử (Execution):** Kích hoạt tiến trình chạy tự động để tiến hành gửi tải lượng theo kịch bản đã hoạch định. Hệ thống sẽ đồng thời theo dõi, ghi nhận hành vi và đo lường độ trễ phản hồi của từng luồng dữ liệu theo thời gian thực.
- **Bước 4: Phân tích và đánh giá kết quả (Result Analysis):** Khai thác các bộ lắng nghe kết quả (Listeners) của JMeter để thu thập báo cáo trực quan. Dựa trên các thông số về thời gian phản hồi, tỷ lệ lỗi và mức độ tiêu hao tài nguyên, kỹ sư sẽ tiến hành định vị các điểm nghẽn hiệu năng (Bottlenecks) của ứng dụng.

1.5 Tìm hiểu về Playwright

1.5.1 Tổng quan về công cụ

Playwright là một framework kiểm thử tự động mã nguồn mở được phát triển bởi Microsoft, nó là một bộ công cụ mạnh mẽ giúp bạn viết mã lệnh (script) để thực hiện các thao tác kiểm thử trên ứng dụng web của mình một cách tự động, giống như cách người dùng thực tế đang tương tác với trang web vậy. Playwright được sử dụng để tự động hóa việc kiểm thử các ứng dụng web hiện đại trên nhiều trình duyệt khác nhau như Chromium (Chrome, Edge), Firefox và WebKit (Safari).

Với khả năng tương thích đa nền tảng và đa ngôn ngữ lập trình như JavaScript, TypeScript, Python, Java và .NET, Playwright giúp các nhóm phát triển và kiểm thử xây dựng các kịch bản kiểm thử end-to-end mạnh mẽ, ổn định và dễ bảo trì.

1.5.2 Ưu nhược điểm của Playwright

1.5.2.1 Ưu điểm

Playwright giới thiệu hàng loạt tính năng hữu ích, bao gồm:

- Hỗ trợ đa trình duyệt: Chỉ với một API duy nhất, bạn có thể kiểm thử trên Chromium (Chrome, Edge), Firefox và WebKit (Safari), đảm bảo ứng dụng chạy mượt mà trên mọi nền tảng.

- Đa ngôn ngữ: Không chỉ JavaScript/TypeScript, Playwright còn hỗ trợ Python, Java và C#, giúp bạn linh hoạt chọn ngôn ngữ quen thuộc để viết test script.
- Auto-waiting thông minh: Tự động chờ đợi phần tử hiển thị, sẵn sàng cho tương tác, loại bỏ nhu cầu xử lý wait thủ công và giảm thiểu lỗi flakes.
- API đơn giản, dễ học: Cấu trúc API rõ ràng, hàm gọi ngắn gọn, giúp tester mới bắt đầu nhanh chóng làm quen và lướt qua các bài test.
- Chụp ảnh và quay video: Tích hợp sẵn khả năng chụp màn hình hoặc ghi lại video quá trình thực thi, hỗ trợ đắc lực cho việc debug và báo cáo kết quả.
- Kiểm thử tính năng nâng cao: Hỗ trợ xử lý iframe, shadow DOM, geolocation, upload/download file, mobile viewport và các tính năng phức tạp khác.
- Tích hợp CI/CD: Dễ dàng kết hợp vào các pipeline GitHub Actions, Jenkins, GitLab CI..., giúp tự động hóa kiểm thử mỗi khi có thay đổi mã nguồn.

1.5.2.2 Nhược điểm

- Cộng đồng và hệ sinh thái nhỏ: So với Selenium, Playwright là công cụ mới hơn, dẫn đến ít tài liệu hướng dẫn, plugin hỗ trợ, và các giải pháp cho các tình huống lỗi đặc thù (edge cases).
- Tốn tài nguyên hệ thống: Khi chạy nhiều trình duyệt (browser instances) song song, Playwright có thể ngốn đáng kể RAM và CPU, đặc biệt trên các máy có cấu hình yếu.
- Độ khó khi học: Đòi hỏi lập trình viên phải quen thuộc với mô hình lập trình bất đồng bộ (async/await) trong JavaScript/TypeScript hoặc các ngôn ngữ khác, gây khó khăn cho người mới.

1.5.3 Tính năng nổi bật của Playwright

***Hỗ trợ đa trình duyệt**

Một trong những ưu điểm lớn nhất của Playwright là khả năng hỗ trợ đa trình duyệt. Không chỉ tương thích với Chromium (được sử dụng bởi Chrome và Edge), Playwright còn hỗ trợ Firefox và WebKit (nền tảng của Safari). Điều này cho phép các nhóm phát triển dễ dàng kiểm thử ứng dụng của mình trên nhiều môi trường trình duyệt khác nhau chỉ với một bộ mã duy nhất, đảm bảo trải nghiệm người dùng nhất quán trên mọi nền tảng.

***Hỗ trợ đa ngôn ngữ lập trình**

Playwright cung cấp API cho nhiều ngôn ngữ lập trình phổ biến như JavaScript, TypeScript, Python, Java và .NET. Nhờ đó, các đội ngũ kiểm thử có thể linh hoạt lựa

chọn ngôn ngữ phù hợp với hệ sinh thái hiện có của mình mà không cần phải học lại từ đầu. Điều này không chỉ rút ngắn thời gian triển khai mà còn giúp dễ dàng tích hợp với các framework và công cụ hiện hành.

***Tính năng auto-waiting (tự động chờ đợi phần tử)**

Khác với nhiều công cụ kiểm thử truyền thống yêu cầu người dùng tự thiết lập thời gian chờ hoặc xử lý bất đồng bộ phức tạp, Playwright được tích hợp sẵn cơ chế auto-waiting thông minh. Hệ thống sẽ tự động đợi các phần tử hiển thị, sẵn sàng tương tác trước khi thực hiện hành động như click, nhập liệu hoặc chuyển trang. Điều này giúp giảm thiểu lỗi kiểm thử do trang tải chậm hoặc thao tác quá sớm, từ đó nâng cao độ chính xác của kịch bản kiểm thử.

***Khả năng chạy song song và tương thích đa nền tảng**

Playwright hỗ trợ thực thi các kịch bản kiểm thử theo kiểu song song (parallel), giúp tiết kiệm đáng kể thời gian khi kiểm thử một số lượng lớn tính năng. Đồng thời, công cụ này có thể hoạt động trên mọi hệ điều hành phổ biến như Windows, macOS và Linux. Việc tích hợp dễ dàng vào các hệ thống CI/CD như GitHub Actions, Jenkins, GitLab CI cũng giúp Playwright trở thành giải pháp lý tưởng cho môi trường kiểm thử tự động hóa liên tục.

***Hỗ trợ chụp ảnh màn hình, quay video quá trình test**

Để phục vụ việc ghi nhận lỗi và phân tích kết quả kiểm thử, Playwright cung cấp tính năng chụp ảnh màn hình và quay video toàn bộ quá trình kiểm thử. Các bản ghi này có thể được lưu lại trong báo cáo kiểm thử hoặc gửi kèm khi báo lỗi, giúp lập trình viên dễ dàng hình dung được ngữ cảnh xảy ra lỗi, từ đó rút ngắn thời gian xử lý và cải thiện hiệu quả làm việc giữa các bộ phận.

***Mô phỏng thiết bị di động và điều kiện mạng linh hoạt**

Playwright cho phép mô phỏng nhiều loại thiết bị như iPhone, iPad, Android với kích thước màn hình và khả năng cảm ứng tương ứng. Ngoài ra, công cụ còn hỗ trợ giả lập các điều kiện mạng như 3G, 4G, mạng yếu hoặc mất kết nối tạm thời. Đây là tính năng cực kỳ hữu ích khi kiểm thử trải nghiệm người dùng trên thiết bị di động, đảm bảo ứng dụng vận hành tốt trong mọi hoàn cảnh thực tế.

1.6 Tìm hiểu về OWASP ZAP

1.6.1 Tổng quan về OWASP ZAP

OWASP ZAP (Zed Attack Proxy) là một nền tảng mã nguồn mở tiên tiến được phát triển bởi tổ chức Open Web Application Security Project (OWASP), với mục tiêu chính là hỗ trợ các nhà phát triển và chuyên gia an ninh mạng trong việc nhận diện, đánh giá các lỗ hổng bảo mật trên ứng dụng web. Được thiết kế như một công cụ kiểm thử xâm nhập (Penetration Testing), OWASP ZAP đóng vai trò là màn lọc hiệu quả giúp phát hiện sớm các nguy cơ bị tấn công, từ đó tối ưu hóa tính an toàn và nâng cao năng lực phòng thủ của hệ thống trước khi đưa vào vận hành thực tế.

1.6.2 Tính năng nổi bật của OWASP ZAP

***Miễn phí và mã nguồn mở**

OWASP ZAP là công cụ mã nguồn mở và miễn phí, giúp các nhóm phát triển dễ dàng triển khai kiểm thử bảo mật mà không tốn nhiều chi phí. Công cụ được cộng đồng hỗ trợ và cập nhật thường xuyên nhằm phát hiện các lỗ hổng bảo mật phổ biến trên ứng dụng web.

***Hệ thống plugin có khả năng mở rộng**

Nền tảng được cấu trúc theo mô hình kiến trúc mô-đun (modular architecture), hỗ trợ mở rộng động các tính năng thông qua ZAP Marketplace. Cơ chế này cho phép các kỹ sư tích hợp linh hoạt các bộ lọc quét nâng cao, hệ thống kết xuất báo cáo chuyên sâu hoặc các công cụ tự động hóa, đáp ứng phần lớn các kịch bản đánh giá an ninh đặc thù.

***Hỗ trợ quét tự động và kiểm thử thủ công**

Giải pháp tích hợp song song hai phương thức tiếp cận an ninh hệ thống. Chế độ quét chủ động (Active Scanning) vận hành các thuật toán tự động nhằm định vị nhanh các lỗ hổng nghiêm trọng trong danh mục OWASP Top 10 như SQL Injection hay Cross-Site Scripting (XSS). Đồng thời, nền tảng cung cấp các bộ công cụ can thiệp sâu (Interception Proxy) phục vụ công tác kiểm thử thủ công, cho phép chuyên gia phân tích cấu trúc luồng truy vấn và phản hồi HTTP ở tầng logic.

***Tương thích đa nền tảng và dễ tích hợp**

OWASP ZAP hỗ trợ Windows, macOS và Linux. Công cụ cũng cung cấp API và CLI giúp dễ tích hợp vào quy trình CI/CD.

1.7 Tìm hiểu công cụ SonarQube

1.7.1 Tổng quan về SonarQube

SonarQube là công cụ phân tích mã nguồn tĩnh giúp lập trình viên kiểm tra và cải thiện chất lượng mã nguồn. Công cụ có khả năng phát hiện lỗi, lỗ hổng bảo mật và các đoạn mã chưa tối ưu trong quá trình phát triển phần mềm. Nền tảng này hoạt động như một màng lọc tự động trong quy trình Tích hợp liên tục (CI), giúp nhận diện sớm các lỗi logic (Bugs), lỗ hổng an ninh (Vulnerabilities) và nợ kỹ thuật (Technical Debt).

1.7.2 Tính năng nổi bật

- **Phân tích mã tĩnh:** Tự động quét sâu vào mã nguồn để nhận diện sớm các sai sót kỹ thuật như đoạn mã trùng lặp (Code Duplication), mã chết (Dead Code) và "mùi mã" (Code Smells).
- **Độ tin cậy và phân tích bảo mật mã:** Phát hiện sớm các lỗ hổng bảo mật ngay từ giai đoạn đầu của chu trình phát triển (SDLC), giúp đội ngũ kỹ sư chủ động phân loại và xử lý rủi ro theo cấp độ nghiêm trọng.
- **Quản lý nợ kỹ thuật:** Giảm thiểu nợ kỹ thuật bằng cách phân tích độ phức tạp thuật toán và cảnh báo các vùng mã chưa đạt độ bao phủ kiểm thử (Test Coverage).
- **Tích hợp CI/CD:** Nhúng liền mạch tiến trình phân tích mã nguồn vào các đường ống CI/CD, giúp tự động kiểm tra chất lượng mã nguồn trong quá trình build và triển khai hệ thống.
- **Hỗ trợ đa ngôn ngữ:** Đáp ứng linh hoạt các môi trường phát triển hỗn hợp với khả năng phân tích hơn 27 ngôn ngữ lập trình phổ biến (C, C++, Java, JavaScript, Python, C#, Kotlin,...).
- **Cổng chất lượng có thể tùy chỉnh:** Cho phép cấu hình các ngưỡng chốt chặn tự động trong pipeline CI/CD để ngăn chặn việc đóng gói và phát hành các phiên bản mã nguồn không đạt chuẩn hoặc kém an toàn.
- **Giám sát và trực quan hóa dữ liệu:** Cung cấp hệ thống quản lý sự cố lỗi kết hợp bảng điều khiển (Dashboard) trực quan để theo dõi tiến trình cải tiến chất lượng mã nguồn theo thời gian.

1.7.3 Ưu điểm

- **Tối ưu hóa vòng đời ứng dụng:** Giảm thiểu độ phức tạp thuật toán, triệt tiêu các đoạn mã trùng lặp và lỗi hỏng bảo mật, từ đó kéo dài tuổi thọ vận hành bền vững của hệ thống phần mềm.
- **Nâng cao năng suất phát triển:** Việc kiểm soát tốt quy mô và cấu trúc mã nguồn giúp cắt giảm thời gian tái cấu trúc (Refactoring), tối ưu hóa tiến độ chỉnh sửa mã lệnh của đội ngũ lập trình viên.
- **Đảm bảo tính chính xác của mã nguồn:** Cơ chế giám sát chất lượng ngay từ giai đoạn đầu giúp chuẩn hóa quy trình viết mã, đảm bảo sản phẩm đầu ra luôn đạt các tiêu chí kỹ thuật nghiêm ngặt.
- **Phát hiện lỗi sớm:** Giúp các nhóm phát hiện sớm các lỗi có trong mã, cảnh báo và tự động sửa lỗi trước khi xuất bản.
- **Định vị và cô lập sai sót sớm:** Chủ động phát hiện các lỗi cú pháp và logic ở dạng sơ khởi, hỗ trợ cảnh báo và đưa ra các gợi ý khắc phục tự động trước khi mã nguồn được đóng gói xuất bản.
- **Chuẩn hóa kỹ nghệ lập trình:** Các phản hồi kỹ thuật định kỳ về "mùi mã" (Code Smells) giúp các nhà phát triển liên tục cải thiện tư duy thuật toán và hạn chế các lỗi hệ thống trong tương lai.
- **Tăng cường khả năng bảo trì (Maintainability):** Duy trì sự đồng nhất về tiêu chuẩn mã hóa (Coding Standards) giúp mã nguồn trở nên trực quan, dễ đọc và dễ kế thừa, giảm thiểu rủi ro khi có sự thay đổi nhân sự trong dự án.
- **Tiết kiệm tối đa chi phí vận hành:** Việc ngăn chặn sớm các lỗi hỏng nghiêm trọng giúp doanh nghiệp giảm thiểu gánh nặng tài chính cho công tác khắc phục sự cố và bảo trì hệ thống về lâu dài.

1.8 Đặc thù kiểm thử chatbot và hệ thống hội thoại

Kiểm thử chatbot có nhiều điểm khác so với kiểm thử phần mềm thông thường. Ngoài việc kiểm tra chức năng, quá trình kiểm thử còn phải đánh giá khả năng hiểu ngôn ngữ tự nhiên, duy trì ngữ cảnh và độ chính xác của phản hồi. Đối với hệ thống chatbot của nhà xe Vũ Hán, việc kiểm thử không chỉ dừng lại ở các luồng logic cố định mà phải đánh giá tính phi tuyến tính của hội thoại, khả năng nhận diện thông tin hành trình và độ chính xác của tri thức được mô hình cung cấp.

1.8.1 Kiểm thử nhận diện ý định (Intent Recognition Testing)

Kiểm thử nhận diện ý định nhằm đánh giá khả năng chatbot hiểu đúng mục đích của người dùng thông qua câu nhập tự nhiên. Đây là thành phần chính quyết định chatbot có phản hồi đúng nghiệp vụ hay không.

- **Kiểm thử đa dạng hóa biểu đạt (Paraphrasing Testing):** Thử nghiệm với nhiều cách diễn đạt khác nhau cho cùng một mục đích. Ví dụ, cùng ý định đặt vé xe, khách có thể nói: "*Tôi muốn mua vé*", "*Đặt cho tôi ghế đi Tuyên Quang*", hoặc "*Còn chỗ xe Vũ Hán tối nay không?*".
- **Kiểm thử ranh giới ý định (Cross-Intent / Ambiguity Testing):** Thử nghiệm với các câu nhập liệu nằm ở ranh giới giữa hai Intent gần nhau để kiểm tra độ nhiễu (ví dụ: "*Hủy lịch đặt vé*" và "*Đổi giờ chạy*").

1.8.2 Kiểm thử trích xuất thực thể (Entity Extraction Testing)

Kiểm thử khả năng bóc tách chính xác các tham số/thông tin hành trình (Entities) có trong câu nói của người dùng để phục vụ cho các logic xử lý phía sau.

- **Thực thể hệ thống (System Entities):** Kiểm thử việc nhận diện định dạng ngày/tháng/năm, thời gian, số lượng, tiền tệ (ví dụ: "*ngày mai*", "*3h chiều*", "*3 ghế*").
- **Thực thể tùy chỉnh (Custom Entities):** Kiểm thử khả năng trích xuất các danh mục đặc thù của hệ thống nhà xe như mã đơn hàng, tên sản phẩm, tên bến xe/điểm đến (ví dụ: "*Bến xe Mỹ Đình*", "*Văn phòng Giáp Bát*", "*xe giường nằm*", "*Limousine*").
- **Xử lý đa thực thể (Multiple Entities):** Nhập câu lệnh chứa nhiều thực thể đồng thời để kiểm tra xem hệ thống có bóc tách đầy đủ hay bị sót (ví dụ: "*Đặt cho tôi 2 vé đi Hà Nội vào thứ Sáu*" → Số lượng: 2, Điểm đến: Hà Nội, Thời gian: thứ Sáu).

1.8.3 Kiểm thử luồng hội thoại nhiều bước (Multi-turn Dialog Management Testing)

Đảm bảo hệ thống duy trì được ngữ cảnh (Context) qua nhiều lượt trao đổi liên tiếp giữa khách hàng và chatbot.

- **Kiểm thử điền thông tin (Slot-Filling):** Khi khách yêu cầu đặt vé nhưng thiếu thông tin bắt buộc, kiểm tra xem chatbot nhà xe có biết đưa ra câu hỏi gợi ý để thu thập đủ thông tin theo kịch bản hay không (ví dụ: Thiếu ngày đi, thiếu số điện thoại).

- **Kiểm thử duy trì ngữ cảnh (Context Retention):** Khách sử dụng các đại từ thay thế ở câu sau (ví dụ: Câu 1: "*Chuyến xe Limousine đi Tuyên Quang còn chỗ không?*" → Câu 2: "*Giá vé của nó là bao nhiêu?*"). Hệ thống phải hiểu "nó" chính là "xe Limousine đi Tuyên Quang".
- **Kiểm thử chuyển hướng ngữ cảnh (Context Switching):** Khách đang thực hiện luồng đặt vé nhưng đột ngột hỏi sang luồng kiểm tra chính sách gửi hàng, sau đó quay lại luồng đặt vé. Hệ thống phải quản lý được trạng thái để không làm mất dữ liệu hành trình đã nhập trước đó.

1.8.4 Kiểm thử tính bền bỉ với ngôn ngữ tự nhiên (Robustness Testing)

Trong thực tế, người dùng có thể nhập dữ liệu không chuẩn hóa, sai chính tả hoặc thiếu thông tin. Vì vậy, chatbot cần được kiểm thử với nhiều trường hợp dữ liệu đầu vào khác nhau nhằm đánh giá khả năng xử lý linh hoạt của hệ thống.

- **Kiểm thử lỗi chính tả và chữ viết tắt:** Hệ thống phải có khả năng chuẩn hóa hoặc sử dụng cơ chế so khớp mờ (Fuzzy matching) để hiểu đúng các từ viết sai hoặc viết tắt thông dụng (ví dụ: "*ddawjt ves*" → đặt vé, "*bx Mỹ Đình*" → bến xe Mỹ Đình, "*vphong*" → văn phòng).
- **Kiểm thử câu hỏi nhập tự nhiên:** Thử nghiệm với các câu dài, hành văn lủng củng, chứa nhiều từ đệm hoặc từ lóng vùng miền (đặc biệt là phương ngữ Nghệ Tĩnh phù hợp với tuyến chạy trọng điểm của nhà xe) xem bộ lọc của chatbot có hoạt động hiệu quả không.

1.8.5 Kiểm thử phản hồi sai lệch tri thức (Hallucination Testing)

Hallucination là hiện tượng chatbot sinh ra thông tin sai hoặc không có trong dữ liệu thực tế của hệ thống. Đối với chatbot hỗ trợ khách hàng, đây là rủi ro nghiêm trọng vì có thể gây: Sai lệch giá vé, Sai lịch trình tuyến xe, Sai thông tin văn phòng hoặc chính sách hỗ trợ.

- **Kiểm thử độ chính xác dữ kiện (Factual Accuracy):** Đối chiếu các câu trả lời do AI tạo ra với cơ sở dữ liệu nguồn của nhà xe Vũ Hán để đảm bảo không có thông tin sai lệch về giá vé, lộ trình, giờ xuất bến hoặc chính sách nhà xe.
- **Kiểm thử chống ảo tưởng (Anti-Hallucination Testing):** Đưa vào các câu hỏi bẫy về các tuyến đường hoặc dịch vụ không tồn tại của nhà xe (ví dụ: "*Nhà xe Vũ Hán có chuyến đi SaPa không?*"). Đầu ra mong đợi là chatbot phải từ chối trả lời hoặc khẳng định nhà xe không phục vụ tuyến này, tuyệt đối không tự ảo tưởng ra lịch trình giả.

1.8.6 Kiểm thử tính nhất quán của phản hồi (Response Consistency Testing)

Chatbot cần đảm bảo tính ổn định và nhất quán trong quá trình phản hồi người dùng. Cùng một câu hỏi hoặc cùng một ngữ cảnh nghiệp vụ, hệ thống phải đưa ra kết quả tương đồng giữa nhiều lần kiểm thử nhằm đảm bảo độ tin cậy của hệ thống.

Ví dụ: “Giá vé Xín Mần – Mỹ Đình bao nhiêu?”. Kết quả phản hồi cần duy trì tính nhất quán giữa nhiều lần truy vấn liên tiếp. Kiểm thử tính nhất quán giúp: Nâng cao trải nghiệm người dùng, Giảm thiểu phản hồi mâu thuẫn, Đảm bảo độ ổn định của hệ thống chatbot.

1.8.7 Kiểm thử Fallback và Chuyển giao nhân viên hỗ trợ (Fallback & Human Escalation)

Trong các trường hợp chatbot không hiểu yêu cầu hoặc yêu cầu vượt ngoài phạm vi xử lý, hệ thống cần có cơ chế fallback nhằm tránh phản hồi sai hoặc lặp vô nghĩa gây ức chế cho hành khách đặt vé xe. Các nội dung cần kiểm thử bao gồm:

- Thông báo fallback khi chatbot không hiểu yêu cầu.
- Khả năng yêu cầu người dùng nhập lại thông tin.
- Cơ chế chuyển sang nhân viên hỗ trợ.
- Khả năng ghi nhận thông tin liên hệ của khách hàng.

Ví dụ: “Dạ em chưa hiểu rõ ý của anh/chị. Anh/chị vui lòng nhập lại câu hỏi để em hỗ trợ nhé.”, “Vui lòng để lại số điện thoại để nhân viên hỗ trợ liên hệ”.

Việc kiểm thử cơ chế fallback giúp nâng cao chất lượng hỗ trợ khách hàng và hạn chế các phản hồi không phù hợp trong quá trình vận hành thực tế.

CHƯƠNG 2: LẬP KẾ HOẠCH VÀ THIẾT KẾ TÀI LIỆU KIỂM THỬ

2.1 Phân tích yêu cầu và đối tượng kiểm thử

2.1.1 Tổng quan hệ thống chatbot nhà xe Vũ Hán

Hệ thống Chatbot nhà xe Vũ Hán là một ứng dụng hỗ trợ khách hàng hoạt động trên nền tảng web, cho phép người dùng tương tác thông qua giao diện hội thoại trực tuyến bằng ngôn ngữ tự nhiên. Chatbot được xây dựng nhằm hỗ trợ tự động hóa quá trình tư vấn và chăm sóc khách hàng, góp phần giảm tải cho nhân viên hỗ trợ và nâng cao trải nghiệm người dùng.

Hệ thống cung cấp các chức năng chính bao gồm:

1. Hỗ trợ tra cứu thông tin tuyến xe.
2. Tra cứu giá vé và lịch trình di chuyển.
3. Hỗ trợ đặt vé xe limousine và xe khách.
4. Hỗ trợ gửi hàng.
5. Giải đáp các câu hỏi thường gặp (FAQ).
6. Chuyển tiếp yêu cầu đến nhân viên hỗ trợ khi cần thiết.

Do đặc thù của hệ thống chatbot sử dụng xử lý ngôn ngữ tự nhiên (NLP), quá trình kiểm thử không chỉ tập trung vào chức năng nghiệp vụ mà còn phải đánh giá khả năng hiểu ngữ nghĩa, duy trì ngữ cảnh hội thoại và độ chính xác của phản hồi.

2.1.2 Đối tượng kiểm thử

Đối tượng kiểm thử của đề tài bao gồm toàn bộ các thành phần liên quan đến quá trình vận hành của hệ thống chatbot nhà xe Vũ Hán trên môi trường triển khai thực tế.

Các thành phần được đưa vào phạm vi kiểm thử gồm:

1. Giao diện hội thoại người dùng (Web Chat UI).
2. Hệ thống xử lý yêu cầu phía máy chủ (Backend API).
3. Module xử lý ngôn ngữ tự nhiên (NLP Processing).
4. Kho dữ liệu tri thức và cơ sở dữ liệu hệ thống.
5. Các API phục vụ hội thoại và xử lý nghiệp vụ.
6. Luồng xử lý hội thoại nhiều bước.
7. Hệ thống phản hồi và cơ chế fallback.

Các hoạt động kiểm thử được thực hiện nhằm đánh giá:

1. Độ chính xác trong nhận diện intent và entity.
2. Tính ổn định của luồng hội thoại.

3. Hiệu năng phản hồi của hệ thống.
4. Khả năng chịu tải đồng thời.
5. Mức độ an toàn và bảo mật dữ liệu.
6. Tính đúng đắn của chức năng.

2.1.3 Phân tích yêu cầu chức năng

Bảng 2.1 Yêu cầu chức năng hệ thống

Mã yêu cầu	Chức năng	Mô tả yêu cầu	Dữ liệu ra	Mức ưu tiên
FR-01	Nhận diện ý định	Xác định chính xác nhu cầu của người dùng thông qua nội dung hội thoại	Intent phù hợp	Rất cao
FR-02	Phân loại loại xe	Phân biệt xe Limousine, xe khách 40 chỗ, xe giường nằm.	Loại xe chính xác	Rất cao
FR-03	Đặt vé xe limousine	Thu thập và xác nhận thông tin đặt vé xe limousine	Giá vé	Cao
FR-04	Đặt vé xe khách	Thu thập và xác nhận thông tin đặt vé xe khách	Danh sách chuyến	Cao

FR-05	Giải đáp FAQ	Trả lời các câu hỏi thường gặp	Kết quả xác nhận	Rất cao
FR-06	Kiểm tra điểm đón trả	Cung cấp thông tin điểm đón/ trả, văn phòng	Nội dung phản hồi	Trung bình
FR-07	Hỗ trợ gửi hàng	Hướng dẫn thủ tục, cung cấp thông tin văn phòng và thông tin nhận hàng	Nội dung phản hồi	Trung bình
FR-08	Chuyển hỗ trợ viên	Chuyển tiếp yêu cầu sang nhân viên hỗ trợ	Thông báo chuyển tiếp	Cao

2.1.4 Phân tích yêu cầu phi chức năng

Bảng 2.2 Yêu cầu phi chức năng

Mã yêu cầu	Loại yêu cầu	Mô tả
NFR-01	Hiệu năng	Thời gian phản hồi trung bình không vượt quá 5 giây
NFR-02	Khả năng chịu tải	Hệ thống hỗ trợ tối thiểu 100 người dùng đồng thời

NFR-03	Độ ổn định	Hệ thống hoạt động liên tục và hạn chế lỗi gián đoạn
NFR-04	Bảo mật	Không để lộ dữ liệu người dùng và thông tin hội thoại

2.1.5 Kiến trúc tổng thể hệ thống chatbot

Hệ thống Chatbot nhà xe Vũ Hán được thiết kế theo kiến trúc phân lớp, phân tách rõ ràng giữa giao diện người dùng, logic nghiệp vụ, tầng xử lý thông minh AI và tầng dữ liệu lưu trữ.

Kiến trúc tổng thể của hệ thống bao gồm 4 thành phần chính được mô tả theo luồng dữ liệu dưới đây:

UI Web Chat → API Backend GateWay → AI/NLP Engine → Knowledge Base/Database

- **Lớp Giao diện (UI Web Chat):** Widget trò chuyện nhúng trực tiếp trên Website/Landing Page của nhà xe Vũ Hán, có nhiệm vụ tiếp nhận chuỗi ký tự nhập vào từ hành khách và hiển thị kết quả phản hồi.
- **Lớp Nghiệp vụ (API Backend):** Đóng vai trò bộ điều phối (Orchestrator). Lớp này tiếp nhận request từ UI, điều hướng qua tầng AI để hiểu ý định, nhận kết quả xử lý rồi gọi xuống Cơ sở dữ liệu để thực hiện các logic nghiệp vụ.
- **Lớp Xử lý thông minh (AI/NLP Engine):** Chứa bộ phân loại Ý định (Intent Classification), trích xuất Thực thể (Entity Extraction) hoặc mô hình RAG kết hợp LLM để xử lý và sinh câu phản hồi tự nhiên dựa trên ngữ cảnh hội thoại.
- **Lớp Dữ liệu (Knowledge Base & Database):** Gồm hệ quản trị cơ sở dữ liệu quan hệ (lưu lịch trình, sơ đồ ghế, thông tin khách hàng, trạng thái thanh toán) và Cơ sở tri thức (lưu trữ văn bản chính sách, quy định của nhà xe).

2.1.6 Phân tích API phục vụ kiểm thử

Hệ thống Chatbot nhà xe Vũ Hán áp dụng kiến trúc hội thoại tập trung, nơi mọi yêu cầu nghiệp vụ đều được điều phối thông qua một endpoint duy nhất. Cách tiếp cận

này giúp tối ưu hóa việc quản lý trạng thái phiên làm việc (sessionId), cho phép Chatbot hiểu sâu ngữ cảnh hội thoại của người dùng.

- API Endpoint: /api/chat/stream
- Method: POST
- Request body:

```
{
  "sessionId": "session-1779695036444",
  "message": "Tôi muốn đặt vé xe từ Mỹ Đình"
}
```

- Response mong đợi (Status: 200 OK):

```
data: {"type":"done","content":"Dạ anh/chị cho em xin thêm vài thông tin để đặt vé ạ:\n\n- Điểm đến\n- Họ tên\n- Số điện thoại\n- Ngày đi\n- Giờ muốn đi\n- Số lượng vé\n\nAnh/chị nhấn theo mẫu giúp em nhé:\n- Tên:\n- SĐT:\n- Điểm đến:\n- Ngày đi:\n- Giờ đi:\n- Số vé:
ạ","data":{"intent":"booking","bookingData":{"pickup":"Mỹ Đình","dropoff":"","vehicle_type":"giuong","status":"incomplete","missing_fields":["customer_name","phone_number","departure_date","departure_time","ticket_count"]}}}
```

2.2 Lập kế hoạch kiểm thử

2.2.1 Mục tiêu kiểm thử

Mục tiêu chính của quá trình kiểm thử là đánh giá chất lượng của hệ thống chatbot nhà xe Vũ Hán, đảm bảo sản phẩm vận hành ổn định, chính xác và đáp ứng đầy đủ các yêu cầu nghiệp vụ đã đề ra. Cụ thể, các mục tiêu chi tiết bao gồm:

- **Xác minh khả năng nhận diện ý định (Intent Recognition):** Đảm bảo chatbot nhận diện chính xác các nhu cầu của khách hàng như: đặt vé xe limousine, đặt vé xe khách, tra cứu điểm đón trả, hỗ trợ gửi hàng và giải đáp các câu hỏi thường gặp (FAQ).
- **Kiểm chứng tính chính xác của dữ liệu trích xuất (Entity Extraction):** Đảm bảo các thông tin quan trọng trong hội thoại như loại xe, thời gian, địa điểm và thông tin cá nhân được hệ thống thu thập đúng và đầy đủ.

- **Đánh giá độ bao phủ nghiệp vụ (Functional Coverage):** Xây dựng bộ test case bao phủ tối thiểu 80% các tình huống hội thoại phổ biến và các kịch bản ngoại lệ nhằm phát hiện sớm các lỗi logic trong luồng phản hồi.
- **Đảm bảo hiệu năng và khả năng chịu tải:** Xác định ngưỡng chịu tải của hệ thống, đảm bảo chatbot hoạt động ổn định và có thời gian phản hồi đạt yêu cầu khi có tối thiểu 100 người dùng truy cập đồng thời.
- **Nâng cao tính an toàn và bảo mật:** Phát hiện và phân tích các lỗ hổng bảo mật phổ biến thông qua kiểm thử bảo mật cơ bản, đảm bảo an toàn dữ liệu cho người dùng và hệ thống.
- **Tối ưu hóa quy trình kiểm thử thông qua tự động hóa:** Triển khai giải pháp kiểm thử tự động (Automation Test) để giảm thiểu sai sót do con người, rút ngắn thời gian kiểm thử định kỳ và tích hợp vào quy trình CI/CD.
- **Cung cấp bằng chứng khách quan về chất lượng phần mềm:** Thông qua các báo cáo kiểm thử (Test Report) và đo lường độ bao phủ (Coverage), cung cấp cái nhìn tổng quan và trung thực về mức độ hoàn thiện của những trường hợp kiểm thử.

2.2.2 Phạm vi kiểm thử

Phạm vi kiểm thử sẽ xác định rõ ràng các ranh giới kiểm thử, bao gồm các thành phần hệ thống sẽ được kiểm tra và các khía cạnh chất lượng được ưu tiên, nhằm tối ưu hóa nguồn lực và đảm bảo tính bao phủ của những trường hợp kiểm thử.

2.2.2.1 Các thành phần và chức năng kiểm thử

Phạm vi kiểm thử chức năng bao gồm:

- **Nhận diện ý định (Intent Identification):** Kiểm thử khả năng phân loại chính xác các yêu cầu từ người dùng như đặt vé, hỏi giá, tìm điểm đón trả, hoặc yêu cầu gặp nhân viên hỗ trợ.
- **Quản lý luồng hội thoại (Conversation Flow):** Kiểm tra tính logic của các kịch bản hội thoại nhiều nhánh, đảm bảo chatbot dẫn dắt người dùng đi đúng quy trình đặt vé limousine và xe khách.
- **Trích xuất thực thể (Entity Extraction):** Kiểm thử độ chính xác khi hệ thống thu thập các thông tin biến đổi như: thời gian khởi hành, số lượng hành khách, loại xe quan tâm và số điện thoại liên hệ.
- **Giải đáp FAQ và hỗ trợ:** Kiểm tra kho dữ liệu câu hỏi thường gặp và khả năng phản hồi thông tin về dịch vụ gửi hàng.

2.2.2.2 Các mức kiểm thử (Test Levels)

Quá trình kiểm thử được triển khai qua các cấp độ để đảm bảo chất lượng từ thành phần nhỏ nhất đến toàn bộ hệ thống:

- **Unit Testing (Kiểm thử đơn vị):** Tập trung kiểm tra các hàm xử lý logic nghiệp vụ, các API xử lý dữ liệu đầu vào và các module trích xuất thông tin.
- **Integration Testing (Kiểm thử tích hợp):** Kiểm tra sự tương tác giữa chatbot với các hệ thống bên thứ ba, cơ sở dữ liệu và các API tích hợp (như API quản lý vé hoặc hệ thống gửi tin nhắn).
- **System Testing (Kiểm thử hệ thống):** Kiểm thử hệ thống chatbot trên môi trường Vercel để đánh giá sự phối hợp giữa giao diện người dùng (UI) và logic xử lý phía sau.

2.2.2.3 Các loại kiểm thử phi chức năng

- **Kiểm thử hiệu năng (Performance Testing):** Thực hiện giả lập tối thiểu 100 người dùng truy cập đồng thời bằng công cụ JMeter để đánh giá thời gian phản hồi và độ ổn định của hệ thống.
- **Kiểm thử bảo mật (Security Testing):** Sử dụng công cụ OWASP ZAP để quét và phát hiện các lỗ hổng phổ biến, đảm bảo tính an toàn cho luồng trao đổi thông tin giữa khách hàng và nhà xe.

2.2.2.4 Các thành phần nằm ngoài phạm vi (Out-of-Scope)

- Kiểm thử phần cứng hoặc hạ tầng máy chủ vật lý phía sau nền tảng đám mây.
- Kiểm thử các tính năng thanh toán trực tiếp.

2.2.3 Nguồn lực và môi trường kiểm thử

Để quá trình kiểm thử được diễn ra đồng bộ và đạt hiệu quả cao, việc xác định rõ ràng các nguồn lực về công cụ và thiết lập môi trường kiểm thử là yêu cầu bắt buộc.

2.2.3.1 Nhân sự và vai trò thực hiện kiểm thử

Nhân sự: Nguyễn Thị Vân, đảm nhiệm toàn bộ các vai trò trong vòng đời kiểm thử phần mềm (STLC) của dự án Chatbot Vũ Hán. Quy trình vận hành được phân định theo các vai trò chuyên môn như sau:

- **Test Leader:** Khảo sát tri thức nhà xe, lập kế hoạch, phân tích rủi ro hệ thống và phê duyệt tiêu chí đóng/mở kiểm thử.
- **Automation Tester:** Chuyển đổi các kịch bản kiểm thử từ file dữ liệu thành mã nguồn kiểm thử tự động bất đồng bộ bằng công cụ Playwright.

- **Performance & Security Tester:** Thiết lập kịch bản giả lập tải trên JMeter và cấu hình quét lỗ hổng ứng dụng bằng OWASP ZAP.
- **DevOps Engineer:** Cấu hình tệp luồng công việc tự động trên GitHub Actions để chạy CI/CD Pipeline.

2.2.3.2 Công cụ và môi trường kiểm thử

- Môi trường kiểm thử (Test Environment): Môi trường Staging triển khai thực tế trên Vercel: <https://chatbot-nhaxevuhan.vercel.app/chat>
- Playwright (v1.59.1): Xây dựng và thực thi Automation Test giao diện và hội thoại.
- Apache JMeter (v5.6.3): Kiểm thử hiệu năng, đo lường thời gian phản hồi (Response Time).
- OWASP ZAP (v2.17.0): Quét bảo mật tự động thông qua giao thức API.
- SonarQube (v2025.1): Phân tích tĩnh, quản lý chất lượng và độ sạch của mã nguồn.

2.3 Chiến lược kiểm thử

2.3.1 Các giai đoạn thực hiện kiểm thử

Giai đoạn 1: Phân tích yêu cầu và Kiểm thử tĩnh (Static Testing)

Khảo sát kho tri thức nhà xe Vũ Hán, xác định 8 luồng chức năng chính của chatbot. Cấu hình SonarQube để quét toàn bộ mã nguồn dự án, đảm bảo mã nguồn sạch, không chứa lỗi logic nghiêm trọng trước khi kiểm thử động.

Giai đoạn 2: Kiểm thử chức năng tự động (Automation Functional Testing)

Xây dựng Framework Playwright theo mô hình POM. Viết thuật toán đối soát từ khóa thông minh để đọc dữ liệu tự động từ file CSV và gửi/kiểm tra hội thoại với Chatbot.

Giai đoạn 3: Kiểm thử phi chức năng (Non-Functional Testing)

Thiết lập cấu hình Thread Group trên Apache JMeter để giả lập 100 người dùng đồng thời nhấn tin tới Chatbot, kiểm tra thời gian phản hồi. Đồng thời, cấu hình OWASP ZAP thực hiện Active Scan để tìm kiếm các lỗ hổng bảo mật Web.

Giai đoạn 4: Tích hợp hệ thống và Nghiệm thu (CI/CD Integration & Exit)

Kết nối toàn bộ chuỗi công cụ vào GitHub Actions Pipeline, chạy nghiệm thu và kết xuất báo cáo tổng hợp.

2.3.2 Chiến lược kiểm thử dựa trên rủi ro (Risk-Based Testing)

Chiến lược kiểm thử dựa trên rủi ro giúp tập trung nguồn lực vào các tính năng chính có tần suất sử dụng cao hoặc có mức độ ảnh hưởng lớn đến trải nghiệm khách hàng. Mức độ rủi ro được xác định dựa trên công thức:

$$\text{Mức độ rủi ro} = \text{Khả năng xảy ra lỗi} \times \text{Mức độ ảnh hưởng}$$

a. Ma trận phân tích rủi ro hệ thống Chatbot Vũ Hán

Bảng 2.3 Ma trận phân tích rủi ro

Mã rủi ro	Chi tiết rủi ro kỹ thuật / nghiệp vụ	Khả năng xảy ra	Mức độ ảnh hưởng	Mức độ rủi ro
R-01	Chatbot nhận diện sai Intent của khách hàng	Medium	High	Critical
R-02	Phản hồi sai thông tin văn phòng/liên hệ hoặc sai giá vé, tuyến đường của các tuyến	Low	High	High
R-03	Hệ thống bị treo, phản hồi chậm (>5 giây) khi có nhiều hành khách cùng truy cập đặt vé vào dịp cao điểm.	High	Medium	High
R-04	Lộ thông tin/số điện thoại, điểm đón trả của hành khách do lỗ hổng bảo mật Web	Low	Critical	High
R-05	Chatbot trả lời sai lệch thông tin chính sách FAQ	Medium	High	Medium

b. Chiến lược giảm thiểu rủi ro tương ứng

- **Đối với rủi ro Nghiêm trọng và Cao (R-01, R-02):** Ưu tiên thiết lập mức độ ưu tiên High cho các kịch bản kiểm thử thuộc nhóm Nhận diện Intent, Đặt vé xe Limousine và Đặt vé xe khách.
- **Đối với rủi ro Hiệu năng và Bảo mật (R-03, R-04):** Triển khai bắt buộc kiểm thử phi chức năng với JMeter (giả lập 100 người dùng đồng thời) và OWASP ZAP để giảm thiểu tối đa nguy cơ nghẽn mạng hoặc rò rỉ dữ liệu.

- Đối với **R-05**: Xây dựng bộ Test Case bao phủ đầy đủ các câu hỏi thường gặp nhằm tăng cường độ chính xác cho AI Agent.

2.3.3 Tiêu chí và điều kiện kiểm thử

a. Điều kiện bắt đầu kiểm thử

- Toàn bộ tài liệu đặc tả yêu cầu nghiệp vụ, kho dữ liệu tri thức của nhà xe Vũ Hán và luồng phản hồi của 08 nhóm chức năng chính đã được xác định.
- Hệ thống mã nguồn Chatbot đã hoàn thiện các module xử lý logic cơ bản và được triển khai thành công trên môi trường Staging của nền tảng đám mây Vercel, đảm bảo giao diện khung chat hiển thị đầy đủ (ô nhập liệu, nút gửi).
- Thiết kế hoàn chỉnh bộ kịch bản kiểm thử bao gồm 08 Test Scenarios tổng quát và Test Case chi tiết.
- Hệ sinh thái các công cụ kiểm thử bao gồm SonarQube, Playwright, Apache JMeter, OWASP ZAP và công cụ quản lý lỗi Jira đã được cài đặt, cấu hình môi trường, khởi tạo dự án và kiểm tra kết nối thành công, sẵn sàng nhận lệnh thực thi.

b. Điều kiện kết thúc kiểm thử

- **Độ bao phủ kịch bản:** 100% các trường hợp kiểm thử chức năng tự động được framework Playwright thực thi đầy đủ trên hệ thống, không bỏ sót bất kỳ kịch bản nào.
- **Tỷ lệ vượt qua (Pass Rate):** Tỷ lệ Pass tổng thể của các kịch bản chức năng phải đạt tối thiểu 80%. Riêng các kịch bản thuộc nhóm có mức độ rủi ro Cao (High) như Đặt vé xe limousine, Đặt vé xe khách và Tra cứu thông tin văn phòng/liên hệ bắt buộc phải đạt tỷ lệ Pass 100%.
- **Chất lượng mã nguồn tĩnh:** Công cụ SonarQube báo cáo mã nguồn Chatbot đạt trạng thái vượt qua cổng chất lượng.
- **Chỉ số hiệu năng kế hoạch:** Bài kiểm thử tải trên JMeter ghi nhận hệ thống vận hành ổn định, tỷ lệ lỗi kết nối (Error Rate) đạt 0% dưới điều kiện giả lập tối thiểu 100 người dùng đồng thời, thời gian phản hồi < 5s.
- **Chỉ số an toàn bảo mật:** Công cụ OWASP ZAP hoàn thành chu kỳ quét tự động và không phát hiện lỗ hổng bảo mật nào ở mức độ nguy hiểm (High/Critical).

c. Tiêu chí pass/fail

- **Tiêu chí PASS (Đạt):** Được ghi nhận khi công cụ tự động gửi câu lệnh đầu vào (Input) và Chatbot phản hồi nội dung văn bản chứa từ khóa (Keywords) cốt lõi,

nhận diện đúng ý định (Intent) và trích xuất chính xác thực thể (Entity) trùng khớp với Kết quả mong đợi (Expected Result).

- **Tiêu chí FAIL (Thất bại):** Được ghi nhận khi Chatbot nhận diện sai Intent; trích xuất sai lệch thông tin thực thể (sai hotline, sai giờ chạy, sai giá vé); hệ thống không phản hồi, trả về các mã lỗi HTTP (404, 500) hoặc thời gian phản hồi câu thoại vượt quá ngưỡng chờ tối đa (Timeout > 15 giây).

d. Tiêu chí dừng kiểm thử

- **Sự cố môi trường:** Môi trường Vercel Staging hoặc API kết nối cơ sở dữ liệu tri thức của Chatbot gặp sự cố sập mạng, mất kết nối liên tục và không thể thực hiện hội thoại.
- **Lỗi nghiêm trọng hàng loạt:** Phát hiện lỗi mã nguồn nghiêm trọng (Blocker) ngay từ bước quét tĩnh SonarQube hoặc lỗi logic hệ thống trong quá trình chạy Playwright, dẫn đến trên **30%** tổng số lượng Test Cases không thể tiếp tục thực thi hoặc liên tục trả về kết quả lỗi Fail.
- **Quy định khôi phục (Resumption Criteria):** Kịch bản kiểm thử chỉ được kích hoạt chạy lại từ đầu sau khi các lập trình viên đã tiến hành sửa đổi mã nguồn, cấu hình lại hạ tầng server hoạt động ổn định và kiểm tra thủ công xác nhận hệ thống đã sẵn sàng.

2.3.4 Chiến lược quản lý lỗi và dữ liệu kiểm thử

a. Quy trình quản lý và phân loại lỗi (Defect Management)

- Toàn bộ các lỗi (defects) hoặc lỗ hổng phát hiện trong quá trình quét mã nguồn tĩnh, kiểm thử tự động giao diện và kiểm thử phi chức năng sẽ được ghi nhận tập trung trên công cụ Jira.
- Mỗi bản ghi lỗi trên Jira bắt buộc phải đi kèm đầy đủ các thông tin: ID kịch bản lỗi, các bước tái hiện, dữ liệu đầu vào (Input), kết quả thực tế, mức độ nghiêm trọng và ảnh chụp màn hình hội thoại lỗi để hỗ trợ lập trình viên khắc phục. Lỗi sau khi được sửa đổi sẽ được kiểm thử lại (Re-testing) trước khi đóng.

b. Chiến lược quản lý dữ liệu kiểm thử (Test Data Strategy)

- **Áp dụng mô hình Kiểm thử hướng dữ liệu (Data-Driven Testing):** Kế hoạch định hướng tách biệt hoàn toàn phần mã nguồn viết kịch bản tự động (Playwright) và phần dữ liệu hội thoại.
- **Chiến lược phân loại và chuẩn bị tập dữ liệu đầu vào:** Để kiểm tra khả năng xử lý ngôn ngữ tự nhiên (NLP) của Chatbot, kế hoạch phân chia dữ liệu đầu vào thành

hai tập mẫu là Tập dữ liệu chuẩn mực (Happy Cases) và Tập dữ liệu biến thể/Biên (Negative Cases).

2.4 Thiết kế kịch bản và bộ Test Case chi tiết

2.4.1 Nhóm chức năng nhận diện Intent

Test Scenario: TS_INTENT_01

Tên kịch bản: Kiểm tra khả năng nhận diện ý định (Intent)

Tiền điều kiện: Khách hàng truy cập vào giao diện Web Chatbot, hệ thống ở trạng thái sẵn sàng kết nối API.

Mục tiêu: Đảm bảo chatbot phân loại đúng intent của khách (hỏi giá, hỏi lịch, đặt vé, FAQ, gửi hàng...)

Từ kịch bản tổng quát này, chúng ta có thể cụ thể hóa thành các trường hợp kiểm thử (Test Cases) chi tiết hơn dưới đây.

Bảng 2.4 Test Case nhận diện Intent

ID	Tiêu đề	Mô tả kịch bản	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Ghi chú
TC_01	Nhận diện intent hỏi giá vé	1. Mở chatbot 2. Gửi câu hỏi về giá 3. Quan sát phản hồi	“Vé Hà Nội đi Mèo Vạc bao nhiêu tiền?”	Xác định intent hỏi giá vé, báo giá vé là 400.000đ	“Dạ tuyến Hà Nội (Mỹ Đình) → Mèo Vạc giá vé 400.000đ ạ, xe giường 40 chỗ. ”	Pass
TC_02	Nhận diện intent hỏi lịch	1. Mở chatbot 2. Gửi câu hỏi về giờ chạy	“Xe đi Xín Mần mấy giờ?”	Xác định intent lịch chạy, báo giờ chạy	“Dạ từ Hà Nội đi Xín Mần có các chuyến sau ạ: VIP 9 chỗ: 05:30 — thời gian đi khoảng 8 tiếng	Pass

					Giường 40 chỗ: 10:00 — thời gian đi khoảng 8 tiếng Giường 40 chỗ: 19:20 — thời gian đi khoảng 8 tiếng”	
TC_03	Nhận diện intent đặt vé	1. Mở chatbot 2. Gửi yêu cầu đặt vé	“Tôi muốn đặt vé đi Đồng Văn”	Xác định intent đặt vé, hỏi thông tin đặt vé	“Dạ em cần anh/chị cho em xin thêm một số thông tin để đặt vé ạ: Họ tên Số điện thoại Ngày đi Giờ muốn đi Loại xe: giường / ghế / VIP Số lượng vé”	Pass
TC_04	Nhận diện intent FAQ	1. Mở chatbot 2. Hỏi câu hỏi thường gặp	“Giá vé trẻ em thế nào?”	Xác định intent FAQ, trả lời về chính sách trẻ em	“Dạ giá vé trẻ em bên em như sau ạ: Dưới 1,1m: miễn phí Từ 1,1m đến 1,4m: 50% giá vé niêm yết Trên 1,4m: tính như người lớn ạ”	Pass
TC_05	Nhận diện	1. Mở chatbot	“Tôi muốn gửi hàng từ	Xác định intent gửi hàng, hướng	“Dạ anh/chị mang hàng ra văn phòng gần nhất để gửi ạ:	

	intent gửi hàng	2. Hỏi về gửi hàng	Hà Nội đi Tuyên Quang”	đẫn gửi tại văn phòng	Văn phòng Mỹ Đình: N5 A2 Ngõ 1 Nguyễn Hoàng, mặt sau bên xe Mỹ Đình Nhận hàng đến 20:00 SĐT: 0912 037 237 Lưu ý: Cước gửi hàng tính theo trọng lượng và kích thước, bên em cần xem hàng trực tiếp để báo giá chính xác ạ.”	Pass
TC_06	Nhận diện intent hỏi văn phòng	1. Mở chatbot 2. Hỏi địa chỉ văn phòng	“Cho hỏi văn phòng Mỹ Đình ở đâu?”	Xác định intent tra thông tin, trả về địa chỉ và SĐT	“Dạ văn phòng Mỹ Đình của Nhà xe Vũ Hán ở: Địa chỉ: N5 A2 Ngõ 1 Nguyễn Hoàng, mặt sau bên xe Mỹ Đình SĐT: 0912 037 237 Giờ làm việc: Đến 20:00 ạ”	Pass
TC_07	Nhận diện intent yêu cầu	1. Mở chatbot 2. Yêu cầu gặp	“Cho tôi gặp nhân viên”	Chatbot phản hồi chuyển CSKH	“Dạ e đã tiếp nhận thông tin. Anh chị chờ giây lát em sẽ chuyển qua bộ	Pass

	nhân viên	nhân viên			phận chuyên trách xử lý ạ”	
TC_08	Nhận diện intent điểm đón/trả	1. Mở chatbot 2. Hỏi điểm đón	“Xe đón ở đâu Hà Nội?”	Chatbot gợi ý điểm đón ở Hà Nội	“Một số điểm đón phổ biến ở Hà Nội: Mỹ Đình Ngã 4 Nội Bài Các điểm hẹn theo tuyến”	Pass
TC_09	Nhận diện xe ghế	1. Mở chatbot 2. Yêu cầu tư vấn xe ghế	“Hà Nội – Tuyên Quang có vé ghế không”	Xác nhận đúng loại xe	“Dạ có vé ghế ạ, tuyến Hà Nội → Tuyên Quang bên em chạy bằng xe ghế 29 chỗ với các giờ: 06:20, 06:55, 07:50, 15:25, 17:50”	Pass
TC_10	Nhận diện xe giường nằm	1. Mở chatbot 2. Hỏi xe	“Đi Na Hang có xe giường không”	Xác nhận đúng loại xe	“Dạ có ạ, tuyến đi Na Hang có xe giường 40 chỗ.”	Pass

		giường nằm				
TC_11	Nhận diện tuyến không hỗ trợ loại xe	1. Mở chatbot 2. Hỏi tuyến không hỗ trợ loại xe	“Tôi muốn đặt vé xe VIP Hà Nội đi Lu”	Báo không có loại xe	“Em lưu ý giúp anh/chị: tuyến này hiện hệ thống đang hiển thị xe giường 40 chỗ, chưa thấy loại VIP limousine cho điểm đến Lu ạ.”	Pass

2.4.2 Nhóm chức năng tra cứu giá vé

Test Scenario: TS_PRICE_02

Tên kịch bản: Kiểm tra chức năng tra cứu giá vé

Tiền điều kiện: Khách hàng truy cập vào giao diện Web Chatbot, hệ thống ở trạng thái sẵn sàng kết nối API, cơ sở dữ liệu bảng giá của toàn bộ tuyến đường đã có trong tri thức.

Mục tiêu: Đảm bảo chatbot trả về đúng giá vé cho các tuyến hợp lệ, xử lý đúng tuyến không tồn tại.

Từ kịch bản tổng quát này, chúng ta có thể cụ thể hóa thành các trường hợp kiểm thử (Test Cases) chi tiết hơn dưới đây.

Bảng 2.5 Test Case tra cứu giá vé

ID	Tiêu đề	Mô tả kịch bản	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Ghi chú
TC_12	Hỏi giá tuyến hợp lệ - Hà Nội đi	1. Mở chatbot 2. Gửi câu hỏi về giá Hà Nội – Mèo Vạc	“Vé Hà Nội đi Mèo Vạc bao nhiêu?”	Trả về giá vé chính xác ($\approx 400k$), loại xe phù hợp	“Dạ tuyến Hà Nội (Mỹ Đình) → Mèo Vạc giá vé 400.000đ cho xe	Pass

	Mèo Vạc				giường 40 chỗ ạ.”	
TC_13	Hỏi giá tuyến hợp lệ - chiều ngược	1. Mở chatbot 2. Gửi câu hỏi về giá vé Mèo Vạc – Hà Nội	“Vé Mèo Vạc về Hà Nội bao nhiêu?”	Trả về giá vé chính xác (chiều về)	“Dạ tuyến Mèo Vạc → Hà Nội (Mỹ Đình) giá vé 400.000đ ạ.”	Pass
TC_14	Hỏi giá tuyến không tồn tại	1. Mở chatbot 2. Gửi câu hỏi về giá vé Thái Nguyên – Hà Nội	“Vé Thái Nguyên đi Hà Nội bao nhiêu?”	Thông báo "chưa có thông tin tuyến", yêu cầu để lại SĐT	“Dạ em chưa có thông tin giá vé tuyến này ạ. Anh/chị để lại SĐT để bên em liên hệ kiểm tra và báo lại nhé ạ.”	Pass
TC_15	Hỏi giá với điểm ngoài vùng "TP Lào Cai"	1. Mở chatbot 2. Hỏi giá đi TP Lào Cai	“Đi TP Lào Cai bao nhiêu?”	Thông báo "Xe không vào TP Lào Cai", gợi ý xuống Lu	“Dạ xe không vào TP Lào Cai ạ. Anh/chị xuống điểm Lu đón xe giúp em, xe qua Lu khoảng 15h	Pass

					hoặc 12h đêm ạ.”	
TC_16	Hỏi giá xe VIP cụ thể	1. Mở chatbot 2. Hỏi giá xe limousine Hà Nội – Hoàng Su Phì	“Tôi muốn đặt vé xe limousine đi từ Hà Nội đến Hoàng Su Phì”	Trả về giá xe VIP, không trả giá xe giường	“Dạ tuyến Hà Nội (Mỹ Đình) → Hoàng Su Phì-Vinh Quang (thị trấn) giá vé VIP 9 chỗ là 300.000đ ạ.”	Pass
TC_17	Hỏi điểm đón/ trả	1. Mở chatbot 2. Hỏi điểm đón ở Vĩnh Phúc	“Đón ở Vĩnh Phúc được không?”	Gợi ý KM14/KM25/KM41	“Dạ được ạ. Với Vĩnh Phúc, anh/chị vui lòng ra nút giao KM14, KM25 hoặc KM41 chỗ nào gần nhất để đón xe ạ.”	Pass

2.4.3 Nhóm chức năng tra cứu lịch chạy

Test Scenario: TS_SCHEDULE_03

Tên kịch bản: Kiểm tra chức năng tra cứu lịch chạy

Tiền điều kiện: Khách hàng truy cập vào giao diện Web Chatbot, hệ thống ở trạng thái sẵn sàng kết nối API, toàn bộ khung giờ xuất bến, giờ chạy qua các nút giao trong ngày của nhà xe đã được đồng bộ cố định.

Mục tiêu: Đảm bảo chatbot trả về lịch chính xác cho cả chiều đi và chiều về

Từ kịch bản tổng quát này, chúng ta có thể cụ thể hóa thành các trường hợp kiểm thử (Test Cases) chi tiết hơn dưới đây.

Bảng 2.6 Test Case tra cứu lịch chạy

ID	Tiêu đề	Mô tả kịch bản	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Ghi chú
TC_18	Hỏi lịch chạy tuyến phổ biến	- Mở chatbot - Gửi câu hỏi về giờ chạy Hà Nội – Xín Mần	“Từ Hà Nội xe đi Xín Mần mấy giờ?”	Trả về danh sách giờ chạy (VD: 5:30, 10:00, 19:20), gộp các chuyến cùng loại	“Dạ từ Hà Nội đi Xín Mần có các chuyến ạ: 05:30 – VIP 9 chỗ – thời gian đi khoảng 8 tiếng 10:00 – Xe giường 40 chỗ – thời gian đi khoảng 8 tiếng 19:20 – Xe giường 40 chỗ – thời gian đi khoảng 8 tiếng”	Pass
TC_19	Hỏi lịch chiều ngược lại	- Mở chatbot - Gửi câu hỏi về giờ chạy Xín Mần – Hà Nội	“Từ Xín Mần về Hà Nội mấy giờ?”	Trả về lịch chạy chính xác (chiều về)	“Dạ từ Xín Mần về Hà Nội có các chuyến: Xe giường 40 chỗ:	Pass

					08:20, 19:30 Xe VIP 9 chỗ: 13:00”	
TC_ 20	Hỏi lịch tuyến không tồn tại	1. Mở chatbot 2. Gửi câu hỏi về lịch chạy Hà Nội – Đà Nẵng	“Xe từ Hà Nội đi Đà Nẵng mấy giờ?”	Phản hồi chưa có tuyến	“Dạ hiện bên em chưa có tuyến Hà Nội đi Đà Nẵng trong hệ thống ạ.”	Pass
TC_ 21	Hỏi lịch theo loại xe cụ thể	1. Mở chatbot 2. Hỏi lịch chạy xe limousine	“Xe VIP từ Hà Nội đi Tuyên Quang mấy giờ?”	Chỉ trả về xe VIP, báo giờ chính xác	“Dạ tuyến VIP 9 chỗ Hà Nội → Tuyên Quang hiện có chuyến 05:30 ạ.”	Pass
TC_ 22	Hỏi thời gian di chuyển	1. Mở chatbot 2. Hỏi thời gian di chuyển Hà Nội – Vĩnh Phúc	“Từ Hà Nội đi Vĩnh Phúc mất bao lâu”	Phản hồi thời gian khoảng 1h30 – 2h	“Dạ tuyến Hà Nội đi Vĩnh Phúc bên em đi khoảng 1 tiếng 30 phút đến 2 tiếng ạ, tùy điểm trả và tình hình đường đi.”	Pass
TC_ 23	Hỏi giờ đón/ trả	1. Mở chatbot 2. Hỏi giờ đón tại Đoàn Hùng	“Đoan Hùng đi Yên Hoa	Phản hồi khoảng 22h30	“Dạ chuyến từ Đoàn Hùng đi	Pass

			đón mấy giờ?”		Yên Hoa xe đón khoảng 22h30 ạ.”	
--	--	--	---------------	--	---------------------------------	--

2.4.4 Nhóm chức năng đặt vé

Test Scenario: TS_BOOKING_04

Tên kịch bản: Kiểm tra chức năng đặt vé

Tiền điều kiện: Khách hàng truy cập vào giao diện Web Chatbot, hệ thống ở trạng thái sẵn sàng kết nối API.

Mục tiêu: Đảm bảo thu thập đủ thông tin, lưu booking thành công, validate dữ liệu đầu vào

Từ kịch bản tổng quát này, chúng ta có thể cụ thể hóa thành các trường hợp kiểm thử (Test Cases) chi tiết hơn dưới đây.

Bảng 2.7 Test Case đặt vé

ID	Tiêu đề	Mô tả kịch bản	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Ghi chú
TC_24	Đặt vé thành công đầy đủ thông tin	1. Mở chatbot 2. Gửi yêu cầu đặt vé đầy đủ thông tin trong 1 tin nhắn 3. Xác nhận	“Đặt 2 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321”	Phản hồi đã ghi nhận thông tin, hướng dẫn thanh toán giữ chỗ.	“Dạ em xác nhận đặt vé cho anh/chị ạ. Lái phụ xe sẽ liên hệ trước 1-2 tiếng để hẹn điểm đón ạ.”	Pass
TC_25	Đặt vé thiếu tên khách hàng	1. Mở chatbot 2. Gửi đặt vé thiếu tên	“Đặt 2 vé giường nằm Hà Nội đi Tuyên Quang ngày	Chatbot hỏi lại tên	“Dạ còn thiếu họ tên của anh/chị ạ. Anh/chị gửi em tên để em hoàn tất đặt 2	Pass

			22/05/2026 chuyến 18:30, Số điện thoại: 0987654321”		vé giường tuyến Hà Nội đi Tuyên Quang lúc 18:30 ngày 22/05/2026 nhé.”	
TC_ 26	Đặt vé thiếu số điện thoại	1. Mở chatbot 2. Gửi đặt vé thiếu số điện thoại	“Đặt 2 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20, tên Nguyễn Văn A”	Chatbot hỏi lại số điện thoại	“Dạ anh/chị vui lòng cho em xin số điện thoại liên hệ để em tiếp tục giữ chỗ 2 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20 ạ.”	Pass
TC_ 27	Đặt vé số lượng vé = 1	1. Mở chatbot 2. Đặt 1 vé	“Đặt 1 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321”	Đặt thành công	“Dạ em xác nhận đặt vé cho anh/chị ạ. Lái phụ xe sẽ liên hệ trước 1-2 tiếng để hẹn điểm đón ạ.”	Pass
TC_ 28	Đặt vé số lượng	1. Mở chatbot 2. Đặt 40 vé giường	“Đặt 40 vé giường nằm Hà Nội đi Xín Mần ngày	Đặt thành công (đúng sức chứa xe	“Dạ em xác nhận đặt vé cho anh/chị ạ. Lái phụ xe sẽ	

	vé = 40		20/05/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321”	giường 40 chỗ)	liên hệ trước 1-2 tiếng để hẹn điểm đón ạ.”	Pass
TC_ 29	Đặt vé số lượng vé = 0	1. Mở chatbot 2. Đặt 0 vé	“Đặt 0 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321”	Báo lỗi "Số vé phải lớn hơn 0" hoặc hỏi lại	“Dạ em chưa thể giữ chỗ với số lượng vé là 0 ạ.”	Pass
TC_30	Đặt vé số lượng vé âm	1. Mở chatbot 2. Đặt -3 vé	“Đặt -3 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321”	Báo lỗi, không lưu booking	“Dạ số lượng vé không hợp lệ, anh/chị vui lòng nhập lại số lượng lớn hơn 0 ạ.”	Pass
TC_31	Đặt vé số lượng vé > sức chứa	1. Mở chatbot 2. Đặt 50 vé giường	“Đặt 50 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20, tên Nguyễn	Báo lỗi "Vượt sức chứa" hoặc đề xuất chia chuyến	“Dạ số lượng 50 vé vượt quá sức chứa của 1 xe giường nằm, tối đa 40 chỗ ạ.”	Pass

			Văn A, Số điện thoại: 0987654321”			
TC_32	Đặt vé SĐT sai định dạng	1. Mở chatbot 2. Đặt vé với SĐT chứa chữ	“Đặt 2 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 098abc4321”	Báo lỗi "SĐT không hợp lệ", yêu cầu nhập lại	“Dạ số điện thoại anh/chị nhập chưa đúng định dạng ạ. Anh/chị vui lòng gửi lại số điện thoại hợp lệ 10 số để em tiếp tục giữ vé nhé.”	Pass
TC_33	Đặt vé ngày trong quá khứ	1. Mở chatbot 2. Đặt vé ngày 01/01/2020	“Đặt 2 vé giường nằm Hà Nội đi Xín Mần ngày 01/01/2020 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321”	Phản hồi ngày đi không hợp lệ, yêu cầu nhập ngày tương lai	“Dạ ngày đi anh/chị chọn đã nằm trong quá khứ nên chưa đặt vé được ạ. Anh/chị vui lòng chọn lại ngày đi từ hôm nay trở đi giúp em nhé.”	Pass
TC_34	Đặt vé ngày không đúng	1. Mở chatbot 2. Đặt vé ngày 31/6/2026	“Đặt 2 vé giường nằm Hà Nội đi Xín Mần ngày 31/06/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện	Phản hồi ngày không đúng, báo khách chọn ngày khác	“Dạ ngày 31/06 không hợp lệ ạ, vì tháng 6 chỉ có 30 ngày. Anh/chị vui lòng chọn	Pass

			thoại: 0987654321”		ngày đi đúng nhé.”	
TC_35	Đặt vé với loại xe sai cho tuyến	1. Mở chatbot 2. Đặt xe VIP đi Đồng Văn (chỉ có xe giường)	“Đặt 1 vé VIP Hà Nội đi Đồng Văn ngày 20/05/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321”	Chatbot giải thích "Đồng Văn chỉ có xe giường", đề xuất loại khác	“Dạ em xác nhận đặt vé cho anh/chị ạ. Lái phụ xe sẽ liên hệ trước 1-2 tiếng để hẹn điểm đón ạ.”	Fail
TC_36	Đặt vé bằng từ "mai"	1. Mở chatbot 2. "Đặt vé đi Tuyên Quang ngày mai"	“Đặt 1 vé giường nằm Hà Nội đi Tuyên Quang ngày mai chuyến 19:30, tên Nguyễn Văn A, Số điện thoại: 0987654321”	Chatbot tự tính ngày mai, không hỏi lại	“Dạ em xác nhận đặt vé cho anh/chị ạ. Lái phụ xe sẽ liên hệ trước 1-2 tiếng để hẹn điểm đón ạ.”	Pass

2.4.5 Kiểm tra alias địa điểm

Test Scenario: TS_ALIAS_05

Tên kịch bản: Kiểm tra alias địa điểm

Tiền điều kiện: Khách hàng truy cập vào giao diện Web Chatbot, hệ thống ở trạng thái sẵn sàng kết nối API.

Mục tiêu: Đảm bảo các tên gọi khác nhau cùng map về một địa điểm chuẩn.

Từ kịch bản tổng quát này, chúng ta có thể cụ thể hóa thành các trường hợp kiểm thử (Test Cases) chi tiết hơn dưới đây.

Bảng 2.8 Test Case Alias địa điểm

ID	Tiêu đề	Mô tả kịch bản	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Ghi chú
TC_37	Alias "Cốc Pài" = Xín Mần	1. Mở chatbot 2. Hỏi giá đi Cốc Pài	"Giá đi Cốc Pài?"	Nhận diện = Xín Mần, phản hồi giá 300k	"Dạ tuyến Hà Nội (Mỹ Đình) → Cốc Pài - Xín Mần / Pà Vây Sử giá vé 300.000đ cho xe giường 40 chỗ ạ."	Pass
TC_38	Alias "Vinh Quang" = Hoàng Su Phì	1. Mở chatbot 2. Nhấn điểm đến là Vinh Quang	"Đi Vinh Quang"	Nhận diện = Hoàng Su Phì	"Dạ tuyến đi Vinh Quang bên em là tuyến Hoàng Su Phì ạ."	Pass
TC_39	Alias "Tam Sơn" = Quản Bạ	1. Mở chatbot 2. Nhấn điểm đến là Tam Sơn	"Đi Tam Sơn"	Nhận diện = Quản Bạ	"Dạ tuyến Hà Nội đi Tam Sơn (Quản Bạ) có xe giường 40 chỗ ạ"	Pass
TC_40	Alias "Pắc Mầu" = Bảo Lâm	1. Mở chatbot 2. Nhấn điểm đến là Pắc Mầu	"Đi Pắc Mầu"	Nhận diện = Bảo Lâm	"Dạ tuyến Hà Nội (Mỹ Đình) → Pắc Mầu - Bảo Lâm giá vé 350.000đ cho xe giường 40 chỗ ạ."	Pass

2.4.6 Câu hỏi thường gặp

Test Scenario: TS_FAQ_06

Tên kịch bản: Kiểm tra chức năng FAQ

Tiền điều kiện: Khách hàng truy cập vào giao diện Web Chatbot, hệ thống ở trạng thái sẵn sàng kết nối API, tập dữ liệu tri thức về chính sách ưu đãi, hành lý, quy định an toàn của nhà xe đã được cấu hình hoàn thiện.

Mục tiêu: Đảm bảo trả lời đúng các câu hỏi thường gặp về giá trẻ em, giảm giá, thẻ đi lại...

Từ kịch bản tổng quát này, chúng ta có thể cụ thể hóa thành các trường hợp kiểm thử (Test Cases) chi tiết hơn dưới đây.

Bảng 2.9 Test Case FAQ

ID	Tiêu đề	Mô tả kịch bản	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Ghi chú
TC_41	Hỏi giá trẻ em < 1.1m (miễn phí)	1. Mở chatbot 2. Gửi câu hỏi về trẻ em cao 1m	“Bé nhà em cao 1m thì có mất vé không?”	Phản hồi dưới 1.1m miễn phí	“Dạ bé cao 1m thì miễn phí vé ạ, vì dưới 1m1 sẽ không tính tiền vé.”	Pass
TC_42	Hỏi giá trẻ em = 1.1m	1. Mở chatbot 2. Gửi câu hỏi về trẻ em cao 1.1m	“Bé cao đúng 1m1 tính sao?”	Phản hồi cao 1.1 - 1.4m tính 50% giá vé	“Dạ bé cao đúng 1m1 thì tính 50% giá vé ạ.”	Pass

TC_ 43	Hỏi giá trẻ em = 1.4m	1. Mở chatbot 2. Gửi câu hỏi về trẻ em cao 1.4m	“Bé cao 1m4 tính sao?”	Phản hồi cao 1.1 - 1.4m tính 50% giá vé	“Dạ bé cao 1m4 thì tính 50% giá vé ạ.”	Pass
TC_ 44	Hỏi giá trẻ em = 1.5m	1. Mở chatbot 2. Gửi câu hỏi về trẻ em cao 1.5m	“Bé cao 1m5 tính sao?”	Phản hồi trên 1.4m tính giá người lớn	“Dạ bé cao 1m5 thì tính giá vé người lớn ạ, vì trên 1,4m sẽ theo giá niêm yết.”	Pass
TC_ 45	Hỏi về xe cabin	1. Mở chatbot 2. Hỏi về xe cabin	“Có xe cabin không?”	Trả lời chỉ có xe giường vùng cao, không có cabin	“Dạ xe cabin chỉ được phép đi đường bằng ạ, còn các tuyến vùng cao, đường đèo thì bên em chỉ có xe giường như đang hoạt động được phép chạy thôi ạ. Xe cabin không được phép hoạt động trên các tuyến đó ạ.”	Pass
TC_ 46	Hỏi về thẻ đi lại	1. Mở chatbot	“Thẻ đi lại sử dụng như thế nào?”	Hướng dẫn mua thẻ đi lại và cách sử dụng thẻ.	“Dạ anh/chị hay đi lại thường xuyên có thể mua thẻ đi lại bên em để được giảm 5% giá vé ạ.”	Pass

		2. Hỏi về thẻ đi lại				
TC_47	Hỏi về giảm giá	1. Mở chatbot 2. Hỏi về giảm giá	“Có giảm giá không?”	Trả lời quy định chung, yêu cầu để lại SĐT cho trường hợp đặc biệt	“Dạ giá khách và hàng bên em đã có quy định chung rồi ạ. Nếu anh/chị ở trong trường hợp đặc biệt thì vui lòng để lại thông tin, bộ phận chuyên trách bên em sẽ liên hệ lại ạ.”	Pass
TC_48	Giới hạn trọng lượng hành lý mang theo	1. Mở chatbot 2. Hỏi về trọng lượng hành lý	“Hành lý ký gửi tối đa bao nhiêu?”	Cung cấp quy định về kích thước và cân nặng tối đa của hành lý được miễn phí đi kèm.	“Dạ em chưa tra được đúng câu hỏi về hành lý ký gửi trong hệ thống ạ.”	Fail
TC_49	Giải đáp thắc mắc liên quan đến thủ tục tài chính doanh nghiệp.	1. Mở chatbot 2. Hỏi về hóa đơn đỏ	“Có xuất hoá đơn VAT không?”	Phản hồi có xuất hóa đơn, yêu cầu để lại thông tin	“Dạ có ạ. Anh/chị để lại thông tin hoặc nhắn vào Zalo OA Xe khách Vũ Hán, bên em sẽ xuất hoá đơn theo thông tin anh/chị gửi trong ngày ạ.”	Pass

2.4.7 Nhóm chức năng gửi hàng

Test Scenario: TS_SHIP_07

Tên kịch bản: Kiểm tra chức năng gửi hàng

Tiền điều kiện: Luồng nghiệp vụ tiếp nhận ký gửi hàng hóa, quy định đóng gói và địa chỉ các văn phòng nhận/trả đồ đã được kích hoạt.

Mục tiêu: Đảm bảo cung cấp đúng hướng dẫn gửi hàng và văn phòng nhận

Từ kịch bản tổng quát này, chúng ta có thể cụ thể hóa thành các trường hợp kiểm thử (Test Cases) chi tiết hơn dưới đây.

Bảng 2.10 Test Case gửi hàng hóa

ID	Tiêu đề	Mô tả kịch bản	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Ghi chú
TC_ 50	Gửi loại hàng hóa đặc thù có kích thước lớn	- Mở chatbot - Gửi câu hỏi gửi xe máy	“Gửi xe máy từ Đồng Văn về Hà Nội bao nhiêu tiền?”	Nhận diện mặt hàng xe máy; Chatbot thực hiện hỏi lại khách gửi xe số hay xe ga để báo giá chính xác.	“Dạ cước gửi xe máy sẽ tính theo trọng lượng và kích thước, nên cần xem xe trực tiếp mới báo giá chính xác ạ.”	Fail
TC_ 51	Kiểm tra tư vấn quy trình gửi bưu phẩm thông thường nói chung.	- Mở chatbot - Gửi câu hỏi về gửi hàng thông thường	“Tôi gửi đồ có giá như thế nào?”	Hướng dẫn các bước làm thủ tục gửi đồ và cung cấp danh sách địa chỉ văn phòng tiếp nhận.	“Dạ cước gửi hàng bên em tính theo trọng lượng và kích thước, nên cần xem hàng trực tiếp để báo giá chính xác ạ. Anh/chị có thể mang	Pass

					hàng ra văn phòng gần nhất để gửi.”	
TC_ 52	Tra cứu thời gian biên đóng/mở cửa kho nhận hàng tại thủ đô.	1. Mở chatbot 2. Gửi câu hỏi về thời gian nhận hàng	“Văn phòng Mỹ Đình nhận hàng đến mấy giờ?”	Phản hồi chính xác thông tin hành khách cần mang hàng ra trước 18h hàng ngày.	“Dạ văn phòng Mỹ Đình nhận hàng đến 18:00 ạ.”	Pass
TC_ 53	Hỏi cước gửi hàng	1. Mở chatbot 2. Gửi câu hỏi cước gửi hàng	“Gửi 1 thùng 5kg về Đồng Văn bao nhiêu?”	Yêu cầu liên hệ trực tiếp văn phòng để báo giá	“Dạ gửi 1 thùng 5kg về Đồng Văn bên em chưa báo giá chính xác online được ạ, vì cước còn tính theo trọng lượng và kích thước hàng. Anh/chị có thể mang ra văn phòng gần nhất để gửi ạ.”	Pass

2.4.8 Nhóm chức năng chuyển chăm sóc khách hàng

Test Scenario: TS_CSKH_08

Tên kịch bản: Kiểm tra chức năng chuyển nhân viên chăm sóc khách hàng

Tiền điều kiện: Tính năng Webhook kết nối và cấu hình chuyển mạch sang tài khoản của tư vấn viên trực tiếp đã được thiết lập sẵn trên hệ thống.

Mục tiêu: Tính năng Webhook kết nối và cấu hình chuyển mạch sang tài khoản của tư vấn viên trực tiếp đã được thiết lập sẵn trên hệ thống.

Từ kịch bản tổng quát này, chúng ta có thể cụ thể hóa thành các trường hợp kiểm thử (Test Cases) chi tiết hơn dưới đây.

Bảng 2.11 Test Case chuyển nhân viên hỗ trợ

ID	Tiêu đề	Mô tả kịch bản	Dữ liệu đầu vào	Kết quả mong đợi	Kết quả thực tế	Ghi chú
TC_ 54	Chuyển CSKH khi khách yêu cầu	1. Mở chatbot 2. Gửi yêu cầu gặp nhân viên	“Cho tôi gặp nhân viên”	Thực hiện báo chờ và chuyển luồng hội thoại cho nhân viên.	“Dạ em đã tiếp nhận thông tin. Anh/chị chờ giây lát em sẽ chuyển qua bộ phận chuyên trách xử lý ạ.”	Pass
TC_ 55	Chuyển CSKH khi khiếu nại	1. Mở chatbot 2. Gửi khiếu nại	“Tôi muốn khiếu nại về chuyến đi hôm qua”	Chuyển CSKH, lưu thông tin khách	“Dạ em đã tiếp nhận thông tin. Anh/chị chờ giây lát em sẽ chuyển qua bộ phận chuyên trách xử lý ạ.”	Pass

TC_56	Chuyển CSKH khi hủy vé	1. Mở chatbot 2. Gửi yêu cầu hủy vé	“Tôi muốn hủy vé”	Thực hiện báo chờ và chuyển luồng hội thoại cho nhân viên.	“Dạ em đã tiếp nhận thông tin. Anh/chị chờ giây lát em sẽ chuyển qua bộ phận chuyên trách xử lý ạ.”	Pass
-------	------------------------	--	-------------------	--	---	------

2.5 Phương pháp đo lường độ bao phủ kiểm thử (Test Coverage)

Đo lường độ bao phủ kiểm thử là một chỉ số quan trọng nhằm đánh giá tính đầy đủ của bộ Test Case so với các yêu cầu chức năng đã đề ra. Đối với hệ thống chatbot nhà xe Vũ Hán, phương pháp đo lường được thực hiện thông qua việc đối soát danh sách Test Case với các nhóm nghiệp vụ thực tế trên hệ thống.

2.5.1 Bảng đối soát độ bao phủ theo chức năng

Bảng 2.12 Bảng đối soát độ bao phủ theo chức năng

STT	Chức năng kiểm thử	Mục tiêu kiểm thử	Danh sách test case tương ứng
1	Nhận diện Intent	Xác minh khả năng hiểu đúng ý định của khách hàng.	TC_01, TC_02, TC_03, TC_04, TC_05, TC_06, TC_07, TC_08
2	Phân loại loại xe	Kiểm tra tính chính xác khi phân biệt xe Limousine, xe ghế 29 chỗ, xe giường nằm.	TC_09, TC_10, TC_11, TC_12, TC_37, TC_38, TC_39
3	Đặt vé xe limousine	Đánh giá luồng thu thập thông tin và xác nhận đặt chỗ cho dòng xe cao cấp VIP.	TC_16, TC_21, TC_35

4	Đặt vé xe khách	Đánh giá tính chính xác quy trình đặt vé và lịch trình tuyến xe giường nằm 29 và 40 chỗ.	TC_09, TC_24, TC_25, TC_26, TC_27, TC_28, TC_29, TC_30, TC_31, TC_32, TC_33, TC_34, TC_35, TC_36
5	Giải đáp FAQ	Kiểm tra độ phủ câu trả lời về các câu hỏi thường gặp như giá vé các tuyến, chính sách trẻ em, sinh viên, hành lý, thú cưng, VAT.	TC_41, TC_42, TC_43, TC_44, TC_45, TC_46, TC_47, TC_48, TC_49
6	Kiểm tra điểm đón trả	Xác minh thông tin bến xe, văn phòng, nút giao cao tốc và các điểm đón dọc đường.	TC_17, TC_23
7	Hỗ trợ gửi hàng	Kiểm tra khả năng hướng dẫn thủ tục, cung cấp thông tin văn phòng và thông tin nhận hàng.	TC_50, TC_51, TC_52, TC_53
8	Chuyển nhân viên/ Hỗ trợ	Đảm bảo hệ thống chuyển luồng CSKH.	TC_54, TC_55, TC_56

2.5.2 Chỉ số đo lường định lượng

Hệ thống sử dụng các công thức sau để báo cáo mức độ hoàn thành kiểm thử:

Độ bao phủ yêu cầu (Requirement Coverage):

$$Coverage(\%) = \left(\frac{\text{Số chức năng đã được kiểm thử}}{\text{Tổng số yêu cầu chức năng (FR)}} \right) \times 100\% = 100\%$$

Bộ test case đã bao phủ được 8/8 chức năng của chatbot: Nhận diện Intent, Phân loại loại xe, Đặt vé xe limousine, Đặt vé xe khách, Giải đáp FAQ, Kiểm tra điểm đón trả, Hỗ trợ gửi hàng, Chuyển nhân viên/ Hỗ trợ.

CHƯƠNG 3: THỰC HIỆN VÀ BÁO CÁO KẾT QUẢ KIỂM THỬ

3.1 Lý do lựa chọn giải pháp và công cụ kiểm thử

Hệ thống chatbot nhà xe Vũ Hán được triển khai trên nền tảng web, phục vụ lượng lớn hành khách đặt vé và tra cứu thông tin hàng ngày. Để đảm bảo hệ thống vận hành ổn định, an toàn và dễ dàng bảo trì, đề án lựa chọn các công cụ dựa trên các bài toán thực tế sau:

- **Playwright (Kiểm thử chức năng):** Với nhiều kịch bản nghiệp vụ phức tạp, việc kiểm thử thủ công mỗi khi cập nhật tri thức là bất khả thi. Playwright được chọn vì tốc độ thực thi nhanh, hỗ trợ đa trình duyệt và đa ngôn ngữ ngoài ra Playwright còn có cơ chế tự động chờ (auto-wait), khả năng chạy song song mạnh mẽ, và kiến trúc hiện đại không phụ thuộc vào driver.
- **JMeter (Kiểm thử hiệu năng):** Đặc thù ngành vận tải thường có lượng truy cập đột biến vào các dịp lễ, Tết. JMeter giúp giả lập tải cao để đảm bảo chatbot không bị treo khi hàng trăm khách hàng cùng nhấn tin một lúc.
- **OWASP ZAP (Kiểm thử bảo mật):** Chatbot thu thập thông tin cá nhân (SĐT, điểm đến). Việc sử dụng OWASP ZAP giúp phát hiện sớm các lỗ hổng như XSS, SQL Injection, bảo vệ dữ liệu khách hàng khỏi các cuộc tấn công mạng.
- **SonarQube (Quản lý chất lượng mã nguồn):** Để chatbot dễ mở rộng và nâng cấp, mã nguồn cần sạch và tối ưu. SonarQube giúp đo lường "nợ kỹ thuật", đảm bảo code đạt tiêu chuẩn chất lượng trước khi triển khai.
- **GitHub Actions (Tự động hóa Pipeline):** Kết nối tất cả công cụ trên thành một luồng tự động (CI/CD). Giúp phát hiện lỗi ngay khi vừa cập nhật code, giảm thiểu rủi ro khi vận hành thực tế.

3.2 Kiểm thử chức năng tự động với Playwright

3.2.1 Cấu hình Framework kiểm thử tự động Playwright

Để kiểm thử Chatbot đạt hiệu quả, tệp *playwright.config.ts* được cấu hình để tối ưu hóa khả năng tương tác với môi trường AI. Chatbot được thiết lập chạy trên trình duyệt Chrome với 4 workers giúp đẩy nhanh quá trình kiểm thử.

```

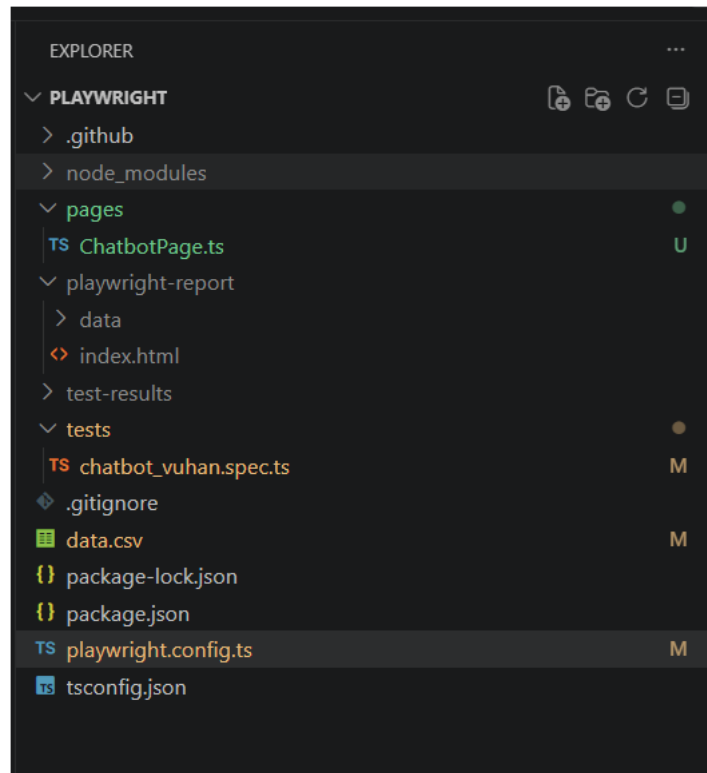
TS playwright.config.ts > [?] default
1  import { defineConfig, devices } from '@playwright/test';
2  export default defineConfig({
3    testDir: './tests',
4    fullyParallel: true,
5    forbidOnly: !!process.env.CI,
6    retries: process.env.CI ? 2 : 0,
7    workers: process.env.CI ? 4 : undefined,
8    reporter: 'html',
9    use: {
10     baseURL: 'https://chatbot-nhaxevuhan.vercel.app/chat',
11     headless: true,
12     viewport: { width: 1280, height: 720 },
13     screenshot: 'on',
14     trace: 'on-first-retry',
15   },
16   projects: [
17     {
18       name: 'chromium',
19       use: { ...devices['Desktop Chrome'] },
20     },
21   ],

```

Hình 3.1: Thiết lập tham số kiểm thử trong Playwright

3.2.2 Kiến trúc thư mục theo mô hình POM

Dự án được tổ chức theo kiến trúc **Page Object Model (POM)**, tách biệt giữa logic kiểm thử và các thành phần giao diện. Điều này giúp hệ thống có khả năng bảo trì cao và dễ dàng mở rộng khi Chatbot bổ sung thêm tính năng mới. Dữ liệu đầu vào, sử dụng giải pháp **Kiểm thử hướng dữ liệu (Data-Driven Testing)** với 70 test cases tự động lưu tại file data.csv.



Hình 3.2: Cấu trúc POM của thư mục dự án

3.2.3 Lớp đối tượng giao diện

Tập tin *ChatbotPage.ts* đóng vai trò là tầng trừu tượng hóa giao diện (Page Object Layer) trong cấu trúc Framework. Đây là:

- **Nơi định nghĩa các thành phần giao diện (Locators):** Quản lý tập trung các bộ chọn HTML của khung chat như ô nhập liệu, nút bấm gửi, vùng hiển thị tin nhắn của trợ lý ảo, các hiệu ứng động và điều hướng trình duyệt vào trang chủ của hệ thống.

```

readonly page: Page;
readonly chatInput: Locator;
readonly sendButton: Locator;
readonly loadingDot: Locator;
readonly assistantMessages: Locator;

constructor(page: Page) {
  this.page = page;
  this.chatInput = page.getByPlaceholder('Nhập câu hỏi của bạn tại đây...');
  this.sendButton = page.locator('button:has(svg.lucide-send)');
  this.loadingDot = page.locator('.loading-dots');
  this.assistantMessages = page.locator('div.message-bubble.assistant-message');
}

async navigate() {
  await this.page.goto('/');
}

```

Hình 3.3: Định nghĩa thành phần giao diện

- **Nơi đóng gói các hành vi tương tác (Actions):** Chuyển đổi các chuỗi thao tác vật lý của người dùng (nhập liệu, click, chờ đợi API) thành các hàm nghiệp vụ có thể tái sử dụng.

```

async sendMessage(message: string) {
  await this.chatInput.fill(message);

  const responsePromise = this.page.waitForResponse(response =>
    response.url().includes('/api/chat') && response.status() === 200
  );

  await this.sendButton.click();
  await responsePromise;

  await this.loadingDot.waitFor({ state: 'hidden', timeout: 30000 });
}

async getLastBotResponse(): Promise<string> {
  const lastMessage = this.assistantMessages.last();
  await lastMessage.waitFor({ state: 'visible', timeout: 20000 });
  await this.page.waitForTimeout(5000);

  const rawText = await lastMessage.innerText();
  return rawText.replace(/\s+/g, ' ').trim().toLowerCase();
}

```

Hình 3.4: Đóng gói hành vi người dùng

Lớp *ChatbotPage* khởi tạo các *Locator* đại diện cho ô nhập tin nhắn và nút gửi. Hàm *sendMessage* thực hiện tuần tự việc nhập chuột, điền chuỗi nội dung văn bản lấy từ file dữ liệu và kích hoạt sự kiện click gửi đi. Hàm *getLastResponse* sử dụng cơ chế đợi tương minh (*waitFor*) cho đến khi bong bóng chat cuối cùng của chatbot hiển thị

trên DOM, sau đó trích xuất nội dung text và loại bỏ khoảng trống thừa để phục vụ đối soát.

3.2.4 Kịch bản thực thi kiểm thử tự động dữ liệu tập trung

Tập tin *chatbot.spec.ts* đóng vai trò là tầng thực thi nghiệp vụ (Execution Layer) trong mô hình Page Object Model. Đây là:

- **Nơi cấu hình nạp dữ liệu kiểm thử động (Data-Driven Testing):** Đọc dữ liệu thô từ tệp *data.csv*, tự động phân tích cú pháp (parse) và khởi tạo vòng lặp chạy tự động tương ứng với số lượng Test Cases được thiết lập.

```
const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
const csvFilePath = './data.csv';

const fileContent = fs.readFileSync(csvFilePath, { encoding: 'utf-8' });

interface TestCase {
  ID: string;
  Module: string;
  Input: string;
  'Expected Result': string;
}

const records: TestCase[] = parse(fileContent, {
  columns: true,
  skip_empty_lines: true,
  trim: true,
  bom: true,
  relax_quotes: true
});

for (const record of records) {
  if (!record.ID || record.ID.trim() === '') continue;
```

Hình 3.5: Đọc dữ liệu và khởi tạo vòng lặp

- **Nơi điều phối quy trình tương tác và kiểm tra kết quả:** Gọi các hàm nghiệp vụ từ lớp *ChatbotPage* để thực hiện gửi câu hỏi, nhận phản hồi và áp dụng thuật toán đối soát từ khóa thông minh để quyết định trạng thái Pass/Fail của một ca kiểm thử.


```

tests > TS chatbot_vuham.spect.ts > ...
28
29 for (const record of records) {
30   if (!record.ID || record.ID.trim() === "") continue;
31
32   test('Kiểm tra ${record.ID}: ${record.Input}', async ({ page }) => {
33     const chatbot = new ChatbotPage(page);
34
35     await chatbot.navigate();
36     await chatbot.sendMessage(record.Input);
37     const actualText = await chatbot.getLastBotResponse();
38
39     const expectedKeywords = record['Expected Result']
40       .split(',')
41       .map(k => k.replace(/\s+/g, ' ').trim().toLowerCase())
42       .filter(k => k.length > 0);
43
44     const matchedKeywords = expectedKeywords.filter(keyword =>
45       actualText.includes(keyword)
46     );
47
48     const passRate = matchedKeywords.length / expectedKeywords.length;
49     const isPass = passRate >= 0.5;
50
51     if (!isPass) {
52       console.log(`--- FAIL: ${record.ID} ---`);
53       console.log(`Nội dung thực tế: ${actualText}`);
54       console.log(`Chỉ khớp ${matchedKeywords.length}/${expectedKeywords.length} từ khóa.`);
55       console.log(`Các từ khớp: ${matchedKeywords.join(', ')}`);
56     }
57
58     expect(isPass, `Tỷ lệ khớp từ khóa quá thấp: ${(passRate * 100).toFixed(0)}%).toBe(true);
59   });
60 }

```

Hình 3.6: Khung định nghĩa của ca kiểm thử

Đoạn mã sử dụng thư viện *csv-parse/sync* để nạp toàn bộ kịch bản từ file *data.csv*. Mỗi dòng dữ liệu biến thành một ca kiểm thử độc lập nằm trong vòng lặp *for...of*. Khung kiểm thử tự động gửi dữ liệu *record.Input*, nhận chuỗi kết quả và áp dụng hàm *.filter()* của mảng kết hợp với phương thức *.includes()* để đối soát phản hồi thực tế của chatbot với kết quả mong đợi, có thể xác định test case PASS hoặc FAIL.

Hệ thống hiện sử dụng phương pháp: Partial Keyword Matching (Khớp từ khóa một phần) để đánh giá phản hồi chatbot. Đây là giải pháp so khớp ở mức độ bề mặt văn bản (Lexical Level Matching), chưa sử dụng biểu thức chính quy (Regex) hay các kỹ thuật so khớp ngữ nghĩa sâu (Semantic Matching/Embedding Similarity).

Mặc dù phương pháp đối soát từ khóa giúp triển khai mã kiểm thử nhanh chóng và dễ dàng áp dụng cho các dữ liệu dạng bảng, việc chỉ dựa vào cơ chế này tạo ra những lỗ hổng lớn trong công tác kiểm soát chất lượng Chatbot:

- **Lỗi bỏ sót (False Negative - Đúng ý nhưng khác câu chữ):** Hệ thống AI và xử lý ngôn ngữ tự nhiên (NLP) có tính sinh ngữ linh hoạt. Chatbot hoàn toàn có thể đưa ra câu trả lời chính xác về mặt nghiệp vụ cho hành khách nhưng sử dụng các từ đồng nghĩa hoặc cách diễn đạt khác.

- **Lỗi nhận sai (False Positive - Chứa keyword nhưng sai logic hoặc ngữ cảnh):** Chatbot có thể trả về một câu chứa đầy đủ từ khóa kiểm tra nhưng cấu trúc câu hoặc logic phủ định lại hoàn toàn sai lệch so với mong đợi của hành khách.

Để khắc phục triệt để các hạn chế trên, bộ mã nguồn kiểm thử tự động trong tương lai cần bổ sung và nâng cấp các tầng khẳng định (Assertions) nâng cao sau:

- **Intent/Entity Assertion (Khẳng định mức độ nhận diện NLU):** Không chỉ kiểm tra chuỗi text hiển thị, mã test cần bắt các gói tin API phản hồi từ Engine NLP (như Rasa, Dialogflow hoặc mô hình nội bộ) để khẳng định Chatbot nhận diện đúng nhãn ý định và bóc tách chính xác các Thực thể hành trình cụ thể.
- **JSON/API-Level Assertion (Khẳng định cấu trúc dữ liệu nền):** Áp dụng kỹ thuật kiểm tra trực tiếp dữ liệu có cấu trúc (Structured Data). Khẳng định rằng trong Metadata hoặc Payload ẩn đi kèm bong bóng chat trả về có chứa đúng các trường dữ liệu, mã trạng thái và giá trị biến phục vụ cho các logic hệ thống kế thừa (như lưu trữ xuống Database hoặc đẩy vào công thanh toán), giảm thiểu việc phụ thuộc hoàn toàn vào chuỗi giao diện trực quan.

3.2.5 Báo cáo kết quả thực thi

Playwright tự động kết xuất tệp báo cáo trực quan dưới dạng HTML tĩnh sau khi kết thúc quá trình chạy test. Số liệu định lượng từ quá trình kiểm thử tự động giao diện và logic phản hồi của Chatbot được thống kê chi tiết như sau:

- Tổng số lượng Test Cases thực thi: 70 test cases
- Số lượng ca kiểm thử Đạt (Passed): 66 test cases
- Số lượng ca kiểm thử Lỗi (Failed): 4 test cases
- Số lượng ca kiểm thử Bỏ qua (Skipped): 0 Test Cases.
- Tổng thời gian thực thi: 2.8 minutes

Q

Search tests

All70

✓ Passed66

✗ Failed4

Flaky0

Skipped0

Project: chromium

12:47:53 27/5/2026 Total time: 2.8m

▼ chatbot_vuhan.spec.ts

✗ Kiểm tra TC-06: Địa chỉ nhà xe ở đâu

chromium

8.5s

chatbot_vuhan.spec.ts:32

✗ Kiểm tra TC-34: Xe đón ở Mỹ Đình chỗ nào

chromium

8.8s

chatbot_vuhan.spec.ts:32

✗ Kiểm tra TC-41: Đi Vinh Quang

chromium

8.4s

chatbot_vuhan.spec.ts:32

✗ Kiểm tra TC-53: Có đón ở Times City không

chromium

8.4s

chatbot_vuhan.spec.ts:32

✓ Kiểm tra TC-01: Chào bạn

chromium

16.7s

chatbot_vuhan.spec.ts:32

✓ Kiểm tra TC-02: Cảm ơn em nhé

chromium

17.0s

chatbot_vuhan.spec.ts:32

✓ Kiểm tra TC-03: Bạn là ai?

chromium

18.7s

chatbot_vuhan.spec.ts:32

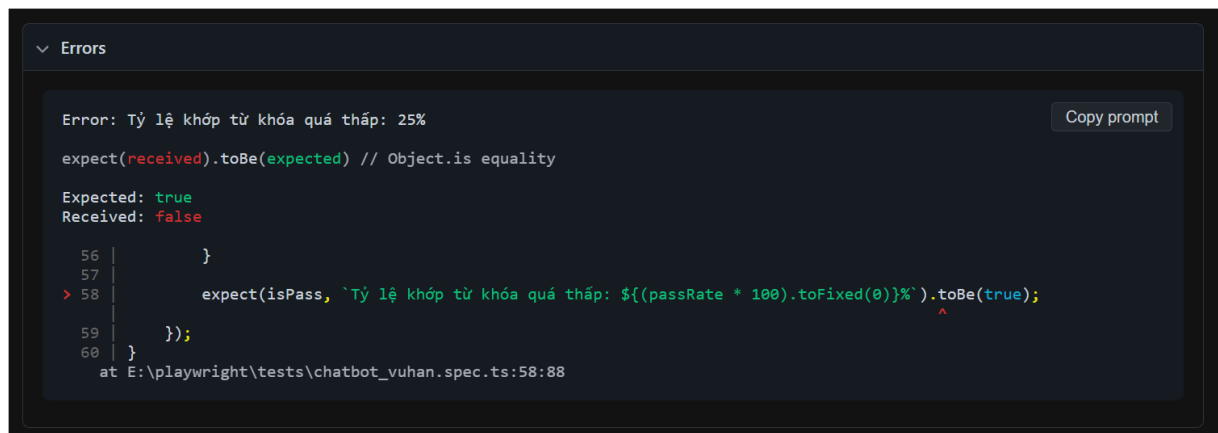
Hình 3.7: Giao diện báo cáo tổng quan của Playwright

Khi click vào một test case cụ thể, sẽ có Test Steps và thời gian chạy cụ thể của từng step và ảnh chụp màn hình UI thực tế.

▼ Test Steps	
Q Filter steps	
> ✓ Before Hooks	2.8s
> ✓ Navigate to "/" — ../pages/ChatbotPage.ts:19	449ms
> ✓ Fill "Chào bạn" getByPlaceholder("Nhập câu hỏi của bạn tại đây...") — ../pages/ChatbotPage.ts:23	624ms
> ✓ Wait for event "response" — ../pages/ChatbotPage.ts:25	2.1s
> ✓ Click locator('button:has(svg.lucide-send)') — ../pages/ChatbotPage.ts:29	225ms
> ✓ Wait for selector locator('.loading-dots') — ../pages/ChatbotPage.ts:32	30ms
> ✓ Wait for selector locator('div.message-bubble.assistant-message').last() — ../pages/ChatbotPage.ts:37	6ms
> ✓ Wait for timeout — ../pages/ChatbotPage.ts:38	5.0s
> ✓ Tỷ lệ khớp từ khóa quá thấp: 100% — chatbot_vuhan.spec.ts:59	0ms
> ✓ After Hooks	85ms

Hình 3.8: Giao diện tab Test Steps

Với những test case rơi vào trạng thái thất bại, tab Errors cung cấp dòng code cụ thể dẫn đến việc dừng kịch bản.



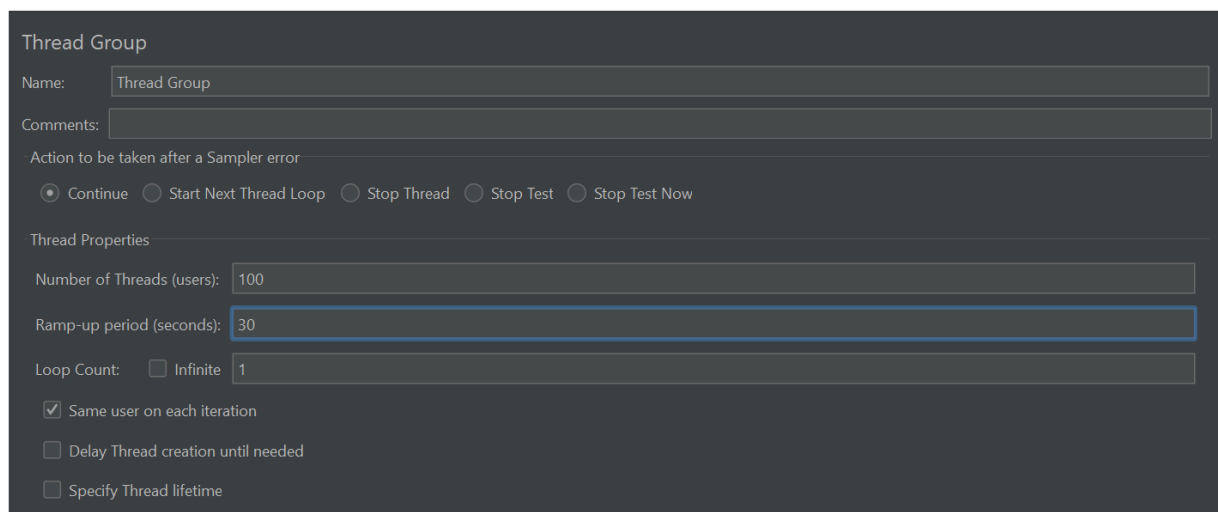
Hình 3.9: Giao diện tab Error

3.3 Kiểm thử hiệu năng với Apache Jmeter

Kiểm thử hiệu năng là bước quan trọng để đánh giá khả năng chịu tải của hệ thống Chatbot trong điều kiện người dùng thực tế cao điểm. Đặt mục tiêu thời gian phản hồi trung bình (Average response time) < 5 giây, P95 < 8 giây

3.3.1 Cấu hình kịch bản kiểm thử tải (Load Testing Setup)

Với cấu hình máy chạy test là: Windows 11, Intel Core i5, RAM 12GB, có kết nối mạng Wifi ổn định. Thiết lập kịch bản có 100 người dùng ảo, thời gian tăng tải là 30 giây với số lần lặp là 1.



Hình 3.10: Giao diện cấu hình Thread Group

Và những thông số mặc định như Protocol và Server Name:

HTTP Request Defaults

Name: HTTP Request Defaults

Comments:

Basic Advanced

Web Server

Protocol [http]: https Server Name or IP: chatbot-nhaxevuhan-uqc3.vercel.app Port Number:

HTTP Request

Path: Content encoding:

Hình 3.11: Cấu hình HTTP Request Defaults

Đồng thời cung cấp dữ liệu đầu vào thông qua CSV Data Set Config, mô phỏng như người dùng thật tương tác với Chatbot.

CSV Data Set Config

Name: CSV Data Set Config

Comments:

Configure the CSV Data Source

Filename: C:/Users/vanng/OneDrive/Tài liệu/chat_questions.csv Browse...

File encoding: UTF-8

Variable Names (comma-delimited): message,label

Ignore first line (only used if Variable Names is not empty): True

Delimiter (use '\t' for tab): ,

Allow quoted data?: False

Recycle on EOF?: True

Stop thread on EOF?: False

Sharing mode: All threads

Hình 3.12: Cấu hình CSV Data Set Config

Qua đó gửi những HTTP Request đến máy chủ ứng dụng để đo lường khả năng xử lý của hạ tầng Back-end (API). Với sessionId sẽ giúp nhận diện đúng phiên làm việc của người dùng và message được lấy từ bộ câu hỏi trong file csv chính là định dạng dữ liệu mà máy chủ yêu cầu.

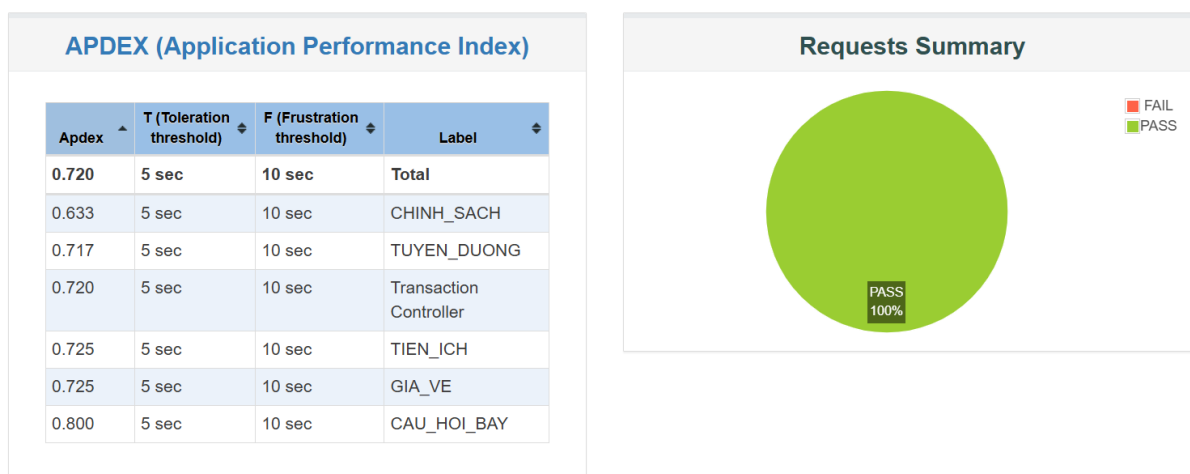
The screenshot shows an HTTP Request tool interface. At the top, there's a 'Name' field with a placeholder '\$(label)' and a 'Comments' field. Below this are tabs for 'Basic' and 'Advanced'. Under the 'Basic' tab, there's a 'Web Server' section with 'Protocol [http]:' set to 'https', 'Server Name or IP:' field, and 'Port Number:' field. Below that is an 'HTTP Request' section with a dropdown set to 'POST' and a 'Path:' field set to '/api/chat'. There are also checkboxes for 'Redirect Automatically', 'Follow Redirects' (checked), 'Use KeepAlive' (checked), 'Use multipart/form-data', and 'Browser-compatible headers'. At the bottom, there are tabs for 'Parameters', 'Body Data', and 'Files Upload'. The 'Body Data' tab is active, showing a JSON body:

```
{
  "sessionId": "${sessionId}",
  "message": "${message}"
}
```

Hình 3.13: Giao diện HTTP Request

3.3.2 Kết quả kiểm thử hiệu năng

Với 100 samples, Chatbot có tỷ lệ thành công đạt 100% PASS. Chỉ số đo lường sự hài lòng của người dùng dựa trên thời gian phản hồi thực tế đạt 0.720 (ở mức hài lòng hoặc có thể chấp nhận được).



Hình 3.14: Báo cáo APDEX

Kết quả kiểm thử tải với 100 người dùng đồng thời cho thấy hệ thống có độ ổn định về mặt kết nối khi không phát sinh lỗi HTTP (Error Rate = 0%). Về mặt hiệu năng xử lý, thời gian phản hồi trung bình (Average Response Time) hiện chưa tối ưu (~ 5 giây) và chỉ số APDEX đạt 0.720, cho thấy trải nghiệm người dùng chưa thực sự tối ưu trong điều kiện tải cao.

Statistics													
Requests		Executions		Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	100	0	0.00%	4713.50	3168	12831	4439.00	5581.70	7480.45	12816.48	7.47	7.79	2.67
CAU_HOI_BAY	15	0	0.00%	5107.87	3907	12831	4561.00	8724.60	12831.00	12831.00	1.15	1.13	0.41
CHINH_SACH	15	0	0.00%	5744.07	4022	11379	4657.00	9153.60	11379.00	11379.00	1.31	1.23	0.47
GIA_VE	20	0	0.00%	4464.80	3168	7657	4453.50	5286.30	7539.95	7657.00	2.59	2.64	0.93
TIEN_ICH	20	0	0.00%	4208.75	3563	5155	4184.00	4941.80	5145.95	5155.00	3.84	4.17	1.35
Transaction Controller	100	0	0.00%	4713.50	3168	12831	4439.00	5581.70	7480.45	12816.48	7.41	7.73	2.64
TUYEN_DUONG	30	0	0.00%	4503.33	3488	5663	4450.50	4980.20	5512.85	5663.00	5.30	5.92	1.90

Hình 3.15: Thống kê kết quả kiểm thử hiệu năng

Sau khi chạy kiểm thử tải, thống kê lại kết quả đo lường hiệu năng bao gồm:

1. Min – thời gian phản hồi nhanh nhất: 3,1s
2. Max – thời gian phản hồi chậm nhất: 12,8s
3. Median – thời gian trung vị (50%): 4,43s
4. P90 – 90% các yêu cầu có thời gian thấp hơn mức này: 5,58s
5. P95 – Chỉ số đánh giá độ tin cậy phổ biến: 7,48s
6. P99 – Các trường hợp ngoại lệ: 12,8s

Dựa trên kết quả đo đạc, hệ thống chatbot Nhà xe Vũ Hán đã đạt mục tiêu thời gian phản hồi trung bình đã đưa ra, hệ thống duy trì khả năng xử lý liên tục và không phát sinh lỗi trong quá trình kiểm thử. Trong tương lai, hệ thống cần được tối ưu thêm về tốc độ xử lý hội thoại và cơ chế truy xuất dữ liệu nhằm cải thiện thời gian phản hồi trong điều kiện tải cao.

3.4 Kiểm thử bảo mật với OWASP ZAP

3.4.1 Quy trình quét tự động (Automated Scanning)

Tính năng Automated Scan giúp tự động dò tìm tất cả các đường dẫn (endpoints) công khai của trang web <https://chatbot-nhaxevuhan.vercel.app/chat>.

Công cụ kích hoạt **Traditional Spider** kết hợp với **AJAX Spider (Chrome)** để bò qua toàn bộ cấu trúc ứng dụng, sau đó dùng bộ quét chủ động (Active Scan) để đánh giá các cấu hình an toàn bảo mật bề nổi.

Hệ thống tự động ghi nhận tổng cộng **8 cảnh báo (Alerts)** liên quan đến cấu hình sai hệ thống (Security Misconfiguration).

- Lỗi cấu hình sai cơ chế chia sẻ tài nguyên (Cross-Domain Misconfiguration): OWASP ZAP phát hiện phản hồi từ server chứa cấu hình CORS lỏng lẻo (*Access-Control-Allow-Origin: **), có thể cho phép các mã độc JavaScript chạy từ các domain lạ (không thuộc quyền quản lý của nhà xe Vũ Hán) có thể gửi request và đọc luồng dữ liệu thời gian thực từ API của hệ thống.
- Thiếu chính sách bảo mật nội dung (Content Security Policy - CSP Header Not Set): Server trả về phản hồi không định nghĩa header Content-Security-Policy gây thiếu lớp phòng thủ chiều sâu để kiểm soát nguồn tải các tệp thực thi (Scripts, Styles, Images). Kẻ tấn công có thể lợi dụng điều này để thực hiện chèn ép các đoạn mã độc hại hoặc khai thác lỗ hổng XSS thuận lợi hơn.
- Thiếu Header chống chèn khung (Missing Anti-clickjacking Header): Ứng dụng không thiết lập thuộc tính *X-Frame-Options* hoặc quy định *frame-ancestors* trong CSP khiến trang web có thể bị nhúng lén vào bên trong một thẻ *<iframe>* của một trang web độc hại khác, dẫn đến nguy cơ người dùng bị lừa click vào các chức năng ẩn (Clickjacking).
- Tiết lộ thông tin hạ tầng Proxy (Proxy Disclosure): Nó xảy ra khi máy chủ (server) phản hồi về trình duyệt có chứa các header (như Via, X-Forwarded-For, hoặc X-Proxy-ID). Những header này tiết lộ rằng yêu cầu của người dùng đã đi qua một máy chủ trung gian (Proxy/Load Balancer/CDN) trước khi đến được ứng dụng.

Ngoài ra còn có những lỗi có mức độ nghiêm trọng thấp như:

- Không bắt buộc bảo mật tầng vận chuyển (Strict-Transport-Security Header Not Set): Thiếu header *Strict-Transport-Security* (HSTS) trong phản hồi của HTTP. Khiến trình duyệt không ép buộc kết nối luôn phải chạy qua HTTPS, tạo cơ hội cho các cuộc tấn công hạ cấp giao thức từ HTTPS xuống HTTP (SSL Stripping).
- Thiếu thuộc tính chống bắt chước định dạng (X-Content-Type-Options Header Missing): Không có sự xuất hiện của header *X-Content-Type-Options: nosniff* làm cho trình duyệt có thể tự động đoán định và thực thi định dạng tệp (MIME-sniffing) sai lệch so với khai báo ban đầu của server, tạo điều kiện cho việc chạy các đoạn mã giả danh tệp tin tĩnh.

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	5
Cross-Domain Misconfiguration	Medium	Systemic
Missing Anti-clickjacking Header	Medium	Systemic
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	4
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	4
Permissions Policy Header Not Set	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Modern Web Application	Informational	5
Re-examine Cache-control Directives	Informational	4
Retrieved from Cache	Informational	Systemic
Storable but Non-Cacheable Content	Informational	Systemic

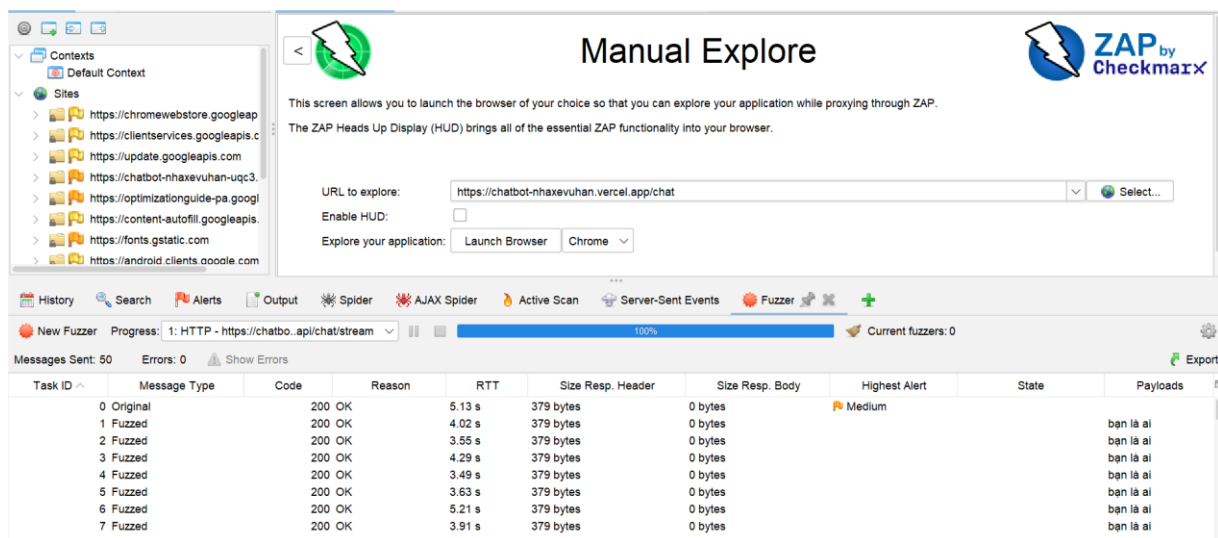
Hình 3.16: Báo cáo Alerts của OWASP ZAP

3.4.2 Kiểm thử thủ công (Manual Explore)

Mặc dù bộ quét tự động (Automated Scan) đã chỉ ra các lỗ hổng về cấu hình Header, nhưng vẫn mong muốn kiểm tra độ ổn định của API Chatbot, khả năng xử lý dữ liệu bất thường và xác định xem hệ thống có cơ chế tự bảo vệ trước các hành vi tấn công từ chối dịch vụ hoặc spam hay không.. Do đó, quy trình kiểm thử thủ công được triển khai bằng cách sử dụng tính năng ZAP Fuzzer.

1. Mở Chatbot trên trình duyệt Chrome (Launch Browser).
2. Gửi message = “Bạn là ai” để tạo ra dữ liệu.
3. Quay lại công cụ ZAP, nhìn vào tab **History** ở phía dưới màn hình.
4. Tìm đúng gói tin gửi tin nhắn đi, click chuột phải, chọn Attack, chọn Fuzz
5. Cấu hình dữ liệu và gửi đi.

Hệ thống Chatbot không có giới hạn tần suất rất dễ bị tấn công từ chối dịch vụ (DoS), gây tê liệt hệ thống hoặc bị kẻ xấu spam script liên tục làm cạn kiệt tài nguyên băng thông và làm gia tăng đột biến chi phí cước gọi API từ các mô hình AI tích hợp phía sau backend.



Hình 3.17: Kiểm thử thủ công với Fuzz

3.5 Phân tích mã nguồn với SonarQube

3.5.1 Phân loại chi tiết các chỉ số chất lượng (Metrics Classification)

Kịch bản tự động hóa thông qua GitHub Actions, kích hoạt tự động theo cơ chế on: push. Dữ liệu phân tích tĩnh được đồng bộ trực tiếp về nền tảng đám mây SonarQube Cloud.

- a. Lỗ hổng bảo mật (Security Vulnerabilities) - Đạt mức C
 - **Tổng số lượng:** 9 Issues
 - Phân loại theo mức độ nghiêm trọng (Severity): **Blocker** – 0%, **High** – 0%, **Medium** – 33%, **Low** – 67%.
- b. Lỗi hệ thống (Bugs) & Code Smells
 - **Trạng thái Bugs:** Vẫn tồn tại các bug mức Major/Minor trong hệ thống.
 - **Code smells:** Các vấn đề về định dạng code, khai báo biến thừa, hoặc cấu trúc chưa tối ưu.
- c. Cấu trúc mã nguồn khác
 - **Duplications (Độ trùng lặp):** 11.3%. Mức này có thể giảm xuống dưới 10% bằng cách tách các logic xử lý lặp lại thành các hàm tiện ích hoặc middleware dùng chung.

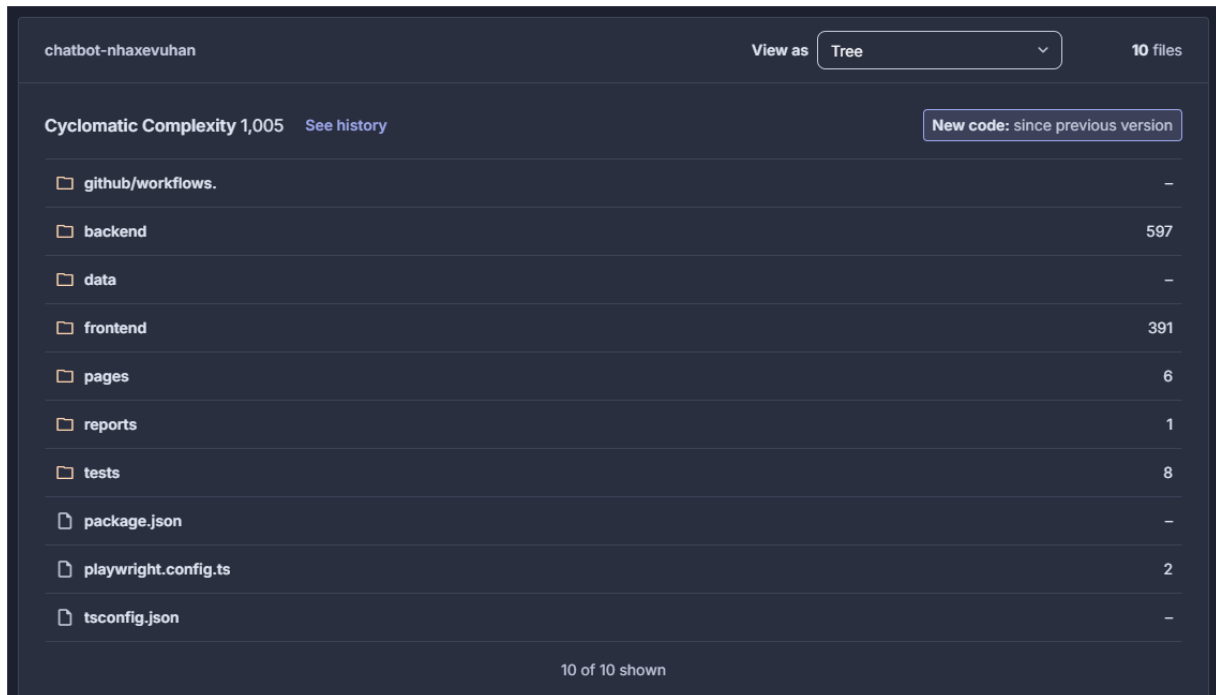


Hình 3.18: Kết quả phân tích mã nguồn với SonarQube

3.5.2 Báo cáo Độ phức tạp mã nguồn (Cyclomatic Complexity)

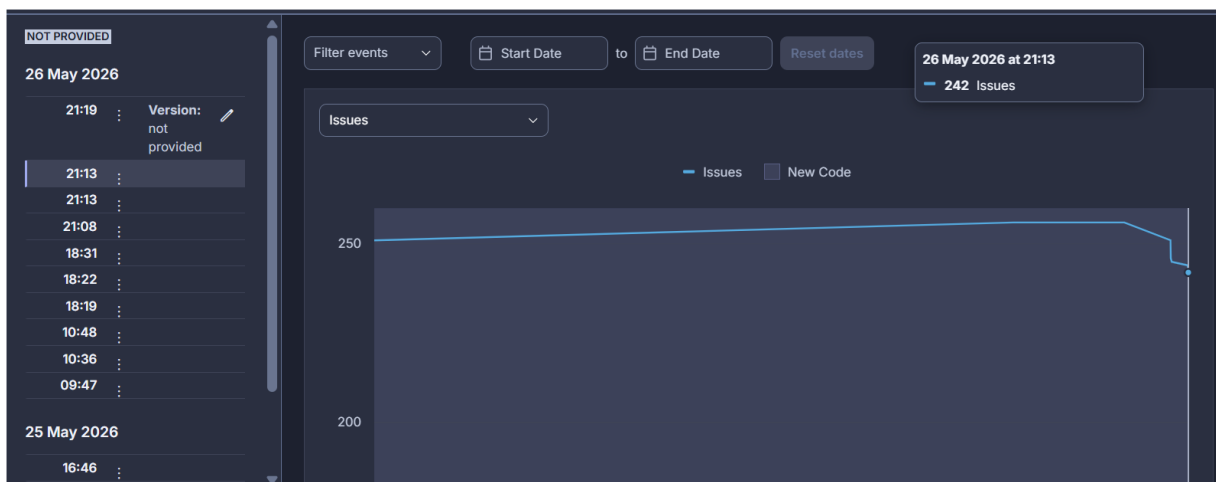
Tổng chỉ số Cyclomatic Complexity được ghi nhận là 1,005 phân bổ trên 10 thành phần (file/thư mục) gốc. Điều này có nghĩa là có ít nhất 1,005 luồng rẽ nhánh tuyến tính độc lập (các khối *if-else*, *switch-case*, *try-catch*, vòng lặp) đang vận hành trong toàn bộ dự án. Độ phức tạp phân bổ không đồng đều mà tập trung vào 2 phân hệ chính:

- **Thư mục *backend* (Complexity: 597 – chiếm 59.4%):** Đây là trung tâm điều hướng, xử lý các luồng API Logic, kết nối cơ sở dữ liệu (Prisma) và xử lý dữ liệu từ chatbot. Đây là khu vực phức tạp nhất và có nguy cơ phát sinh Bugs tiềm ẩn cao nhất.
- **Thư mục *frontend* (Complexity: 391 - chiếm 38.9%):** Nơi xử lý giao diện người dùng, các trạng thái render (state management) và tương tác trực quan với luồng chat.



Hình 3.19: Báo cáo Cyclomatic Complexity

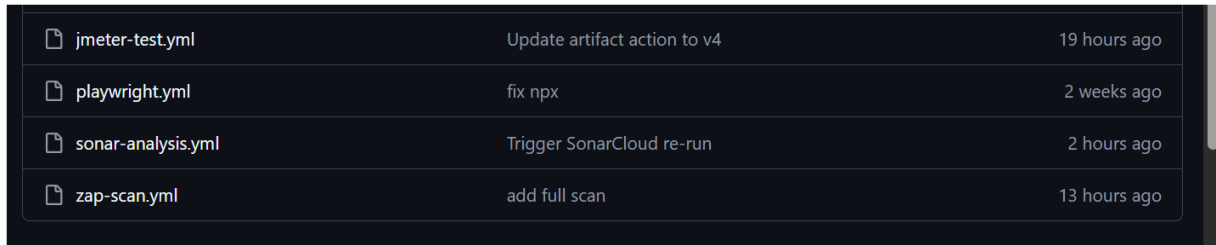
Trong các lần phân tích đầu tiên, số lượng issues có xu hướng tăng nhẹ do hệ thống liên tục được bổ sung chức năng và cập nhật mã nguồn mới. Sau đó đã có điểm bẻ gãy xu hướng mạnh mẽ, tổng số Issues giảm mạnh từ mức đỉnh xuống còn 242 Issues. Đây là kết quả trực tiếp của việc loại bỏ hoàn toàn các lỗi bảo mật nghiêm trọng (Blocker/High), đưa dự án thoát khỏi trạng thái báo động đỏ.



Hình 3.20: Biểu đồ xu hướng chất lượng

3.6 Tự động hóa quy trình Kiểm thử (CI/CD Pipeline)

Trong khuôn khổ đề tài, nhằm tối ưu hóa hiệu năng, tách biệt các tầng kiểm thử và đảm bảo tính dễ bảo trì của mã nguồn, hệ thống Tích hợp liên tục (CI/CD) được thiết kế theo mô hình mô-đun hóa với 04 luồng công việc (Workflow) độc lập tương ứng với 04 file cấu hình .yml đặt trong thư mục `.github/workflows/`



<code>jmeter-test.yml</code>	Update artifact action to v4	19 hours ago
<code>playwright.yml</code>	fix npx	2 weeks ago
<code>sonar-analysis.yml</code>	Trigger SonarCloud re-run	2 hours ago
<code>zap-scan.yml</code>	add full scan	13 hours ago

Hình 3.21 Mô hình thư mục `.github/workflows/`

- *Sonar-analysis.yml*: Chịu trách nhiệm quét và phân tích chất lượng mã nguồn tĩnh.
- *Playwright.yml*: Thực thi bộ kịch bản kiểm thử chức năng giao diện và kiểm thử ngữ nghĩa chuỗi phản hồi Stream của Chatbot.
- *Jmeter-test.yml*: Thực hiện giả lập tải và đánh giá hiệu năng chịu tải của hệ thống.
- *Zap-scan.yml*: Thực hiện rà quét các lỗ hổng an ninh mạng phổ biến theo tiêu chuẩn OWASP.

```

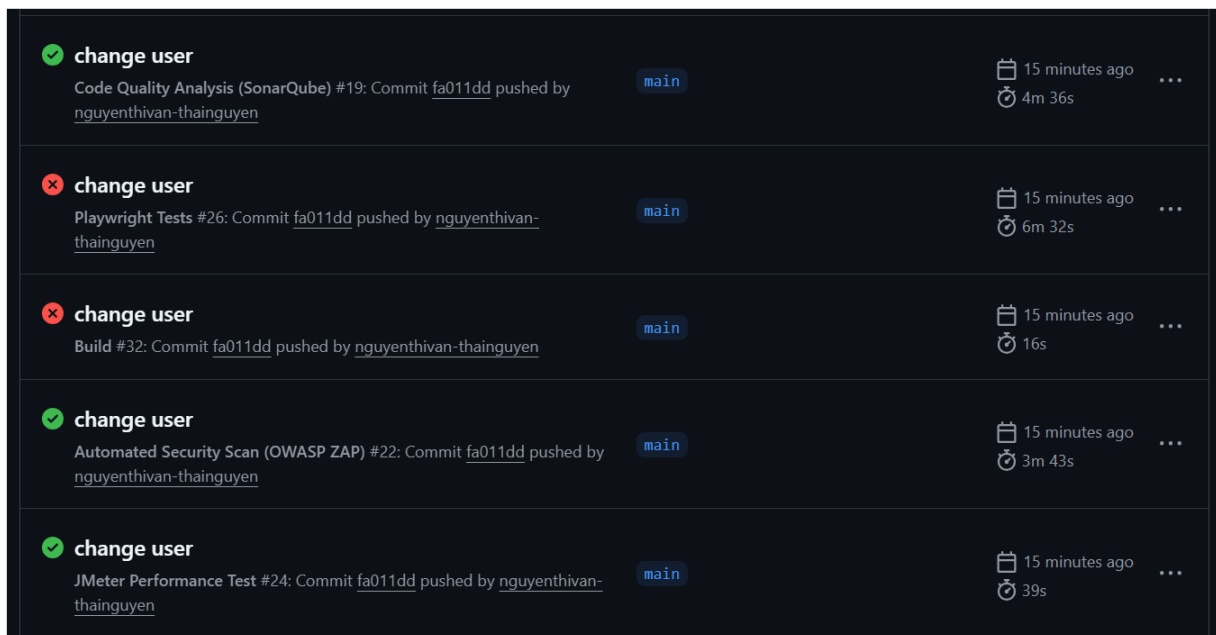
.github > workflows > playwright.yml
1 name: Playwright Tests
2 on:
3   push:
4     branches: [ main, master ]
5   pull_request:
6     branches: [ main, master ]
7 jobs:
8   test:
9     timeout-minutes: 60
10    runs-on: ubuntu-latest
11    steps:
12      - uses: actions/checkout@v4
13      - uses: actions/setup-node@v4
14        with:
15          node-version: lts/*
16      - name: Install dependencies
17        run: npm ci
18      - name: Install Playwright Browsers
19        run: npx playwright install --with-deps
20      - name: Run Playwright tests
21        run: xvfb-run npx playwright test
22      - name: Upload Report
23        uses: actions/upload-artifact@v4
24        if: always()
25        with:
26          name: playwright-report
27          path: playwright-report/
28          retention-days: 30
29

```

Hình 3.22: Ví dụ về file yml của công cụ Playwright

Pipeline được thiết lập kích hoạt tự động thông qua hai sự kiện lỗi của hệ thống kiểm soát phiên bản Git:

- **Sự kiện push:** Khi có bất kỳ mã nguồn mới nào được đẩy lên nhánh chính (main), hệ thống lập tức khởi động kịch bản nhằm thực hiện kiểm thử hồi quy (Regression Testing), đảm bảo các tính năng cập nhật không làm phá vỡ logic cũ.
- **Sự kiện pull:** Khi tạo yêu cầu gộp mã từ các nhánh tính năng vào nhánh main. Pipeline đóng vai trò như người gác cổng, ngăn chặn các đoạn mã lỗi được tích hợp vào môi trường production.

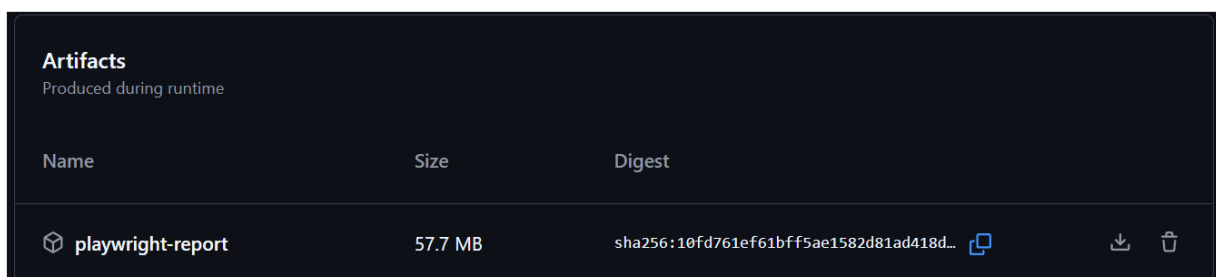


Hình 3.23: Giao diện GitHub Actions

Khi một job (ví dụ: Playwright Tests) bị thất bại do có kịch bản kiểm thử không vượt qua (Assert Fail), lỗi này không làm ảnh hưởng đến tiến trình của các job khác như *jmeter-test* hay *zap-scan*. Pipeline vẫn tiếp tục thực thi các job còn lại cho đến khi hoàn thành.

Sau khi pipeline kết thúc lượt chạy, hệ thống tự động tổng hợp các kết quả đầu ra thành các **Artifacts (Hiện vật báo cáo)** lưu trữ trực tiếp trên GitHub Action để phục vụ việc tra cứu:

1. **Playwright HTML Report (*playwright-report*):** Chứa thông tin chi tiết về từng bước chạy của kịch bản, thời gian phản hồi, ảnh chụp màn hình (Screenshot) tại các bước bị lỗi khi tương tác trực tiếp với Chatbot.



Hình 3.24 Artifacts của Playwright

2. **JMeter Report (*jmeter-report*):** Dashboard dạng HTML trực quan, cung cấp các biểu đồ về độ trễ (Latency), số lượng request xử lý thành công/thất bại trên giây (Throughput), và tỷ lệ lỗi khi kiểm thử tải.

Artifacts			
Produced during runtime			
Name	Size	Digest	
 jmeter-report	1.01 MB	sha256:a4f6eba97a6c3020fe4e1f133d9b87...	  

Hình 3.25 Artifacts của JMeter

- ZAP Report (zap-report):** File HTML liệt kê đầy đủ các lỗ hổng bảo mật được phát hiện (XSS, SQL Injection, các cảnh báo cấu hình sai sót header...), phân loại theo mức độ nghiêm trọng (High, Medium, Low).

Artifacts			
Produced during runtime			
Name	Size	Digest	
 zap-security-report	19.2 KB	sha256:717860ba8659f6470d96e31df8a4a2...	  

Hình 3.26 Artifacts của ZAP

- SonarQube Link:** Bản báo cáo trực tuyến đánh giá độ phủ của code (Code Coverage), các lỗi tiềm ẩn (Bugs), lỗ hổng bảo mật trong mã nguồn (Vulnerabilities) và nợ kỹ thuật (Code Smells) được thể hiện tại https://sonarcloud.io/project/overview?id=hoanganhtus_chatbot-nhaxevuhan

Quy trình tích hợp liên tục (CI) giúp rút ngắn chu kỳ phản hồi lỗi. Lập trình viên có thể biết ngay đoạn code mình vừa viết có làm hỏng tính năng cũ hoặc gây ra lỗ hổng bảo mật nào hay không chỉ sau vài phút.

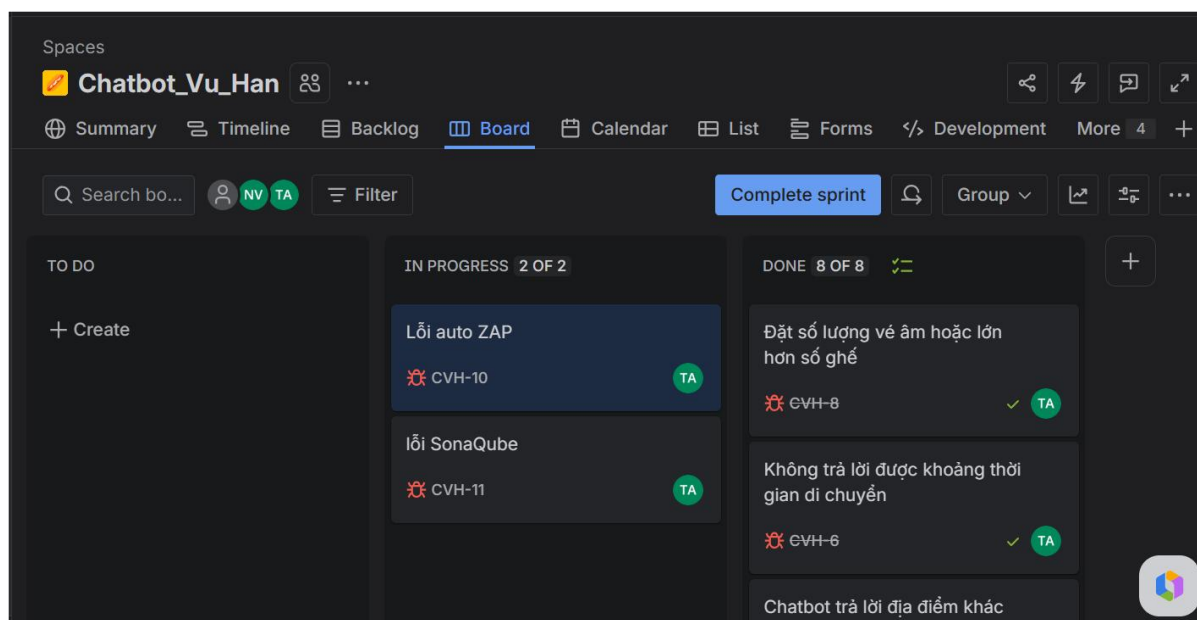
3.7 Quản lý lỗi bằng Jira

Trong quá trình kiểm thử chatbot nhà xe Vũ Hán, bên cạnh việc sử dụng các công cụ như Playwright, JMeter, OWASP ZAP và SonarQube để phát hiện lỗi, còn sử dụng Jira để quản lý và theo dõi quá trình xử lý lỗi. Việc áp dụng Jira giúp lưu trữ thông tin lỗi tập trung, theo dõi trạng thái xử lý và đảm bảo các lỗi được khắc phục đầy đủ trước khi đánh giá chất lượng hệ thống.

Sau khi phát hiện lỗi từ quá trình kiểm thử, tiến hành tạo các Issue trên Jira và phân loại theo từng chức năng hoặc từng nguồn phát hiện lỗi. Mỗi Issue bao gồm các

thông tin như tên lỗi, mô tả lỗi, trạng thái xử lý và người thực hiện. Các lỗi được quản lý theo ba trạng thái chính:

- **To Do:** Lỗi mới được ghi nhận và chưa xử lý.
- **In Progress:** Lỗi đang được sửa chữa.
- **Done:** Lỗi đã được khắc phục và kiểm thử lại thành công.



Hình 3.27 Bảng quản lý lỗi của dự án

Qua quá trình kiểm thử, nhóm đã ghi nhận các lỗi liên quan đến chức năng chatbot như trả lời sai địa điểm, không phản hồi được thời gian di chuyển, xử lý chưa chính xác số lượng vé đặt hoặc các cảnh báo được phát hiện bởi OWASP ZAP và SonarQube. Các lỗi sau khi được phát hiện đều được ghi nhận trên Jira để theo dõi và xử lý.

Bảng 3.1 Thống kê trạng thái lỗi trên Jira

Trạng thái	Kết quả
To Do	0
In Progress	2
Done	8
Tổng cộng	10

Kết quả thống kê cho thấy trong tổng số 10 lỗi được ghi nhận, có 8 lỗi đã được xử lý và đóng thành công, chiếm 80% tổng số lỗi. Hai lỗi còn lại đang trong quá trình cải thiện, chủ yếu liên quan đến các cảnh báo bảo mật từ OWASP ZAP và các khuyến nghị nâng cao chất lượng mã nguồn từ SonarQube. Các lỗi này không ảnh hưởng trực

tiếp đến chức năng chính của hệ thống nhưng vẫn cần được tiếp tục tối ưu trong các phiên bản tiếp theo.

Việc sử dụng Jira đã hỗ trợ nhóm quản lý lỗi một cách hiệu quả, giúp theo dõi toàn bộ vòng đời của lỗi từ khi phát hiện đến khi khắc phục hoàn tất. Đồng thời, công cụ cũng giúp kiểm soát tiến độ xử lý lỗi, tránh bỏ sót các vấn đề phát sinh trong quá trình phát triển và kiểm thử hệ thống. Đây là một bước quan trọng góp phần nâng cao chất lượng phần mềm và đảm bảo tính ổn định của chatbot trước khi triển khai thực tế.

3.8 Báo cáo tổng hợp và đánh giá chất lượng phần mềm

Sau khi hoàn thành quá trình kiểm thử, tiến hành tổng hợp và đánh giá chất lượng hệ thống chatbot nhà xe Vũ Hán dựa trên các kết quả thu được từ kiểm thử chức năng, kiểm thử tự động, kiểm thử hiệu năng, kiểm thử bảo mật và phân tích chất lượng mã nguồn.

Bảng 3.2 Tổng hợp kết quả đánh giá chất lượng phần mềm

Hạng mục	Kết quả
Kiểm thử chức năng	56/56 Test Case Pass (100%)
Kiểm thử tự động Playwright	66/70 Test Case Pass (94,29%)
Kiểm thử hiệu năng JMeter	Đạt yêu cầu (100%)
Kiểm thử bảo mật OWASP ZAP	3 cảnh báo Medium, 4 cảnh báo Low
Phân tích mã nguồn SonarQube	Medium: 33%, Low: 67%
Lỗi mức Critical	0
Lỗi mức High	0

Kết quả kiểm thử cho thấy toàn bộ 56 test case chức năng đều đạt kết quả mong đợi, đảm bảo các yêu cầu nghiệp vụ chính của chatbot được thực hiện chính xác. Bộ kiểm thử tự động bằng Playwright đạt tỷ lệ thành công 94,29% với 66 trên tổng số 70 kịch bản kiểm thử được thực thi thành công.

Đối với kiểm thử hiệu năng, kết quả từ JMeter cho thấy hệ thống đáp ứng tốt tải thử nghiệm được thiết lập, không xuất hiện hiện tượng quá tải hoặc gián đoạn dịch vụ trong quá trình kiểm thử. Điều này chứng minh chatbot có khả năng phục vụ đồng thời nhiều người dùng trong cùng một thời điểm.

Về mặt bảo mật, công cụ OWASP ZAP phát hiện 3 cảnh báo mức Medium và 4 cảnh báo mức Low. Các cảnh báo này chủ yếu liên quan đến các khuyến nghị tăng cường bảo mật và không ảnh hưởng trực tiếp đến hoạt động của hệ thống. Bên cạnh đó, kết quả phân tích mã nguồn bằng SonarQube cho thấy các vấn đề còn tồn tại chủ yếu ở mức Medium và Low (code smell/maintainability issue), không ghi nhận lỗi ở mức Critical hoặc High.

Từ các kết quả trên có thể nhận thấy mật độ lỗi của hệ thống ở mức thấp. Các chức năng nghiệp vụ chính hoạt động ổn định, hiệu năng đáp ứng yêu cầu và không phát hiện các lỗi nghiêm trọng ảnh hưởng đến quá trình sử dụng của người dùng. Những vấn đề còn lại chủ yếu là các cảnh báo về tối ưu mã nguồn và tăng cường bảo mật, có thể tiếp tục được cải thiện trong các phiên bản phát triển tiếp theo.

Nhìn chung, chatbot nhà xe Vũ Hán đã đáp ứng được các mục tiêu kiểm thử đề ra, đạt chất lượng đủ tốt để triển khai và sử dụng trong thực tế, đồng thời vẫn còn khả năng mở rộng và hoàn thiện thêm trong tương lai.

KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

Dưới sự giúp đỡ tận tình của giáo viên hướng dẫn là THS. Nguyễn Thu Phương, em đã hoàn thành bài báo cáo đồ án tốt nghiệp với đề tài **“Nghiên cứu và xây dựng bộ test case cùng giải pháp kiểm thử tự động cho chatbot nhà xe Vũ Hán”**. Trong thời gian thực hiện đề tài, em đã đạt được một số kết quả như sau:

1. Đã tìm hiểu và hệ thống hóa được quy trình kiểm thử phần mềm chuyên nghiệp, từ việc xây dựng bộ Test Case bài bản cho chatbot nhà xe Vũ Hán đến việc ứng dụng thành công các công cụ hỗ trợ như Playwright, JMeter, OWASP ZAP và SonarQube.
2. Đã xây dựng thành công bộ script kiểm thử tự động, giúp chuyển đổi các kịch bản thủ công sang các đoạn mã thực thi tự động, tăng hiệu suất và độ tin cậy của hệ thống.
3. Đã tích hợp quy trình CI/CD cơ bản để tự động hóa việc thực thi các bài kiểm thử, giúp quá trình kiểm soát chất lượng phần mềm diễn ra liên tục và ổn định.
4. Kết quả đạt được bước đầu đã khẳng định tính khả thi của việc áp dụng kiểm thử tự động trong dự án thực tế, giúp giảm thiểu sai sót và tối ưu hóa thời gian vận hành cho hệ thống chatbot.

Tuy nhiên, vẫn còn nhiều điểm hạn chế:

1. Các kịch bản kiểm thử mới chỉ tập trung vào các luồng nghiệp vụ chính, chưa bao phủ hết các tình huống biên phức tạp.
2. Cấu trúc mã nguồn của bộ kiểm thử và quy trình tích hợp tự động còn cần thêm thời gian để hoàn thiện và tinh chỉnh nhằm đạt hiệu suất tối ưu nhất.
3. Chưa tìm hiểu và thực hành được hết các chức năng của những công cụ kiểm thử.

Hướng phát triển đề tài:

Tiếp tục hoàn thiện bộ test case để tăng độ bao phủ cho hệ thống, đồng thời nghiên cứu tối ưu hóa quy trình tự động hóa để tăng tốc độ phản hồi kết quả kiểm thử. Cố gắng áp dụng các chức năng của công cụ hỗ trợ vào kiểm thử những hệ thống khác nhằm tích lũy kinh nghiệm cho công việc tương lai.

TÀI LIỆU THAM KHẢO

- [1] *Giáo trình Công nghệ phát triển ứng dụng*, Trường Đại học Công nghệ Thông tin và Truyền thông, Đại học Thái Nguyên Thái Nguyên, 2026.
- [2] *Bài giảng Kiểm thử và đảm bảo chất lượng phần mềm*, Trường Đại học Công nghệ Thông tin và Truyền thông, Đại học Thái Nguyên Thái Nguyên, 2024.
- [3] *Giáo trình nhập môn Công nghệ số*, Trường Đại học Công nghệ Thông tin và Truyền thông, Đại học Thái Nguyên, Thái Nguyên, 2026.
- [4] Tài liệu về công cụ Playwright: <https://playwright.dev/>
- [5] Tài liệu về công cụ Apache JMeter: <https://jmeter.apache.org/>
- [6] Tìm hiểu về Apache JMeter. Retrieved from <https://lanit.com.vn/jmeter-la-gi-uu-diem-cach-hoat-dong.html>
- [7] Tài liệu về công cụ OWASP ZAP: <https://www.zaproxy.org/>
- [8] Tài liệu về công cụ SonarQube: <https://www.sonarsource.com/>
- [9] Tài liệu về kiểm thử tự động. Retrieved from <https://viblo.asia/p/kiem-thu-thu-cong-manual-testing-va-kiem-thu-tu-dong-automated-testing-QWkwGnpER75g>
- [10] Nguyễn Thị Xuân, “Tìm hiểu về JMETER và ứng dụng vào việc kiểm thử trang web,” Đồ án tốt nghiệp, Trường đại học Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên., 2013. [Online]. Available: <https://repository.ictu.edu.vn/wp-content/uploads/2026/05/26674.pdf>.
- [11] Trần Thị Hồng Trang, “Nghiên cứu và ứng dụng kiểm thử hiệu năng sử dụng Jmeter cho website bán hàng online,” Đồ án tốt nghiệp, Trường đại học Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên., 2018. [Online]. Available: <https://repository.ictu.edu.vn/wp-content/uploads/2026/05/24879.pdf>.
- [12] Vũ Thị Lua, “Nghiên cứu và ứng dụng kiểm thử hiệu năng sử dụng công cụ Jmeter cho website Tìm việc làm,” Đồ án tốt nghiệp, Trường đại học Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên., 2018. [Online]. Available: <https://repository.ictu.edu.vn/wp-content/uploads/2026/05/24860.pdf>.
- [13] Đỗ Thị Huyền, “Nghiên cứu kỹ thuật kiểm thử phần mềm tự động sử dụng công cụ Jmeter và ứng dụng kiểm thử hiệu năng website,” Đồ án tốt nghiệp, Trường đại học

Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên., 2016. [Online]. Available: <https://repository.ictu.edu.vn/wp-content/uploads/2026/05/23530.pdf>.

[1]Hoàng Minh Giang, “Nghiên cứu kiểm thử an toàn ứng dụng Web bằng công cụ OWASP ZAP,” Đồ án tốt nghiệp, Trường đại học Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên., 2018. [Online]. Available: <https://repository.ictu.edu.vn/wp-content/uploads/2026/05/24807.pdf>.

PHỤ LỤC

File data.csv

"ID",	"Module",	"Input",	"Expected Result"
"TC-01",	"Nhận diện Intent",	"Chào bạn",	"Xe Vũ Hán, giúp gì, chào, hỗ trợ"
"TC-02",	"Nhận diện Intent",	"Cảm ơn em nhé",	"không có gì, cảm ơn"
"TC-03",	"Nhận diện Intent",	"Bạn là ai?",	"trợ lý ảo, Nhà xe Vũ Hán, hỗ trợ, giúp gì"
"TC-04",	"Nhận diện Intent",	"Alo nhà xe Vũ Hán đúng không",	"đúng, Vũ Hán"
"TC-05",	"Nhận diện Intent",	"Tôi muốn gửi hàng từ Hà Nội đi Tuyên Quang",	"văn phòng, trực tiếp"
"TC-06",	"Thông tin nhà xe",	"Địa chỉ nhà xe ở đâu",	"Hà Nội, Mỹ Đình, Tuyên Quang, Xín Mần"
"TC-07",	"Thông tin nhà xe",	"Số điện thoại văn phòng Mỹ Đình",	"số điện thoại, văn phòng Mỹ Đình, 0912 037 237"
"TC-08",	"Thông tin nhà xe",	"Nhà xe có chạy Tuyên Quang Hà Nội không",	"có"
"TC-09",	"FAQ",	"Xe có điều hòa không",	"có, trang bị, điều hòa"
"TC-10",	"Gửi hàng",	"Gửi đồ từ Hà Giang xuống Mỹ Đình hết bao nhiêu",	"mang hàng ra văn phòng gần nhất, trọng lượng, kích thước, trực tiếp"
"TC-11",	"Gửi hàng",	"Có nhận chở xe máy không",	"có, nhận chở xe máy"
"TC-12",	"Dịch vụ",	"Xe có đón trả tận nhà không",	"tận nơi ở Hà Nội, huyện, tỉnh, xe đi qua được"
"TC-13",	"Dịch vụ",	"Có phụ giúp xách hành lý không",	"có, phụ giúp xách hành lý"
"TC-14",	"Dịch vụ",	"Trên xe có sạc điện thoại không",	"có ,sạc"
"TC-15",	"Dịch vụ",	"Xe có giường nằm không",	"có,xe giường nằm 40"
"TC-16",	"Dịch vụ",	"Có xe ghế không",	"có"
"TC-17",	"Giá vé",	"Giá xe Vip/Limosine từ Hà Nội đi Tân Quang là bao nhiêu?",	"250"
"TC-18",	"Giá vé",	"Giá xe Limosine từ Hà Nội đi Tuyên Quang là bao nhiêu?",	"limousine, 150"
"TC-19",	"Giá vé",	"Từ Tuyên Quang đi Lý Bôn bao nhiêu?",	"240"
"TC-20",	"Lịch trình",	"Sơn Phú về Hà Nội có chuyến mấy giờ?",	"10h30, 20h45"

"TC-21", "Lịch trình", "Từ Hà Nội đi Tuyên Quang có những chuyến nào?", "05:30, 06:20, 06:55, 07:50, 15:25, 17:50, 18:30, 19:30"
"TC-22", "Lịch trình", "Hà Nội đi Tuyên Quang muộn nhất là mấy giờ?", "19:30"
"TC-23", "Lịch trình", "Xe chạy đường tuyến Hà Nội (Mỹ Đình) đi Tuyên Quang là quốc lộ hay cao tốc", "quốc lộ, cao tốc, kết hợp"
"TC-24", "Lịch trình", "Thời gian đi tuyến Hà Nội (Mỹ Đình) - Tuyên Quang mất bao lâu", "2, 2 giờ 30, 3"
"TC-25", "Lịch trình", "Na Hang về Hà Nội có chuyến nào?", "11:40, 21:30"
"TC-26", "Đặt vé", "Thanh toán vé Limousine thế nào?", "chuyển khoản, thanh toán trực tiếp"
"TC-27", "Đặt vé", "Mua vé tại văn phòng được không", "có thể, tại văn phòng"
"TC-28", "Đặt vé", "Có cần cọc tiền trước không", "không yêu cầu, chuyển khoản, giữ chỗ"
"TC-29", "Đặt vé", "Thanh toán bằng chuyển khoản được không", "chuyển khoản, giữ chỗ, thông tin, 8686111085, Techcombank, Bùi Thị Minh Hằng"
"TC-30", "Địa điểm", "Xe có đi Thái Nguyên không?", "chưa, không, kiểm tra"
"TC-31", "Địa điểm", "Xe qua Mê Linh không?", "có"
"TC-32", "Đặt vé", "Trẻ em có giá vé thế nào", "Dưới 1,1m, Miễn phí, Từ 1,1m đến 1,4m, 50%, Trên 1,4m"
"TC-33", "Đặt vé", "Có thẻ xe không", "có, thẻ đi lại, 10 lượt đi, giảm 5%"
"TC-34", "Vị trí", "Xe đón ở Mỹ Đình chỗ nào", "văn phòng, bên xe"
"TC-35", "Vị trí", "Có đón ở sân bay Nội Bài không", "có"
"TC-36", "Vị trí", "Xe trả khách ở đâu Tuyên Quang", "Tuyên Quang"
"TC-37", "Vị trí", "Có đi qua bệnh viện Bạch Mai không", "có đưa đón một số bệnh viện lớn tại Hà Nội, có, bệnh viện Bạch Mai"
"TC-38", "Vị trí", "Xe qua thành phố Cao Bằng không?", "Bảo Lâm, không, thành phố, TP, Cao Bằng"
"TC-39", "Vị trí", "Xe đi qua thành phố Lào Cai không?", "không, Lu"
"TC-40", "Alias", "Giá đi Cốc Pài?", "Xín Mần, 300"

"TC-41","Alias","Đi Vinh Quang","Hoàng Su Phì"

"TC-42","Alias","Đi Pắc Mầu","Bảo Lâm"

"TC-43","Khác","Tết này xe có chạy không","có, chạy, hoạt động"

"TC-44","Khác","Có tuyến lái xe không","vẫn, đang tuyến, gửi hồ sơ, sơ yếu lý lịch có xác nhận, giấy khám sức khỏe, bằng cấp"

"TC-45","Khác","Nhà xe Vũ Hán có mấy chi nhánh","Mỹ Đình, Giáp Bát, Tuyên Quang, Xín Mần, Mèo Vạc"

"TC-46","Lịch trình","Xe có dừng ở Xín Mần không","có dừng, Xín Mần"

"TC-47","Giá vé","Vé đi Mỹ Đình - Kiến Thiết bao nhiêu","150,000đ"

"TC-48","Dịch vụ","Xe Limousine có bao nhiêu ghế","xe Limousine, 9"

"TC-49","Đón/trả","Tuyên Quang đi Quán Bạ đón mấy giờ?","21h30, 23h"

"TC-50","Đón/trả","Đoan Hùng đi Yên Hoa đón mấy giờ?","22h30"

"TC-51","Lịch trình","Xe có chuyến nào chạy đêm không","có"

"TC-52","Lịch trình","Tôi muốn đi từ Hà Nội tới thị xã Phú Thọ thì trả ở điểm nào?","IC9, Thị xã Phú Thọ, Km54/IC8/Nhà hàng Quang Khải"

"TC-53","Vị trí","Có đón ở Times City không","có"

"TC-54","Vị trí","Có đón ở cầu Nhật Tân không","có"

"TC-55","Vị trí","Trên Đồng Văn trả khách ở đâu?","trả khách, Đồng Văn"

"TC-56","Vị trí","Xe có qua Sân vận động Mỹ Đình không","có"

"TC-57","Đặt vé","Đặt 2 vé giường nằm Hà Nội đi Xín Mần ngày 20/05/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321","quá khứ, đã qua"

"TC-58","Đặt vé","Đặt 2 vé giường nằm Hà Nội đi Xín Mần ngày 20/06/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 098abc4321","định dạng, hợp lệ"

"TC-59","Đặt vé","Đặt -3 vé giường nằm Hà Nội đi Xín Mần ngày 20/09/2026 chuyến 19:20, tên Nguyễn Văn A, Số điện thoại: 0987654321","không, hợp lệ, số lượng"

"TC-60","FAQ","Có xuất hóa đơn đỏ không","có"

"TC-61","FAQ","Ghế có massage không","không có, massage"

"TC-62","FAQ","Tôi mang theo nhiều đồ thì có phải trả phụ phí không?","hành lý, kiện đồ, 25"

"TC-63","FAQ","Xe có bảo hiểm hành khách không","có, bảo hiểm hành khách, tất cả các chuyến xe "

"TC-64","CSKH","Cho tôi gặp nhân viên","chuyển, bộ phận"

"TC-65","CSKH","Tôi muốn khiếu nại về chuyến đi hôm qua","chuyển, bộ phận"

"TC-66","CSKH","Tôi muốn hủy vé","chuyển, bộ phận"

"TC-67","Khác","Thẻ đi lại sử dụng như thế nào?","5%, "

"TC-68","Khác","Nhà xe đi qua bao nhiêu tỉnh","Hà Nội, Hà Giang, Tuyên Quang, Cao Bằng, Lào Cai, Vĩnh Phúc"

"TC-69","Khác","Làm đại lý thì như thế nào?","đại lý, số điện thoại, giấy phép kinh doanh, tuyển mong muốn, Zalo OA Xe khách Vũ Hán, trao đổi, liên hệ, chị Hằng, 0866111085"

"TC-70","Khác","Tôi muốn thuê xe","chuyển, chuyên trách"

File chat_questions.csv

message,label

Xe từ Hà Nội đi Đồng Văn lúc mấy giờ?,TUYEN_DUONG

Chuyến muộn nhất từ Hà Giang về Mỹ Đình là mấy giờ?,TUYEN_DUONG

Có xe nào đi từ Bắc Quang lúc 2 giờ sáng không?,TUYEN_DUONG

Lịch trình xe chạy tuyến Thái Nguyên - Mèo Vạc,TUYEN_DUONG

Xe có đi qua bệnh viện tỉnh Hà Giang không?,TUYEN_DUONG

Tôi muốn đi từ sân bay Nội Bài về Hà Giang thì đón ở đâu?,TUYEN_DUONG

Xe Vũ Hán có chạy tuyến đường 2 không?,TUYEN_DUONG

Thời gian dự kiến đến Quán Bạ là mấy giờ nếu đi chuyến 21h?,TUYEN_DUONG

Có xe trung chuyển tận nơi ở thành phố Hà Giang không?,TUYEN_DUONG

Xe có dừng nghỉ dọc đường ở Tuyên Quang không?,TUYEN_DUONG

Đón xe ở ngã tư Nội Bài thì đứng chỗ nào chính xác?,TUYEN_DUONG

Xe có chạy vào ngày lễ 30/4 không?,TUYEN_DUONG

Sáng mai có chuyến nào sớm nhất đi từ Hà Nội?,TUYEN_DUONG

Xe trả khách ở bến xe nào tại Hà Nội?,TUYEN_DUONG

Lộ trình xe đi qua những huyện nào của Hà Giang?,TUYEN_DUONG

Giá vé giường nằm đơn và giường đôi khác nhau thế nào?,GIA_VE

Trẻ em dưới 5 tuổi có phải mua vé không?,GIA_VE

Có giảm giá cho sinh viên khi đi thực tập không?,GIA_VE

Giá vé gửi xe máy theo người là bao nhiêu?,GIA_VE

Đặt vé khứ hồi có được giảm % nào không?,GIA_VE

Giá vé ngày Tết có tăng không bạn?,GIA_VE

Gửi một bao tải quần áo 20kg từ Hà Nội lên Hà Giang giá bao nhiêu?,GIA_VE

Tôi muốn bao nguyên xe 16 chỗ thì giá thế nào?,GIA_VE

Có chương trình tích điểm đổi vé không?,GIA_VE

Thanh toán bằng chuyển khoản hay tiền mặt?,GIA_VE

Trên xe có cổng sạc điện thoại USB không?,TIEN_ICH

Wifi trên xe có xem được Youtube mượt không?,TIEN_ICH

Xe có nhà vệ sinh khép kín bên trong không?,TIEN_ICH

Có phục vụ chăn đắp và gói sạch không?,TIEN_ICH

Xe đời mới hay đời cũ vậy bạn?,TIEN_ICH

Điều hòa trên xe có chỉnh được độ lạnh tại chỗ không?,TIEN_ICH

Có nước uống và khăn lạnh miễn phí không?,TIEN_ICH

Xe có cho mang vật nuôi lên không?,TIEN_ICH

Rèm cửa có che nắng tốt không?,TIEN_ICH

Ghế nằm có chức năng massage không?,TIEN_ICH

Xe có khoang chứa đồ rộng cho vali lớn không?,TIEN_ICH

Trên xe có yêu cầu đi chân đất không?,TIEN_ICH

Có hỗ trợ thuốc say xe cho khách không?,TIEN_ICH

Xe có tivi để xem phim không?,TIEN_ICH

Khoảng cách giữa các giường có rộng không?,TIEN_ICH

Tôi muốn đổi vé sang ngày mai có mất phí không?,CHINH_SACH

Nếu xe đến muộn thì tôi khiếu nại ở đâu?,CHINH_SACH

Tôi lỡ chuyển xe thì có được hoàn tiền không?,CHINH_SACH

Muốn hủy vé trước 2 tiếng thì xử lý thế nào?,CHINH_SACH

Tôi bỏ quên điện thoại trên xe làm sao để lấy lại?,CHINH_SACH

Tại sao giá vé trên web khác với nhân viên báo?,CHINH_SACH

Có được đổi chỗ ngồi sau khi đã lên xe không?,CHINH_SACH

Nếu xe hỏng dọc đường thì nhà xe xử lý thế nào?,CHINH_SACH

Tôi muốn xuất hóa đơn đỏ cho công ty thì làm thế nào?,CHINH_SACH

Nhà xe có cam kết không nhồi nhét khách không?,CHINH_SACH

Tài xế có hút thuốc trên xe không?,CHINH_SACH

Nhân viên phụ xe có thái độ không tốt thì báo cáo ai?,CHINH_SACH

Tôi muốn thay đổi số điện thoại nhận vé thì làm sao?,CHINH_SACH

Có được mang đồ ăn mùi lên xe không?,CHINH_SACH

Vé đã mua có thể chuyển nhượng cho người khác không?,CHINH_SACH

Đi Hà Giang,CAU_HOI_BAY

Tôi muốn đặt vé cho 2 người lớn 1 trẻ em và 1 xe máy tối thứ 6,CAU_HOI_BAY

Nhà xe Vũ Hán có tốt bằng nhà xe khác không?,CAU_HOI_BAY

Alo xe chạy chưa?,CAU_HOI_BAY

Có chỗ nằm tầng 1 gần tài xế không tôi bị say xe nặng?,CAU_HOI_BAY

Price for ticket?,CAU_HOI_BAY

Gửi hàng về Bắc Quang giá rẻ nhất là bao nhiêu?,CAU_HOI_BAY

Tại sao tôi gọi tổng đài mãi không ai nghe máy?,CAU_HOI_BAY

Xe có đón ở cổng bệnh viện Bạch Mai không?,CAU_HOI_BAY

Tôi muốn đi vào ngày 30 tháng 2,CAU_HOI_BAY

Cho hỏi xe của hãng nào mà chạy ầu thế?,CAU_HOI_BAY

Có xe đi từ Hà Giang về Hải Phòng không?,CAU_HOI_BAY

Tư vấn cho tôi chuyến đi du lịch Hà Giang 3 ngày 2 đêm,CAU_HOI_BAY

Đặt vé xong có tin nhắn xác nhận về điện thoại luôn không?,CAU_HOI_BAY

Nhà xe có hỗ trợ đặt phòng khách sạn ở Đồng Văn không?,CAU_HOI_BAY

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày tháng năm

GIÁO VIÊN HƯỚNG DẪN

LỜI CAM ĐOAN

*(kèm theo Thông báo số: ... /TB-ĐH CNTT&TT ngày ... tháng năm 2026
của Hiệu trưởng Trường ĐH CNTT&TT)*

Em xin cam đoan đã thực hiện việc kiểm tra mức độ tương đồng nội dung (ĐA/KLTN) qua phần mềm Turnitin một cách trung thực và đạt kết quả mức độ tương đồng %. Bản đề tài (ĐA/KLTN) kiểm tra qua phần mềm là bản cứng đã nộp để bảo vệ trước hội đồng.

Nếu sai em xin hoàn toàn chịu trách nhiệm.

Xin chân thành cảm ơn.

Thái Nguyên, ngày 28 tháng 5 năm 2026

TÁC GIẢ CỦA SẢN PHẨM

(Ký và ghi rõ họ tên)

ĐỀ CƯƠNG ĐỒ ÁN / KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên nhận đề tài

Họ và tên: Nguyễn Thị Vân

Mã sinh viên: DTC225180342

Lớp: KTPMK21A

Điện thoại: 0392135496

Email: dtc225180342@ictu.edu.vn

2. Thông tin về giảng viên hướng dẫn

Họ và tên: Nguyễn Thu Phương

Học hàm, học vị: Thạc sĩ

Khoa: Công nghệ thông tin

Bộ môn: Công nghệ phần mềm

Điện thoại: 0982483420

Email: ntphuong@ictu.edu.vn

3. Đề tài: Nghiên cứu và xây dựng bộ test case cùng giải pháp kiểm thử tự động cho chatbot nhà xe Vũ Hán.

4. Thời gian thực hiện: 10 tuần, từ 09/03/2026 đến ngày 22/05/2026

5. Mục tiêu:

- Phân tích các luồng nghiệp vụ chính của chatbot nhà xe Vũ Hán.
 - Xác định các nhóm chức năng cần kiểm thử như: Nhận diện intent, Phân loại loại xe, Đặt vé xe limousine, Đặt vé xe khách, Giải đáp FAQ, Kiểm tra điểm đón trả, Hỗ trợ gửi hàng, Chuyển nhân viên hỗ trợ
- Xây dựng bộ test case bao phủ các tình huống hội thoại phổ biến và các tình huống ngoại lệ.
- Xác định expected output cho từng test case dưới dạng rõ ràng và có thể đo kiểm.
- Đề xuất giải pháp automation test để tự động chạy các hội thoại kiểm thử.

- Xây dựng tiêu chí đánh giá kết quả kiểm thử như độ đúng intent, độ đúng action, độ đúng dữ liệu trích xuất và độ nhất quán của câu trả lời.
- Hỗ trợ phát hiện lỗi logic, lỗi thiếu bao phủ và lỗi sai phản hồi trong chatbot.

6. Yêu cầu về chuẩn năng lực

Mã CLO	Nội dung CLO	% của CLO	Mã PI	Nội dung PI	% của PI
1	Phân tích được yêu cầu và xác định phạm vi kiểm thử	20	1.1	Phân tích được tài liệu yêu cầu phần mềm	10
			1.2	Xác định được phạm vi và mức kiểm thử	5
			1.3	Phân tích được rủi ro và xây dựng chiến lược kiểm thử	5
2	Thiết kế và xây dựng được hệ thống kiểm thử tự động	30	2.1	Xây dựng được kế hoạch kiểm thử Test Plan	10
			2.2	Thiết kế được Test Case và Test Scenario	10
			2.3	Xây dựng được bộ kiểm thử tự động (Automation)	5
			2.4	Đo lường được độ bao phủ kiểm thử (Coverage)	5
3	Thực hiện kiểm thử và phân tích chất lượng phần mềm	35	3.1	Thực hiện kiểm thử hiệu năng (Performance Testing)	15
			3.2	Thực hiện kiểm thử bảo mật cơ bản (Security Testing)	10
			3.3	Phân tích chất lượng mã nguồn và quản lý lỗi)	10
4	Tích hợp kiểm thử và báo cáo chất	15	4.1	Tích hợp kiểm thử vào quy trình CI/CD	10

	lượng		4.2	Lập báo cáo và đánh giá chất lượng phần mềm	5
--	-------	--	-----	---	---

7. Yêu cầu về sản phẩm

a. Mô tả tóm tắt yêu cầu về sản phẩm của đồ án/khoá luận tốt nghiệp

Đề tài tập trung xây dựng bộ test case và đề xuất giải pháp kiểm thử tự động cho chatbot nhà xe Vũ Hán. Trong thực tế, chatbot của nhà xe phải xử lý nhiều nghiệp vụ khác nhau như nhận diện nhu cầu khách hàng, phân loại loại xe quan tâm, hỗ trợ đặt vé limousine hoặc xe khách, giải đáp câu hỏi thường gặp, kiểm tra điểm đón trả, hỗ trợ gửi hàng và chuyển nhân viên trong các trường hợp đặc biệt. Với đặc thù hội thoại nhiều nhánh và nhiều quy tắc nghiệp vụ, việc kiểm thử thủ công toàn bộ các tình huống sẽ tốn nhiều thời gian, khó bao phủ đầy đủ và dễ bỏ sót lỗi logic.

b. Yêu cầu chi tiết

PI	Yêu cầu tối thiểu
1.1	Có bảng phân tích yêu cầu chức năng và phi chức năng phục vụ kiểm thử Phân tích được kiến trúc hệ thống (mô hình phân lớp, API, CSDL)
1.2	Xác định rõ các mức kiểm thử: Unit, Integration, System Xác định các thành phần ưu tiên kiểm thử
1.3	Có bảng phân tích rủi ro Xây dựng chiến lược kiểm thử dựa trên mức độ rủi ro
2.1	Có tài liệu Test Plan đầy đủ: mục tiêu, phạm vi, nguồn lực, tiến độ
2.2	Tối thiểu 50 Test Case Bao phủ $\geq 80\%$ yêu cầu chức năng
2.3	Tối thiểu 60 test tự động (UI hoặc API) Có báo cáo chạy test tự động
2.4	Có báo cáo Coverage Đạt tối thiểu 60% độ bao phủ
3.1	Sử dụng công cụ JMeter hoặc k6 Mô phỏng ≥ 100 người dùng đồng thời

	Phân tích thời gian phản hồi
3.2	Sử dụng công cụ OWASP ZAP Phát hiện và phân tích các lỗ hổng phổ biến
3.3	Sử dụng công cụ phân tích mã nguồn (ví dụ SonarQube) Báo cáo Cyclomatic Complexity Quản lý lỗi bằng Jira hoặc tương đương
4.1	Cấu hình pipeline tự động chạy test khi cập nhật mã nguồn Sinh báo cáo tự động
4.2	Có Test Report đầy đủ Thống kê tỷ lệ Pass/Fail, Defect Density Biểu đồ xu hướng chất lượng

8. Yêu cầu về quyền báo cáo:

Quyền báo cáo phải đảm bảo:

- Do sinh viên tự xây dựng,
- Có cấu trúc nội dung phù hợp yêu cầu và đảm bảo tính logic,
- Tuân thủ yêu cầu về khuôn mẫu và định dạng của trường,
- Không có lỗi chính tả,
- Không sử dụng nội dung vi phạm bản quyền,
- Không vi phạm quy định chống đạo văn của trường.

Cấu trúc của quyền báo cáo:

- Chương 1. Cơ sở lý thuyết
(Những công nghệ chính sử dụng trong đồ án)
- Chương 2. Lập kế hoạch và thiết kế tài liệu kiểm thử
- Chương 3. Thực hiện và báo cáo kết quả kiểm thử
- Tài liệu tham khảo
- Phụ lục (nếu có)

9. Tiến độ thực hiện

ST T	Thời gian	Công việc	Kết quả dự kiến
1	Tuần 1	Phân tích tài liệu yêu cầu phần mềm	Tài liệu Requirement Analysis: mô tả hệ thống, các chức năng chính, use case

2	Tuần 2	Xác định phạm vi và mức kiểm thử	Tài liệu Test Scope & Test Levels (Unit, Integration, System, Acceptance)
3	Tuần 3	Phân tích rủi ro và xây dựng chiến lược kiểm thử	Risk Analysis + Test Strategy Document
4	Tuần 4	Xây dựng kế hoạch kiểm thử	Test Plan hoàn chỉnh (mục tiêu, phạm vi, môi trường, lịch trình)
5	Tuần 5	Thiết kế Test Scenario	Danh sách Test Scenarios cho các chức năng chính
6	Tuần 6	Thiết kế Test Case chi tiết	Bộ Test Case Document (input, expected result, priority)
7	Tuần 7	Xây dựng bộ kiểm thử tự động	Automation Scripts (Selenium / Playwright / Unit Test)
8	Tuần 8	Đo lường độ bao phủ kiểm thử	Test Coverage Report (statement/branch/function coverage)
9	Tuần 9	Kiểm thử hiệu năng và bảo mật cơ bản	Performance Test Report và Security Test Result
10	Tuần 10	Phân tích chất lượng mã nguồn, tích hợp CI/CD và báo cáo	CI/CD Pipeline Demo + Final Test Report + Quality Assessment

Thái Nguyên, ngày 08 tháng 03 năm 2026

LÃNH ĐẠO BỘ MÔN


Nguyễn Thế Vinh

GIẢNG VIÊN HƯỚNG DẪN


Nguyễn Thu Phương